

10. offline フルレース simulation 本体

solar_ws0129-main

2026-07-29

Contents

1 対象	5
2 このファイルを読む理由	5
3 呼び出し関係	5
3.1 どこから呼ばれるか	5
3.2 次に読むと理解が進むファイル	5
3.3 同一リポジトリ内の主要依存	5
4 ソース冒頭の把握	6
4.1 代表コード断片	6
5 主要構造の読み取り	6
5.1 主要クラス・関数	6
5.2 ファイルを上から読んだときの定義順	8
5.3 import 群の詳細	9
6 このファイルを読む前に必要な基礎知識	11
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	12
6.2 名前、代入、型、参照	12
6.3 関数、引数、戻り値、スコープ	12
6.4 module、package、import、console_scripts	13
6.5 例外、try/finally、with、資源解放	13
6.6 class、独自クラス、インスタンス、self、継承	13
6.7 先頭アンダースコア、__init__、名前付けの作法	14
6.8 list、dict、deque、NumPy 配列、pandas DataFrame	14
6.9 CLI、PowerShell、Bash、環境変数、終了コード	14
6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	14
6.11 Cross-Entropy Method を式と実装で理解する	15
6.12 warm start は何を保存し、どう効くか	15
6.13 車両力学、電力収支、効率 map	16
6.14 SoC、内部抵抗、端子電圧、温度、MHE	16
6.15 天候、route、補間、時刻、単位	16

6.16	freshness、filter、guard、fallback、fail-safe	16
6.17	制御、状態、入力、モデル予測制御 MPC	16
6.18	上位 MPC と下位 MPC の役割	17
6.19	CSV/YAML の data contract と検証	17
6.20	単体試験、SILS、replay、観測可能性	17
7	関数・クラスを上から順に解説	18
7.1	L63 関数 load_yaml	18
7.2	L73 関数 sim_log	19
7.3	L77 関数 select_optimized_vector	19
7.4	L115 関数 limit_step_duration_to_distance	21
7.5	L129 関数 terminal_soc_predictions	22
7.6	L145 関数 advance_rate_limiter_to_distance_boundary	23
7.7	L195 関数 snap_execution_stop_speed_kmh	25
7.8	L212 関数 choose_integration_step_seconds	26
7.9	L218 関数 choose_stationary_integration_step_seconds	27
7.10	L230 関数 write_model_snapshot	28
7.11	L278 関数 get_workspace_revision	30
7.12	L331 関数 timestamp_ns	32
7.13	L335 関数 load_stops	32
7.14	L365 関数 load_profile	34
7.15	L373 関数 forecast_distance_column	35
7.16	L383 関数 load_forecast_dataframe	36
7.17	L410 関数 merge_forecast_dataframes	37
7.18	L423 関数 build_forecast_grid_payload	39
7.19	L476 関数 interp_forecast_grid	41
7.20	L523 関数 load_progress_reference_dataframe	43
7.21	L543 関数 ensure_parent_dir	45
7.22	L549 クラス DetailCsvStream	45
7.23	L552 関数 DetailCsvStream.__init__	46
7.24	L559 関数 DetailCsvStream.write	47
7.25	L570 関数 DetailCsvStream.close	48
7.26	L576 関数 DetailCsvStream.__enter__	49
7.27	L579 関数 DetailCsvStream.__exit__	50
7.28	L583 関数 _deep_copy_cfg	50
7.29	L587 関数 _set_nested	51
7.30	L601 関数 apply_overrides	52
7.31	L618 関数 build_config_tag	53
7.32	L633 関数 tag_output_path	54
7.33	L645 関数 apply_profile_cfg_to_args	55
7.34	L798 関数 apply_profile_defaults	60
7.35	L806 関数 resolve_config_path	60

7.36	L820	関数 evaluate_model_validation_gate	61
7.37	L1032	関数 get_profile_val	65
7.38	L1040	関数 parse_utc_arg	66
7.39	L1052	関数 resolve_forecast_mode	67
7.40	L1064	関数 forecast_row_index	68
7.41	L1082	関数 format_metric	70
7.42	L1094	関数 decimate_xy	71
7.43	L1105	関数 build_svg_chart	72
7.44	L1146	関数 flatten_params_for_report	74
7.45	L1149	関数 flatten_params_for_report.visit	75
7.46	L1166	関数 write_simulation_report	76
7.47	L1288	関数 mpc_solve	79
7.48	L1305	関数 mpc_solve.quad_penalty	81
7.49	L1312	関数 mpc_solve.cost	82
7.50	L1402	関数 soc_guard_speed	84
7.51	L1409	関数 soc_guard_speed.z_next_for	86
7.52	L1451	関数 soc_day_guard_speed	87
7.53	L1458	関数 soc_day_guard_speed.z_next_for	89
7.54	L1485	関数 main	90
7.55	L1852	関数 main.integration_step_seconds	93
7.56	L1859	関数 main.forecast_has_coverage	94
7.57	L1868	関数 main.integrate_drive_time_between	95
7.58	L1885	関数 main.reference_value_at_time	96
7.59	L1907	関数 main.reference_distance_at_time	98
7.60	L1958	関数 main.time_to_index	98
7.61	L2095	関数 main.remaining_day_budget	99
7.62	L2117	関数 main.budget_speed_limit	101
7.63	L2136	関数 main.forecast_at_time	102
7.64	L2236	関数 main.apply_stop_poa	105
7.65	L2244	関数 main.reference_speed_command	106
7.66	L2264	関数 main.resolved_headwind_ms	107
7.67	L2274	関数 main.integrate_stationary_duration	108
7.68	L2325	関数 main.step_wait	110
7.69	L2332	関数 main.mpc_solve_distance	111
7.70	L2400	関数 main.mpc_solve_distance.expand_ctrl	115
7.71	L2403	関数 main.mpc_solve_distance.build_balance_seed	116
7.72	L2507	関数 main.mpc_solve_distance.cost	119
7.73	L2528	関数 main.mpc_solve_distance.cost.finish_cost	123
7.74	L2818	関数 main.mpc_solve_distance.save_solver_progress	124
7.75	L3022	関数 main.propagate_execution_step	126
7.76	L3201	関数 main.write_execution_detail	130

7.77 CLI 引数	133
8 処理の流れ	135
9 このファイルの位置づけ	135

1 対象

項目	内容
ファイル	scripts/solar_sim.py
ソース SHA-256	72dbec56f03ec2611bd7c68bfd8ac9df1c3a9b173b9a1b97e754631bce626eac
種別	Python
区分	offline core
役割	profile、forecast、route、maps を使って全レースを逐次再生し、upper/lower 相当の実行を CSV と HTML に落とす。
起動文脈	simulate や historical-simulate で直接呼ばれる同期版の基準実装。

2 このファイルを読む理由

profile、forecast、route、maps を使って全レースを逐次再生し、upper/lower 相当の実行を CSV と HTML に落とす。

- SolarCarModel を直に持つ同期版なので、数理理解の最短入口。
- full summary JSON、detail CSV、upper plan CSV、HTML report を生成する。
- live の mpc_node とかなり同型の距離上位計画ロジックを持つ。

3 呼び出し関係

3.1 どこから呼ばれるか

- scripts/solar_control.sh

3.2 次に読むと理解が進むファイル

- mpc_solarcar/model.py
- mpc_solarcar/upper_horizon.py
- mpc_solarcar/upper_solver.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/model.py
- mpc_solarcar/route_utils.py
- mpc_solarcar/schedule_utils.py
- mpc_solarcar/signal_utils.py
- mpc_solarcar/solar_profile.py
- mpc_solarcar/upper_cost.py
- mpc_solarcar/upper_horizon.py
- mpc_solarcar/upper_policy.py
- mpc_solarcar/upper_solver.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
os.makedirs(parent, exist_ok=True)

class DetailCsvStream:
    """Write the 1 Hz execution trace without retaining the full race in RAM."""

    def __init__(self, path):
        self.path = str(path)
        self._file = None
        self._writer = None
        self.fieldnames = []
        self.row_count = 0

    def write(self, row):
        if self._writer is None:
            ensure_parent_dir(self.path)
            self.fieldnames = list(row.keys())
            opener = gzip.open if self.path.lower().endswith('.gz') else open
            self._file = opener(self.path, 'wt' if opener is gzip.open else 'w', encoding='utf-8', newline='')
            self._writer = csv.DictWriter(self._file, fieldnames=self.fieldnames, extrasaction='ignore')
            self._writer.writeheader()
            self._writer.writerow(row)
```

5 主要構造の読み取り

主要クラスは DetailCsvStream。主要関数は load_yaml, sim_log, select_optimized_vector, limit_step_duration_to_distance, terminal_soc_predictions, advance_rate_limiter_to_distance_boundary, snap_execution_stop_speed_kmh, choose_integration_step_seconds。CLI 引数宣言は 96 件。

5.1 主要クラス・関数

行	種別	名前	補足
549	class	DetailCsvStream	
63	function	load_yaml	args: path
73	function	sim_log	args: message
77	function	select_optimized_vector	args: res, x0
115	function	limit_step_duration_to_distance	args: step_sec, speed_kmh, s0_kmh, race_kmh
129	function	terminal_soc_predictions	args: upper_solve_log
145	function	advance_rate_limiter_to_distance_boundary	args: limiter, target_kmh

195	function	snap_execution_stop_ speed_kmh	args: speed_kmh, target_kmh
212	function	choose_integration_ step_seconds	args: simulation_step_sec, weather_step_sec
218	function	choose_stationary_ integration_step_ seconds	args: simulation_step_sec, weather_step_sec, requested_step_sec
230	function	write_model_snapshot	args: detail_csv, parameters, map_paths
278	function	get_workspace_ revision	args: root_dir
331	function	timestamp_ns	args: value
335	function	load_stops	args: path
365	function	load_profile	args: path
373	function	forecast_distance_ column	args: df
383	function	load_forecast_ dataframe	args: path, tzname
410	function	merge_forecast_ dataframes	args: primary, fallback
423	function	build_forecast_grid_ payload	args: df
476	function	interp_forecast_grid	args: payload, col, t_utc, s_km, default
523	function	load_progress_ reference_dataframe	args: path
543	function	ensure_parent_dir	args: path
552	function	__init__	args: self, path
559	function	write	args: self, row
570	function	close	args: self
576	function	__enter__	args: self
579	function	__exit__	args: self, exc_type, exc, tb
583	function	_deep_copy_cfg	args: cfg
587	function	_set_nested	args: cfg, dotted_key, value
601	function	apply_overrides	args: cfg, overrides
618	function	build_config_tag	args: profile_path, profile_cfg, overrides
633	function	tag_output_path	args: path_value, tag, default_ext
645	function	apply_profile_cfg_to_ args	args: profile_path, profile_cfg, args
798	function	apply_profile_ defaults	args: args
806	function	resolve_config_path	args: profile_path, value
820	function	evaluate_model_ validation_gate	args: cfg, profile_path
1032	function	get_profile_val	args: df, s_km, field, default
1040	function	parse_utc_arg	args: raw_value
1052	function	resolve_forecast_mode	args: mode, df
1064	function	forecast_row_index	args: df, sim_t
1082	function	format_metric	args: value, digits, default
1094	function	decimate_xy	args: xs, ys, max_points
1105	function	build_svg_chart	args: xs, ys

1146	function	flatten_params_for_report	args: cfg
1149	function	visit	args: prefix, value
1166	function	write_simulation_report	args: path, summary, detail_df, params_rows
1288	function	mpc_solve	args: model, data, z0, Tb0, s0_kmh, v0_kmh, v_max_kmh, term_soc_min, w_dv, w_dv_limit, dv_max_kmhps, w_T, w_speed_limit, w_current, speed_profile, soc_target, soc_band, w_soc_target, w_soc_band, schedule, soc_day_end_max, w_soc_day_max, soc_finish_target, soc_finish_tol, w_soc_progress, w_soc_terminal, race_kmh, soc_day_end_target, soc_day_end_tol, w_soc_day_track
1305	function	quad_penalty	args: x, cap
1312	function	cost	args: v
1402	function	soc_guard_speed	args: model, v_kmh, z, Tb, s_kmh, d0, route_profile, mode, soc_guard
1409	function	z_next_for	args: v_kmh_local
1451	function	soc_day_guard_speed	args: model, v_kmh, z, Tb, s_kmh, d0, route_profile, target_soc, tol
1458	function	z_next_for	args: v_kmh_local
1485	function	main	
1852	function	integration_step_seconds	
1859	function	forecast_has_coverage	args: t_utc
1868	function	integrate_drive_time_between	args: t_start_utc, t_end_utc
1885	function	reference_value_at_time	args: t_utc, field
1907	function	reference_distance_at_time	args: t_utc
1958	function	time_to_index	args: dt_utc
2095	function	remaining_day_budget	args: sim_t_local, _k_idx

5.2 ファイルを上から読んだときの定義順

行	内容
23	<code>_numeric_threads</code> に <code>str(os.environ.get('SOLAR_NUMERIC_THREADS', '1'))</code> の結果を代入する。
24	<code>('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'NUMEXPR_NUM_THREADS')</code> を順に走査し、各要素を <code>_thread_variable</code> に入れて処理する。
36	<code>ROOT</code> に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。
37	条件 <code>str(ROOT) not in sys.path</code> を判定し、真なら内部処理を行う。
60	<code>DISTANCE_EPS_KM</code> に <code>1e-06</code> の結果を代入する。
63	関数 <code>load_yaml</code> を定義する。

73	関数 <code>sim_log</code> を定義する。
77	関数 <code>select_optimized_vector</code> を定義する。
115	関数 <code>limit_step_duration_to_distance</code> を定義する。
129	関数 <code>terminal_soc_predictions</code> を定義する。
145	関数 <code>advance_rate_limiter_to_distance_boundary</code> を定義する。
195	関数 <code>snap_execution_stop_speed_kmh</code> を定義する。
212	関数 <code>choose_integration_step_seconds</code> を定義する。
218	関数 <code>choose_stationary_integration_step_seconds</code> を定義する。
230	関数 <code>write_model_snapshot</code> を定義する。
278	関数 <code>get_workspace_revision</code> を定義する。
331	関数 <code>timestamp_ns</code> を定義する。
335	関数 <code>load_stops</code> を定義する。
365	関数 <code>load_profile</code> を定義する。
373	関数 <code>forecast_distance_column</code> を定義する。
383	関数 <code>load_forecast_dataframe</code> を定義する。
410	関数 <code>merge_forecast_dataframes</code> を定義する。
423	関数 <code>build_forecast_grid_payload</code> を定義する。
476	関数 <code>interp_forecast_grid</code> を定義する。
523	関数 <code>load_progress_reference_dataframe</code> を定義する。
543	関数 <code>ensure_parent_dir</code> を定義する。
549	クラス <code>DetailCsvStream</code> を定義する。
583	関数 <code>_deep_copy_cfg</code> を定義する。
587	関数 <code>_set_nested</code> を定義する。
601	関数 <code>apply_overrides</code> を定義する。
618	関数 <code>build_config_tag</code> を定義する。
633	関数 <code>tag_output_path</code> を定義する。
645	関数 <code>apply_profile_cfg_to_args</code> を定義する。
798	関数 <code>apply_profile_defaults</code> を定義する。
806	関数 <code>resolve_config_path</code> を定義する。
820	関数 <code>evaluate_model_validation_gate</code> を定義する。
1032	関数 <code>get_profile_val</code> を定義する。
1040	関数 <code>parse_utc_arg</code> を定義する。
1052	関数 <code>resolve_forecast_mode</code> を定義する。
1064	関数 <code>forecast_row_index</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
2	<code>import argparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L1486, L1534。
3	<code>import csv</code>	CSV の逐次読込・逐次書込を行うため。このファイル内での主な使用位置は L565。
4	<code>import copy</code>	設定辞書や <code>payload</code> を安全に複製するため。このファイル内での主な使用位置は L584。
5	<code>import gzip</code>	<code>detail CSV</code> などの圧縮出力を行うため。このファイル内での主な使用位置は L563, L564。

6	<code>importhashlib</code>	snapshot ID や入力資産の <code>digest</code> を作るため。このファイル内での主な使用位置は L238, L252, L624。
7	<code>importhtml</code>	HTML report の文字列を安全に埋め込むため。このファイル内での主な使用位置は L1131, L1136, L1206, L1210, L1214, L1222, L1253, L1254。
8	<code>importjson</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L253, L1206, L2830, L2950, L4001, L4005。
9	<code>importmath</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L134, L138, L885, L886, L887, L888, L889, L890, ...。
10	<code>importtime</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L2055, L3713。
11	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L23, L30, L544, L546, L626, L639, L739, L742, ...。
12	<code>importsubprocess</code>	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L286, L299, L308, L317。
13	<code>importsys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L37, L38, L4028。
14	<code>importtraceback</code>	例外時に crash log を残すため。このファイル内での主な使用位置は L4025。
15	<code>fromcollectionsimportdeque</code>	固定長の時系列や遅延キューを効率よく保持するため。このファイル内での主な使用位置は L2075。
16	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L36, L235, L270, L278, L810, L813, L835, L901, ...。
17	<code>fromzoneinfoimportZoneInfo</code>	zoneinfo から ZoneInfo を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L393, L2030, L2032, L3736, L3756。
18	<code>fromdatetimeimportdatetime, timedelta, timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L476, L626, L628, L1046, L1048, L1049, L1064, L1364, ...。
32	<code>importnumpyasnp</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L78, L93, L100, L135, L433, L470, L484, L486, ...。
33	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L332, L369, L373, L374, L383, L384, L387, L410, ...。
34	<code>importyaml</code>	profile、stop、schedule、summary YAML を読み書きするため。このファイル内での主な使用位置は L68, L271, L612, L623, L3785。
40	<code>frommpc_solarcar.modelimportSolarCarModel, Params</code>	車体物理・電気モデル本体から SolarCarModel, Params を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/model.py。このファイル内での主な使用位置は L1689, L1703。

41	<code>frommpc_solarcar.route_utilsimportaverage_profile, interpolate_profile</code>	route_utils.py から average_profile, interpolate_profile を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/route_utils.py。このファイル内での主な使用位置は L1035, L2460, L2656。
42	<code>frommpc_solarcar.schedule_utilsimportDriveSchedule</code>	schedule_utils.py から DriveSchedule を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/schedule_utils.py。このファイル内での主な使用位置は L1664。
43	<code>frommpc_solarcar.signal_utilsimportSmoothRateLimiter</code>	signal_utils.py から SmoothRateLimiter を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/signal_utils.py。このファイル内での主な使用位置は L2078。
44	<code>frommpc_solarcar.solar_profileimportget_path, get_section, load_profileasload_workflow_profile</code>	profile YAML 読込と検証から get_path, get_section, load_profile as load_workflow_profile を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_profile.py。このファイル内での主な使用位置は L649, L650, L651, L659, L660, L661, L662, L663, ...。
45	<code>frommpc_solarcar.upper_costimportactive_upper_cost_terms, load_upper_cost_config, quad_penalty, upper_stage_cost, upper_terminal_cost</code>	上位 MPC 目的関数から active_upper_cost_terms, load_upper_cost_config, quad_penalty, upper_stage_cost, upper_terminal_cost を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_cost.py。このファイル内での主な使用位置は L1350, L1360, L1362, L1366, L1370, L1376, L1378, L1379, ...。
52	<code>frommpc_solarcar.upper_horizonimportbuild_upper_distance_horizon, plan_segment_index</code>	上位 MPC 距離メッシュ生成から build_upper_distance_horizon, plan_segment_index を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_horizon.py。このファイル内での主な使用位置は L2341, L3431, L3470, L3590。
53	<code>frommpc_solarcar.upper_policyimportabsolute_control_distances, shift_upper_policy_warm_start</code>	上位速度計画の補間と warm start から absolute_control_distances, shift_upper_policy_warm_start を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_policy.py。このファイル内での主な使用位置は L2366, L2374。
57	<code>frommpc_solarcar.upper_solverimporthybrid_bounded_minimize</code>	上位探索ソルバから hybrid_bounded_minimize を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_solver.py。このファイル内での主な使用位置は L2873。
1394	<code>fromscipy.optimizeimportminimize</code>	目的関数と制約・bounds に基づく連続数値最適化を解くため。このファイル内での主な使用位置は L1396。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport -> main()を呼び

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体を作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、`__init__`、名前付けの作法

‘`self.step_solar`’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘`__init__`’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘`_`’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘`_base_csv`’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’ となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.9 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた `sampling` と局所最適化を組み合わせる大域最適化法である。有限 `Cartesian grid` の全列挙は、その `grid` 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- SciPy 公式: [scipy.optimize.shgo](#)

6.11 Cross-Entropy Method を式と実装で理解する

CEM は候補を生成する確率分布を持ち、良かった `elite` 候補から分布を更新する反復的な確率最適化である。このリポジトリの `upper_solver.py` は各制御点速度を独立正規分布で生成し、上下限へ `clip` する。

$$u_i^{(j)} \sim \mathcal{N}(\mu_i^{(g)}, (\sigma_i^{(g)})^2), \quad u_i^{(j)} \leftarrow \text{clip}(u_i^{(j)}, l_i, h_i)$$
$$\mu_i^{(g+1)} = \frac{1}{K} \sum_{j \in \mathcal{E}_g} u_i^{(j)}, \quad \sigma_i^{(g+1)} = \max \left(\text{Std}_{j \in \mathcal{E}_g} u_i^{(j)}, 0.05(h_i - l_i) \right)$$

ここで `E_g` は `cost` が小さい上位 `K` 候補である。平均は良い領域へ移り、標準偏差は探索幅を表す。標準偏差の下限は探索が完全に潰れることを避ける。

現行 `hybrid_bounded_minimize` は、`deterministic seed` を評価し、上位候補を `L-BFGS-B` で局所 `refine` し、設定と `seed` 間不一致に応じて CEM を実行し、最後に再度局所 `refine` する。したがって CEM 単独ではなく `hybrid solver` である。

CEM で落とした候補を永久保存しないこと自体は通常的最適化として自然だが、`off-nominal` 状態からの再利用には別の `policy library` 設計が必要である。状態を無制限に全組合せ保存する代わりに、`SoC`、進捗、時刻、予報誤差、停止状態などの `scenario` を設計し、近傍 `policy` を検索して MPC で再最適化する。

根拠資料

- Rubinstein and Kroese: [The Cross-Entropy Method](#)
- SciPy 公式: [scipy.optimize.minimize](#)

6.12 warm start は何を保存し、どう効くか

warm start は前回または `offline` 探索で得た入力系列を、次の最適化の初期候補として渡すことである。答えを固定するのではなく、探索開始点と候補 `library` の一員を与える。

$$u_i^{0,\text{new}} = \text{interp} \left(s_i^{\text{new}}; s_j^{\text{old}}, u_j^{*,\text{old}} \right)$$

距離基準計画では、現在より後ろの旧制御点を捨て、新しい絶対距離制御点へ補間して `shift` する。初期状態や天候が変われば `cost` 評価は新条件で行われるので、`warm start` が不適切でも他 `seed`、CEM、安全 `fallback` が補う設計が必要である。

warm start の効き方は、局所法なら収束先と反復数、CEM なら初期 `mean` または候補 `pool`、`receding horizon` なら前回解の時間・距離 `shift` として現れる。どの位置に渡しているかを呼出引数まで追う。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- Rubinstein and Kroese: [The Cross-Entropy Method](#)

6.13 車両力学、電力収支、効率 map

$$F_{\text{aero}} = \frac{1}{2} \rho C_d A (v + v_{\text{wind}})^2, \quad F_{\text{roll}} \approx mg C_{rr} \cos \theta, \quad F_{\text{grade}} = mg \sin \theta$$

車輪機械 power は概ね ‘ $P_{\text{mech}} = F_{\text{total}} \cdot v + P_{\text{inertia}}$ ’ で、駆動時と回生時で異なる効率 map を通して DC 側 power へ変換する。map は速度・torque などを軸にした実測または同定 table であり、範囲外の補間・clip 規則もモデルの一部である。

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

符号規約は必ずコードで確認する。本プロジェクトでは正の pack power を放電側として SoC を減らす処理が中心であり、発電と回生は pack 負荷を下げる方向に働く。

6.14 SoC、内部抵抗、端子電圧、温度、MHE

$$V = V_{\text{oc}}(z, T) - I R_{\text{total}}(z, T, I), \quad z_{k+1} = z_k - \frac{\eta(I, T) I \Delta t}{Q_{\text{eff}}}$$

SoC は直接完全には観測できないため、電流積算、OCV、端子電圧、温度、容量、効率を組み合わせる。内部抵抗は SoC、温度、電流方向で変わり、発熱と電圧制約の両方へ影響する。

MHE は有限時間窓の状態と観測誤差をまとめて最適化し、SoC や温度を推定する。古い測定や欠損値を同じ重みで使うと推定が壊れるため、timestamp freshness と観測可用性を入口で確認する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.15 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.16 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.17 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’ と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

6.18 上位 MPC と下位 MPC の役割

上位層は長い距離または時間を見て、エネルギー、到着、停止、速度制限、終端 SoC を考えた速度計画を出す。下位層は短い周期で実測速度を見て、上位速度へ追従する駆動・回生入力を求める。

予報・route・SoC -> 上位MPC -> 将来速度列 -> 下位MPC -> throttle/回生/drive mode -> driverまたはvehicle -> 新しいtelemetry

上位解が遅い間も下位出力を止めないこと、古い計画を安全に保持すること、上位と下位で単位と時刻基準を一致させることが実装上重要である。

6.19 CSV/YAML の data contract と検証

data contract は列名、型、単位、timezone、欠損可否、並び順、重複、許容範囲、先頭行、encoding を事前に決めた仕様である。単に CSV として読めることは、モデル入力として正しいことを意味しない。

同定用実測、map、route、forecast、stop、schedule はそれぞれ grain が異なる。生成時に schema validation、物理範囲、時間単調性、route 範囲、coverage を検査し、検査結果を artifact として残す。

学習用データと独立検証データを分離し、RMSE だけでなく bias、時系列残差、energy 積算誤差、終端 SoC、温度・電圧制約、外挿領域を評価する。

6.20 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 公式: Executors](#)

7 関数・クラスを上から順に解説

7.1 L63 関数 load_yaml

- 行範囲: L63–L70
- 所属: module 直下
- 定義: load_yaml(path)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: open, safe_load
- 戻り値の要点: {} / yaml.safe_load(f) or {} / {}
- 主なローカル代入名: f
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 not path を判定し、真なら内部処理を行う。
2. {} を返す。
3. 例外処理を伴う try ブロックを実行する。
4. with 文で open(path, 'r', encoding='utf-8') を管理しながら処理する。
5. yaml.safe_load(f) or {} を返す。
6. Exception を捕捉した場合:
7. {} を返す。

代表コード断片

```
def load_yaml(path):
    if not path:
        return {}
    try:
        with open(path, 'r', encoding='utf-8') as f:
            return yaml.safe_load(f) or {}
    except Exception:
        return {}
```

7.2 L73 関数 `sim_log`

- 行範囲: L73–L74
- 所属: module 直下
- 定義: `sim_log(message:str)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `print`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `print(...)` を実行する。

代表コード断片

```
def sim_log(message: str):  
    print(f"[solar_sim] {message}", flush=True)
```

7.3 L77 関数 `select_optimized_vector`

- 行範囲: L77–L112
- 所属: module 直下
- 定義: `select_optimized_vector(res,x0,*,label:str)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `all`, `asarray`, `bool`, `getattr`, `isfinite`, `sim_log`, `str`, `strip`
- 戻り値の要点: `x_arr / x0_arr / x0_arr / x0_arr`
- 主なローカル代入名: `message`, `raw_x`, `status`, `success`, `x0_arr`, `x_arr`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、`try` 1

この定義を読むための Python 構文

- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. x0_arr に np.asarray(x0, dtype=float) の結果を代入する。
2. 条件 res is None を判定し、真なら内部処理を行う。
3. sim_log(...) を実行する。
4. x0_arr を返す。
5. success に bool(getattr(res, 'success', False)) の結果を代入する。
6. status に getattr(res, 'status', None) の結果を代入する。
7. message に str(getattr(res, 'message', '')) or '').strip() の結果を代入する。
8. raw_x に getattr(res, 'x', None) の結果を代入する。
9. 条件 raw_x is None を判定し、真なら内部処理を行う。
10. sim_log(...) を実行する。
11. x0_arr を返す。
12. 例外処理を伴う try ブロックを実行する。
13. x_arr に np.asarray(raw_x, dtype=float) の結果を代入する。
14. Exception を捕捉した場合:
15. sim_log(...) を実行する。
16. x0_arr を返す。
17. 条件 x_arr.shape != x0_arr.shape or not np.all(np.isfinite(x_arr)) を判定し、真なら内部処理を行う。
18. sim_log(...) を実行する。
19. x0_arr を返す。
20. 条件 not success を判定し、真なら内部処理を行う。
21. sim_log(...) を実行する。
22. x0_arr を返す。
23. x_arr を返す。

代表コード断片

```
def select_optimized_vector(res, x0, *, label: str):
    x0_arr = np.asarray(x0, dtype=float)
    if res is None:
        sim_log(f"{label} fallback: optimizer returned no result; using the initial guess.")
        return x0_arr
    success = bool(getattr(res, 'success', False))
    status = getattr(res, 'status', None)
    message = str(getattr(res, 'message', '')) or '').strip()
    raw_x = getattr(res, 'x', None)
    if raw_x is None:
        sim_log(
```

```

        f"{label} fallback: optimizer produced no candidate "
        f"(success={success} status={status} message={message!r}); using the initial
guess."
    )
    return x0_arr
try:
    x_arr = np.asarray(raw_x, dtype=float)
except Exception:
    sim_log(
        f"{label} fallback: optimizer candidate could not be converted to float "
        f"(success={success} status={status} message={message!r}); using the initial
guess."
    )
    return x0_arr
if x_arr.shape != x0_arr.shape or not np.all(np.isfinite(x_arr)):
    sim_log(
        f"{label} fallback: optimizer candidate was non-finite or mismatched "
        f"(success={success} status={status} message={message!r}); using the initial
guess."
    )
    return x0_arr
if not success:
    sim_log(
        f"{label} fallback: optimizer reported failure "
        f"(status={status} message={message!r}); using the initial guess."
    )
    return x0_arr
...

```

7.4 L115 関数 `limit_step_duration_to_distance`

- 行範囲: L115–L126
- 所属: module 直下
- 定義: `limit_step_duration_to_distance(step_sec:float,speed_kmh:float,s0_km:float,race_km:float|None)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`, `min`
- 戻り値の要点: `min(step_sec, max_step_sec) / step_sec / 0.0 / step_sec`
- 主なローカル代入名: `max_step_sec`, `remaining_km`, `speed_kmh`, `step_sec`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `step_sec` に `max(0.0, float(step_sec))` の結果を代入する。
2. 条件 `race_km is None` を判定し、真なら内部処理を行う。
3. `step_sec` を返す。
4. `remaining_km` に `max(0.0, float(race_km) - float(s0_km))` の結果を代入する。
5. 条件 `remaining_km <= DISTANCE_EPS_KM` を判定し、真なら内部処理を行う。
6. `0.0` を返す。
7. `speed_kmh` に `max(0.0, float(speed_kmh))` の結果を代入する。
8. 条件 `speed_kmh <= 1e-09` を判定し、真なら内部処理を行う。
9. `step_sec` を返す。
10. `max_step_sec` に `remaining_km * 3600.0 / speed_kmh` の結果を代入する。
11. `min(step_sec, max_step_sec)` を返す。

代表コード断片

```
def limit_step_duration_to_distance(step_sec: float, speed_kmh: float, s0_km: float, race_km: float | None) -> float:
    step_sec = max(0.0, float(step_sec))
    if race_km is None:
        return step_sec
    remaining_km = max(0.0, float(race_km) - float(s0_km))
    if remaining_km <= DISTANCE_EPS_KM:
        return 0.0
    speed_kmh = max(0.0, float(speed_kmh))
    if speed_kmh <= 1.0e-9:
        return step_sec
    max_step_sec = remaining_km * 3600.0 / speed_kmh
    return min(step_sec, max_step_sec)
```

7.5 L129 関数 `terminal_soc_predictions`

- 行範囲: L129–L142
- 所属: module 直下
- 定義: `terminal_soc_predictions(upper_solve_log: list[dict]) -> tuple[float, float]`
- docstring: Return initial full-horizon and latest nontrivial terminal-SoC predictions.
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `dict`, `float`, `get`, `int`, `isfinite`
- 戻り値の要点: `(float(initial_soc), float(latest_nontrivial_soc)) / (math.nan, math.nan)`
- 主なローカル代入名: `candidates`, `initial_soc`, `latest_nontrivial_soc`, `nontrivial`, `prediction`, `row`, `steps`, `terminal_soc`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `candidates` に `[]` の結果を代入する。
2. `upper_solve_log` を順に走査し、各要素を `row` に入れて処理する。
3. `prediction` に `dict(row.get('selected_prediction', {})) or {}` の結果を代入する。
4. `terminal_soc` に `float(prediction.get('terminal_soc', math.nan))` の結果を代入する。
5. 条件 `np.isfinite(terminal_soc)` を判定し、真なら内部処理を行う。
6. `candidates.append(...)` を実行する。
7. 条件 `not candidates` を判定し、真なら内部処理を行う。
8. `(math.nan, math.nan)` を返す。
9. `initial_soc` に `candidates[0][1]` の結果を代入する。
10. `nontrivial` に `[terminal_soc for steps, terminal_soc in candidates if steps > 1]` の結果を代入する。
11. `latest_nontrivial_soc` に `nontrivial[-1] if nontrivial else candidates[-1][1]` の結果を代入する。
12. `(float(initial_soc), float(latest_nontrivial_soc))` を返す。

代表コード断片

```
def terminal_soc_predictions(upper_solve_log: list[dict]) -> tuple[float, float]:
    """Return initial full-horizon and latest nontrivial terminal-SoC predictions."""
    candidates = []
    for row in upper_solve_log:
        prediction = dict(row.get('selected_prediction', {})) or {}
        terminal_soc = float(prediction.get('terminal_soc', math.nan))
        if np.isfinite(terminal_soc):
            candidates.append((int(row.get('prediction_steps', 0) or 0), terminal_soc))
    if not candidates:
        return math.nan, math.nan
    initial_soc = candidates[0][1]
    nontrivial = [terminal_soc for steps, terminal_soc in candidates if steps > 1]
    latest_nontrivial_soc = nontrivial[-1] if nontrivial else candidates[-1][1]
    return float(initial_soc), float(latest_nontrivial_soc)
```

7.6 L145 関数 `advance_rate_limiter_to_distance_boundary`

- 行範囲: L145–L192
- 所属: `module` 直下
- 定 義: `advance_rate_limiter_to_distance_boundary(limiter, target_kmh: float, *, start_time_sec: float, step_sec: float, s0_km: float, distance_limit_km: float | None) -> tuple[float, float]`
- docstring: Advance a limiter by the actual substep, including a short boundary remainder.
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `float`, `limit_step_duration_to_distance`, `max`, `range`, `update`

- 戻り値の要点: `(speed_kmh, actual_dt) / (float(limiter.value), 0.0) / (speed_kmh, next_dt)`
- 主なローカル代入名: `_`, `actual_dt`, `initial_value`, `limited_dt`, `next_dt`, `requested_dt`, `speed_kmh`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `requested_dt` に `max(0.0, float(step_sec))` の結果を代入する。
2. 条件 `requested_dt <= 0.0` を判定し、真なら内部処理を行う。
3. `(float(limiter.value), 0.0)` を返す。
4. `initial_value` に `float(limiter.value)` の結果を代入する。
5. `actual_dt` に `requested_dt` の結果を代入する。
6. `speed_kmh` に `initial_value` の結果を代入する。
7. `range(8)` を順に走査し、各要素を `_` に入れて処理する。
8. `limiter.value` に `initial_value` の結果を代入する。
9. `limiter.last_time` に `float(start_time_sec)` の結果を代入する。
10. `speed_kmh` に `float(limiter.update(target_kmh, now=float(start_time_sec) + actual_dt))` の結果を代入する。
11. `limited_dt` に `limit_step_duration_to_distance(requested_dt, speed_kmh, s0_km, distance_limit_km)` の結果を代入する。
12. `next_dt` に `max(0.0, float(limited_dt))` の結果を代入する。
13. 条件 `abs(next_dt - actual_dt) <= 1e-09` を判定し、真なら内部処理を行う。
14. `(speed_kmh, next_dt)` を返す。
15. `actual_dt` に `next_dt` の結果を代入する。
16. `limiter.value` に `initial_value` の結果を代入する。
17. `limiter.last_time` に `float(start_time_sec)` の結果を代入する。
18. `speed_kmh` に `float(limiter.update(target_kmh, now=float(start_time_sec) + actual_dt))` の結果を代入する。
19. `(speed_kmh, actual_dt)` を返す。

代表コード断片

```
def advance_rate_limiter_to_distance_boundary(
    limiter,
    target_kmh: float,
    *,
    start_time_sec: float,
    step_sec: float,
    s0_kmh: float,
    distance_limit_kmh: float | None,
) -> tuple[float, float]:
    """Advance a limiter by the actual substep, including a short boundary remainder."""
    requested_dt = max(0.0, float(step_sec))
    if requested_dt <= 0.0:
        return float(limiter.value), 0.0

    initial_value = float(limiter.value)
    actual_dt = requested_dt
    speed_kmh = initial_value
    for _ in range(8):
        limiter.value = initial_value
        # The stored value is the state at the beginning of this substep. Do
        # not let a stale limiter timestamp add time to a short boundary step.
        limiter.last_time = float(start_time_sec)
        speed_kmh = float(
            limiter.update(
                target_kmh,
                now=float(start_time_sec) + actual_dt,
            )
        )
        limited_dt = limit_step_duration_to_distance(
            requested_dt,
            speed_kmh,
            s0_kmh,
            distance_limit_kmh,
        )
        next_dt = max(0.0, float(limited_dt))
    ...
```

7.7 L195 関数 snap_execution_stop_speed_kmh

- 行範囲: L195–L209
- 所属: module 直下
- 定義: `snap_execution_stop_speed_kmh(speed_kmh:float,target_kmh:float,*,stop_requested:bool,deadband_kmh:float,quantize_step_kmh:float)->tuple[float,bool]`
- docstring: Remove the final quantized crawl once a stop command has nearly settled.
- このブロックが直接呼ぶ主な関数・メソッド: `bool`, `float`, `max`
- 戻り値の要点: `(speed, False) / (0.0, True)`
- 主なローカル代入名: `speed`, `target`, `threshold`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. speed に `max(0.0, float(speed_kmh))` の結果を代入する。
2. target に `max(0.0, float(target_kmh))` の結果を代入する。
3. threshold に `max(1e-06, float(deadband_kmh), float(quantize_step_kmh))` の結果を代入する。
4. 条件 `bool(stop_requested) and target <= 1e-09 and (speed <= threshold + 1e-09)` を判定し、真なら内部処理を行う。
5. `(0.0, True)` を返す。
6. `(speed, False)` を返す。

代表コード断片

```
def snap_execution_stop_speed_kmh(
    speed_kmh: float,
    target_kmh: float,
    *,
    stop_requested: bool,
    deadband_kmh: float,
    quantize_step_kmh: float,
) -> tuple[float, bool]:
    """Remove the final quantized crawl once a stop command has nearly settled."""
    speed = max(0.0, float(speed_kmh))
    target = max(0.0, float(target_kmh))
    threshold = max(1.0e-6, float(deadband_kmh), float(quantize_step_kmh))
    if bool(stop_requested) and target <= 1.0e-9 and speed <= threshold + 1.0e-9:
        return 0.0, True
    return speed, False
```

7.8 L212 関数 `choose_integration_step_seconds`

- 行範囲: L212–L215
- 所属: module 直下
- 定義: `choose_integration_step_seconds(simulation_step_sec:float,weather_step_sec:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`, `min`
- 戻り値の要点: `max(60.0, min(simulation_step, weather_step))`
- 主なローカル代入名: `simulation_step`, `weather_step`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. simulation_step に float(max(simulation_step_sec, 1.0)) の結果を代入する。
2. weather_step に float(max(weather_step_sec, 1.0)) の結果を代入する。
3. max(60.0, min(simulation_step, weather_step)) を返す。

代表コード断片

```
def choose_integration_step_seconds(simulation_step_sec: float, weather_step_sec: float) -> float:
    simulation_step = float(max(simulation_step_sec, 1.0))
    weather_step = float(max(weather_step_sec, 1.0))
    return max(60.0, min(simulation_step, weather_step))
```

7.9 L218 関数 choose_stationary_integration_step_seconds

- 行範囲: L218–L227
- 所属: module 直下
- 定 義: choose_stationary_integration_step_seconds(simulation_step_sec:float, weather_step_sec:float,requested_step_sec:float=60.0)->float
- docstring: Use a fine state step after weather interpolation during long stops.
- このブロックが直接呼ぶ主な関数・メソッド: float, max, min
- 戻り値の要点: min(requested, simulation_step, weather_step)
- 主なローカル代入名: requested, simulation_step, weather_step
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. requested に max(1.0, float(requested_step_sec)) の結果を代入する。
2. simulation_step に max(1.0, float(simulation_step_sec)) の結果を代入する。
3. weather_step に max(1.0, float(weather_step_sec)) の結果を代入する。
4. min(requested, simulation_step, weather_step) を返す。

代表コード断片

```
def choose_stationary_integration_step_seconds(
    simulation_step_sec: float,
    weather_step_sec: float,
    requested_step_sec: float = 60.0,
) -> float:
    """Use a fine state step after weather interpolation during long stops."""
    requested = max(1.0, float(requested_step_sec))
    simulation_step = max(1.0, float(simulation_step_sec))
    weather_step = max(1.0, float(weather_step_sec))
    return min(requested, simulation_step, weather_step)
```

7.10 L230 関数 write_model_snapshot

- 行範囲: L230–L275
- 所属: module 直下
- 定義: `write_model_snapshot(detail_csv:str, parameters:dict, map_paths:dict) -> tuple[str, str]`
- docstring: Write immutable model provenance once instead of repeating map paths at 1 Hz.
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `bool`, `dict`, `dumps`, `encode`, `endswith`, `ensure_parent_dir`, `hexdigest`, `int`, `is_file`, `items`, `iter`
- 戻り値の要点: `(snapshot_id, output)`
- 主なローカル代入名: `block`, `canonical`, `digest`, `entry`, `key`, `maps`, `output`, `path`, `payload`, `raw`, `raw_base`, `raw_path`, `snapshot_id`, `stream`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 2、`try` 0

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `maps` に `{}` の結果を代入する。
2. `sorted((map_paths or {}).items())` を順に走査し、各要素を `(key, raw_path)` に入れて処理する。
3. `raw` に `str(raw_path or '')` の結果を代入する。
4. `path` に `Path(raw)` の結果を代入する。
5. `entry` に `{'path': raw, 'exists': bool(path.is_file())}` の結果を代入する。
6. 条件 `path.is_file()` を判定し、真なら内部処理を行う。

7. digest に hashlib.sha256() の結果を代入する。
8. with 文で path.open('rb') を管理しながら処理する。
9. iter(lambda: stream.read(1024 * 1024), b'') を順に走査し、各要素を block に入れて処理する。
10. entry['sha256'] に digest.hexdigest() の結果を代入する。
11. entry['size_bytes'] に int(path.stat().st_size) の結果を代入する。
12. 上の条件が偽の場合:
13. entry['sha256'] に "" の結果を代入する。
14. entry['size_bytes'] に 0 の結果を代入する。
15. maps[str(key)] に entry の結果を代入する。
16. canonical に {'parameters': dict(sorted((parameters or {}).items())), 'maps': maps} の結果を代入する。
17. snapshot_id に hashlib.sha256(json.dumps(canonical, sort_keys=True, ensure_ascii=False, default=str).encode('utf-8')).hexdigest()[:16] の結果を代入する。
18. raw_base に str(detail_csv) の結果を代入する。
19. 条件 raw_base.lower().endswith('.gz') を判定し、真なら内部処理を行う。
20. raw_base に raw_base[:-3] の結果を代入する。
21. 条件 raw_base.lower().endswith('.csv') を判定し、真なら内部処理を行う。
22. raw_base に raw_base[:-4] の結果を代入する。
23. output に raw_base + '_model_snapshot.yaml' の結果を代入する。
24. payload に {'schema_version': 1, 'model_snapshot_id': snapshot_id, 'detail_csv': str(detail_csv), 'parameters_repeated_in_detail_csv': True, 'maps_referenced_by_snapshot_id': True, **canonical} の結果を代入する。
25. ensure_parent_dir(...) を実行する。
26. Path(output).write_text(...) を実行する。
27. (snapshot_id, output) を返す。

代表コード断片

```
def write_model_snapshot(detail_csv: str, parameters: dict, map_paths: dict) -> tuple[str, str]:
    """Write immutable model provenance once instead of repeating map paths at 1 Hz."""
    maps = {}
    for key, raw_path in sorted((map_paths or {}).items()):
        raw = str(raw_path or "")
        path = Path(raw)
        entry = {"path": raw, "exists": bool(path.is_file())}
        if path.is_file():
            digest = hashlib.sha256()
            with path.open("rb") as stream:
                for block in iter(lambda: stream.read(1024 * 1024), b''):
                    digest.update(block)
            entry["sha256"] = digest.hexdigest()
            entry["size_bytes"] = int(path.stat().st_size)
        else:
            entry["sha256"] = ""
            entry["size_bytes"] = 0
        maps[str(key)] = entry
```

```

canonical = {
    "parameters": dict(sorted((parameters or {}).items())),
    "maps": maps,
}
snapshot_id = hashlib.sha256(
    json.dumps(canonical, sort_keys=True, ensure_ascii=False, default=str).encode("utf-8")
).hexdigest()[:16]
raw_base = str(detail_csv)
if raw_base.lower().endswith(".gz"):
    raw_base = raw_base[:-3]
if raw_base.lower().endswith(".csv"):
    raw_base = raw_base[:-4]
output = raw_base + "_model_snapshot.yaml"
payload = {
    "schema_version": 1,
    "model_snapshot_id": snapshot_id,
    "detail_csv": str(detail_csv),
}
...

```

7.11 L278 関数 `get_workspace_revision`

- 行範囲: L278–L328
- 所属: module 直下
- 定義: `get_workspace_revision(root_dir:Path)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool`, `lower`, `run`, `strip`
- 戻り値の要点: `info / info / info / info`
- 主なローカル代入名: `branch`, `dirty`, `head`, `info`, `inside`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、try 2

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `info` に `{'git_available': False, 'git_head': '', 'git_branch': '', 'git_dirty': None}` の結果を代入する。
2. 例外処理を伴う try ブロックを実行する。
3. `inside` に `subprocess.run(['git', 'rev-parse', '--is-inside-work-tree'], cwd=root_dir, capture_output=True, text=True, check=False)` の結果を代入する。
4. Exception を捕捉した場合:

5. info を返す。
6. 条件 `inside.returncode != 0 or inside.stdout.strip().lower() != 'true'` を判定し、真なら内部処理を行う。
7. info を返す。
8. `info['git_available']` に True の結果を代入する。
9. 例外処理を伴う try ブロックを実行する。
10. head に `subprocess.run(['git', 'rev-parse', 'HEAD'], cwd=root_dir, capture_output=True, text=True, check=False)` の結果を代入する。
11. 条件 `head.returncode == 0` を判定し、真なら内部処理を行う。
12. `info['git_head']` に `head.stdout.strip()` の結果を代入する。
13. branch に `subprocess.run(['git', 'rev-parse', '--abbrev-ref', 'HEAD'], cwd=root_dir, capture_output=True, text=True, check=False)` の結果を代入する。
14. 条件 `branch.returncode == 0` を判定し、真なら内部処理を行う。
15. `info['git_branch']` に `branch.stdout.strip()` の結果を代入する。
16. dirty に `subprocess.run(['git', 'status', '--short'], cwd=root_dir, capture_output=True, text=True, check=False)` の結果を代入する。
17. 条件 `dirty.returncode == 0` を判定し、真なら内部処理を行う。
18. `info['git_dirty']` に `bool(dirty.stdout.strip())` の結果を代入する。
19. Exception を捕捉した場合:
20. info を返す。
21. info を返す。

代表コード断片

```
def get_workspace_revision(root_dir: Path) -> dict:
    info = {
        'git_available': False,
        'git_head': '',
        'git_branch': '',
        'git_dirty': None,
    }
    try:
        inside = subprocess.run(
            ['git', 'rev-parse', '--is-inside-work-tree'],
            cwd=root_dir,
            capture_output=True,
            text=True,
            check=False,
        )
    except Exception:
        return info
    if inside.returncode != 0 or inside.stdout.strip().lower() != 'true':
        return info
    info['git_available'] = True
    try:
        head = subprocess.run(
            ['git', 'rev-parse', 'HEAD'],
            cwd=root_dir,
            capture_output=True,
            text=True,
```

```

        check=False,
    )
    if head.returncode == 0:
        info['git_head'] = head.stdout.strip()
    branch = subprocess.run(
        ['git', 'rev-parse', '--abbrev-ref', 'HEAD'],
        cwd=root_dir,
        capture_output=True,
        text=True,
    )
...

```

7.12 L331 関数 timestamp_ns

- 行範囲: L331–L332
- 所属: module 直下
- 定義: `timestamp_ns(value)->int`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `int`
- 戻り値の要点: `int(pd.Timestamp(value).value)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `int(pd.Timestamp(value).value)` を返す。

代表コード断片

```

def timestamp_ns(value) -> int:
    return int(pd.Timestamp(value).value)

```

7.13 L335 関数 load_stops

- 行範囲: L335–L362
- 所属: module 直下
- 定義: `load_stops(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `float`, `get`, `isinstance`, `load_yaml`, `max`, `str`

- 戻り値の要点: stops
- 主なローカル代入名: cfg, dwell_sec, item, raw_stops, s_km, stops
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 3

この定義を読むための Python 構文

- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. cfg に load_yaml(path) の結果を代入する。
2. raw_stops に cfg.get('stops', []) if isinstance(cfg, dict) else [] の結果を代入する。
3. stops に [] の結果を代入する。
4. raw_stops or [] を順に走査し、各要素を item に入れて処理する。
5. 条件 isinstance(item, dict) を判定し、真なら内部処理を行う。
6. 例外処理を伴う try ブロックを実行する。
7. s_km に float(item.get('s_km', item.get('dist_km', 0.0))) の結果を代入する。
8. Exception を捕捉した場合:
9. Continue 文を実行する。
10. 例外処理を伴う try ブロックを実行する。
11. dwell_sec に float(item.get('dwell_sec', item.get('duration_sec', 1800.0))) の結果を代入する。
12. Exception を捕捉した場合:
13. dwell_sec に 1800.0 の結果を代入する。
14. stops.append(...) を実行する。
15. 上の条件が偽の場合:
16. 例外処理を伴う try ブロックを実行する。
17. s_km に float(item) の結果を代入する。
18. Exception を捕捉した場合:
19. Continue 文を実行する。
20. stops.append(...) を実行する。
21. stops を返す。

代表コード断片

```
def load_stops(path):
    cfg = load_yaml(path)
    raw_stops = cfg.get('stops', []) if isinstance(cfg, dict) else []
    stops = []
    for item in raw_stops or []:
        if isinstance(item, dict):
            try:
                s_km = float(item.get('s_km', item.get('dist_km', 0.0)))
            except Exception:
                continue
            try:
                dwell_sec = float(item.get('dwell_sec', item.get('duration_sec', 1800.0)))
            except Exception:
                dwell_sec = 1800.0
            stops.append({
                's_km': s_km,
                'dwell_sec': max(0.0, dwell_sec),
                'label': str(item.get('label', item.get('name', '')) or ''),
                'window_open_utc': str(item.get('window_open_utc', '')) or '',
                'window_close_utc': str(item.get('window_close_utc', '')) or '',
            })
        else:
            try:
                s_km = float(item)
            except Exception:
                continue
            stops.append({'s_km': s_km, 'dwell_sec': 1800.0, 'label': ''})
    return stops
```

7.14 L365 関数 load_profile

- 行範囲: L365–L371
- 所属: module 直下
- 定義: load_profile(path)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: read_csv
- 戻り値の要点: None / pd.read_csv(path) / None
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `not path` を判定し、真なら内部処理を行う。
2. `None` を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `pd.read_csv(path)` を返す。
5. `Exception` を捕捉した場合:
6. `None` を返す。

代表コード断片

```
def load_profile(path):
    if not path:
        return None
    try:
        return pd.read_csv(path)
    except Exception:
        return None
```

7.15 L373 関数 `forecast_distance_column`

- 行範囲: L373–L380
- 所属: `module` 直下
- 定義: `forecast_distance_column(df:pd.DataFrame)->str`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `isinstance`
- 戻り値の要点: `"/"/'s_km'/'route_progress_km'`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `not isinstance(df, pd.DataFrame)` を判定し、真なら内部処理を行う。
2. `"/` を返す。
3. 条件 `'s_km' in df.columns` を判定し、真なら内部処理を行う。
4. `'s_km'` を返す。
5. 条件 `'route_progress_km' in df.columns` を判定し、真なら内部処理を行う。
6. `'route_progress_km'` を返す。
7. `"/` を返す。

代表コード断片

```
def forecast_distance_column(df: pd.DataFrame) -> str:
    if not isinstance(df, pd.DataFrame):
        return ''
    if 's_km' in df.columns:
        return 's_km'
    if 'route_progress_km' in df.columns:
        return 'route_progress_km'
    return ''
```

7.16 L383 関数 load_forecast_dataframe

- 行範囲: L383–L407
- 所属: module 直下
- 定義: load_forecast_dataframe(path:str,tzname:str)->pd.DataFrame
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: ZoneInfo, copy, drop_duplicates, forecast_distance_column, notna, read_csv, reset_index, sort_values, str, to_datetime, tz_convert, tz_localize
- 戻り値の要点: df / df
- 主なローカル代入名: dedup_cols, df, dist_col, sort_cols, t, tzname
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. df に pd.read_csv(path) の結果を代入する。
2. 条件‘time’ not in df.columns を判定し、真なら内部処理を行う。
3. df を返す。
4. t に pd.to_datetime(df[‘time’], format=‘mixed’, errors=‘coerce’) の結果を代入する。
5. tzname に str(tzname or ‘UTC’) の結果を代入する。
6. 条件 t.dt.tz is None を判定し、真なら内部処理を行う。
7. 条件 tzname.upper() == ‘UTC’ を判定し、真なら内部処理を行う。
8. t に t.dt.tz_localize(‘UTC’) の結果を代入する。
9. 上の条件が偽の場合:
10. t に t.dt.tz_localize(ZoneInfo(tzname), ambiguous=‘NaT’, nonexistent=‘NaT’).dt.tz_convert(‘UTC’) の結果を代入する。

11. 上の条件が偽の場合:
12. `t` に `t.dt.tz_convert('UTC')` の結果を代入する。
13. `df` に `df.copy()` の結果を代入する。
14. `df['time']` に `t` の結果を代入する。
15. `dist_col` に `forecast_distance_column(df)` の結果を代入する。
16. `sort_cols` に `['time'] + ([dist_col] if dist_col else [])` の結果を代入する。
17. `dedup_cols` に `['time'] + ([dist_col] if dist_col else [])` の結果を代入する。
18. `df` に `df.loc[df['time'].notna()].sort_values(sort_cols).drop_duplicates(subset=dedup_cols, keep='last').reset_index(drop=True)` の結果を代入する。
19. `df` を返す。

代表コード断片

```
def load_forecast_dataframe(path: str, tzname: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    if 'time' not in df.columns:
        return df
    t = pd.to_datetime(df['time'], format='mixed', errors='coerce')
    tzname = str(tzname or 'UTC')
    if t.dt.tz is None:
        if tzname.upper() == 'UTC':
            t = t.dt.tz_localize('UTC')
        else:
            t = t.dt.tz_localize(ZoneInfo(tzname), ambiguous='NaT', nonexistent='NaT').dt.
tz_convert('UTC')
    else:
        t = t.dt.tz_convert('UTC')
    df = df.copy()
    df['time'] = t
    dist_col = forecast_distance_column(df)
    sort_cols = ['time'] + ([dist_col] if dist_col else [])
    dedup_cols = ['time'] + ([dist_col] if dist_col else [])
    df = (
        df.loc[df['time'].notna()]
        .sort_values(sort_cols)
        .drop_duplicates(subset=dedup_cols, keep='last')
        .reset_index(drop=True)
    )
    return df
```

7.17 L410 関数 `merge_forecast_dataframes`

- 行範囲: L410–L420
- 所属: module 直下
- 定義: `merge_forecast_dataframes(primary:pd.DataFrame, fallback:pd.DataFrame) ->pd.DataFrame`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `combine_first`, `copy`, `len`, `reset_index`, `set_index`, `sort_index`

- 戻り値の要点: `merged / fallback.copy() if fallback is not None else pd.DataFrame() / primary.copy() / primary.copy()`
- 主なローカル代入名: `fallback_idx, merged, primary_idx`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `primary is None or len(primary) == 0` を判定し、真なら内部処理を行う。
2. `fallback.copy() if fallback is not None else pd.DataFrame()` を返す。
3. 条件 `fallback is None or len(fallback) == 0` を判定し、真なら内部処理を行う。
4. `primary.copy()` を返す。
5. 条件 `'time' not in primary.columns or 'time' not in fallback.columns` を判定し、真なら内部処理を行う。
6. `primary.copy()` を返す。
7. `primary_idx` に `primary.set_index('time')` の結果を代入する。
8. `fallback_idx` に `fallback.set_index('time')` の結果を代入する。
9. `merged` に `primary_idx.combine_first(fallback_idx).sort_index().reset_index()` の結果を代入する。
10. `merged` を返す。

代表コード断片

```
def merge_forecast_dataframes(primary: pd.DataFrame, fallback: pd.DataFrame) -> pd.DataFrame:
    if primary is None or len(primary) == 0:
        return fallback.copy() if fallback is not None else pd.DataFrame()
    if fallback is None or len(fallback) == 0:
        return primary.copy()
    if 'time' not in primary.columns or 'time' not in fallback.columns:
        return primary.copy()
    primary_idx = primary.set_index('time')
    fallback_idx = fallback.set_index('time')
    merged = primary_idx.combine_first(fallback_idx).sort_index().reset_index()
    return merged
```

7.18 L423 関数 `build_forecast_grid_payload`

- 行範囲: L423–L473
- 所属: module 直下
- 定義: `build_forecast_grid_payload(df:pd.DataFrame)->dict|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Index, apply, array, bfill, copy, drop_duplicates, dropna, ffill, forecast_distance_column, interpolate, len, max`
- 戻り値の要点: `{'dist_col': dist_col, 'time_ns': np.array([timestamp_ns(value) for value in time_index], dtype=np.int64), 's_grid': s_grid, 'matrices': matrices} / None / None / None`
- 主なローカル代入名: `col, dist_col, matrices, pivot, s_grid, time_index, value, work`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 1、try 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `dist_col` に `forecast_distance_column(df)` の結果を代入する。
2. 条件 `not dist_col or 'time' not in df.columns` を判定し、真なら内部処理を行う。
3. `None` を返す。
4. `work` に `df.copy()` の結果を代入する。
5. `work[dist_col]` に `pd.to_numeric(work[dist_col], errors='coerce')` の結果を代入する。
6. `work` に `work.dropna(subset=['time', dist_col]).sort_values(['time', dist_col])` の結果を代入する。
7. 条件 `work.empty` を判定し、真なら内部処理を行う。
8. `None` を返す。
9. `time_index` に `pd.Index(work['time'].drop_duplicates().sort_values())` の結果を代入する。
10. `s_grid` に `np.array(sorted(work[dist_col].dropna().unique()), dtype=float)` の結果を代入する。
11. 条件 `len(time_index) < 2 or len(s_grid) < 2` を判定し、真なら内部処理を行う。
12. `None` を返す。
13. 条件 `len(work) <= max(len(time_index), len(s_grid))` を判定し、真なら内部処理を行う。
14. `None` を返す。
15. `work` に `work.drop_duplicates(subset=['time', dist_col], keep='last')` の結果を代入する。

16. matrices に {} の結果を代入する。
17. ('GHI', 'DNI', 'DHI', 'POA_drive', 'POA_stop_ideal', 'Tamb_C', 'Tcell_C', 'Tcell_drive_C', 'Tcell_stop_ideal_C', 'headwind_ms') を順に走査し、各要素を col に入れて処理する。
18. 条件 col not in work.columns を判定し、真なら内部処理を行う。
19. Continue 文を実行する。
20. pivot に work.pivot(index='time', columns=dist_col, values=col).reindex(index=time_index, columns=s_grid).apply(pd.to_numeric, errors='coerce').interpolate(axis=0, limit_direction='both').interpolate(axis=1, limit_direction='both').ffill().bfill().ffill(axis=1).bfill().ffill(axis=0) の結果を代入する。
21. matrices[col] に pivot.to_numpy(dtype=float) の結果を代入する。
22. 条件 not matrices を判定し、真なら内部処理を行う。
23. None を返す。
24. {'dist_col': dist_col, 'time_ns': np.array([timestamp_ns(value) for value in time_index], dtype=np.int64), 's_grid': s_grid, 'matrices': matrices} を返す。

代表コード断片

```
def build_forecast_grid_payload(df: pd.DataFrame) -> dict | None:
    dist_col = forecast_distance_column(df)
    if not dist_col or 'time' not in df.columns:
        return None
    work = df.copy()
    work[dist_col] = pd.to_numeric(work[dist_col], errors='coerce')
    work = work.dropna(subset=['time', dist_col]).sort_values(['time', dist_col])
    if work.empty:
        return None
    time_index = pd.Index(work['time'].drop_duplicates().sort_values())
    s_grid = np.array(sorted(work[dist_col].dropna().unique()), dtype=float)
    if len(time_index) < 2 or len(s_grid) < 2:
        return None
    if len(work) <= max(len(time_index), len(s_grid)):
        return None
    work = work.drop_duplicates(subset=['time', dist_col], keep='last')
    matrices = {}
    for col in (
        'GHI',
        'DNI',
        'DHI',
        'POA_drive',
        'POA_stop_ideal',
        'Tamb_C',
        'Tcell_C',
        'Tcell_drive_C',
        'Tcell_stop_ideal_C',
        'headwind_ms',
    ):
        if col not in work.columns:
            continue
        pivot = (
            work.pivot(index='time', columns=dist_col, values=col)
            .reindex(index=time_index, columns=s_grid)
            .apply(pd.to_numeric, errors='coerce')
        )
        matrices[col] = pivot.to_numpy(dtype=float)
    ...
```

7.19 L476 関数 `interp_forecast_grid`

- 行範囲: L476–L520
- 所属: module 直下
- 定義: `interp_forecast_grid(payload:dict|None,col:str,t_utc:datetime,s_km:float|None,default:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `float`, `get`, `int`, `len`, `max`, `searchsorted`, `timestamp_ns`
- 戻り値の要点: `float((1.0 - wt) * v0 + wt * v1) / float(default) / float(default)`
- 主なローカル代入名: `denom_s`, `denom_t`, `i0`, `i1`, `i_hi`, `j0`, `j1`, `j_hi`, `matrix`, `s_val`, `sg`, `t_ns`, `tg`, `v0`, `v00`, `v01`, `v1`, `v10`, `v11`, `ws`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `not payload` を判定し、真なら内部処理を行う。
2. `float(default)` を返す。
3. `matrix` に `payload.get('matrices', {})`.`get(col)` の結果を代入する。
4. `tg` に `payload.get('time_ns')` の結果を代入する。
5. `sg` に `payload.get('s_grid')` の結果を代入する。
6. 条件 `matrix is None or tg is None or sg is None or (len(tg) == 0) or (len(sg) == 0)` を判定し、真なら内部処理を行う。
7. `float(default)` を返す。
8. `t_ns` に `int(np.clip(timestamp_ns(t_utc), int(tg[0]), int(tg[-1])))` の結果を代入する。
9. `s_val` に `float(s_km if s_km is not None else sg[0])` の結果を代入する。
10. `s_val` に `float(np.clip(s_val, float(sg[0]), float(sg[-1])))` の結果を代入する。
11. `i_hi` に `int(np.searchsorted(tg, t_ns, side='left'))` の結果を代入する。
12. 条件 `i_hi <= 0` を判定し、真なら内部処理を行う。
13. `i0`, `i1` に 0 の結果を代入する。
14. `wt` に 0.0 の結果を代入する。
15. 上の条件が偽の場合:

16. 条件 $i_hi \geq \text{len}(tg)$ を判定し、真なら内部処理を行う。
17. $i0, i1$ に $\text{len}(tg) - 1$ の結果を代入する。
18. wt に 0.0 の結果を代入する。
19. 上の条件が偽の場合:
20. $i0$ に $i_hi - 1$ の結果を代入する。
21. $i1$ に i_hi の結果を代入する。
22. denom_t に $\max(\text{int}(tg[i1]) - \text{int}(tg[i0]), 1)$ の結果を代入する。
23. wt に $\text{float}((t_ns - \text{int}(tg[i0])) / \text{denom_t})$ の結果を代入する。
24. j_hi に $\text{int}(\text{np.searchsorted}(sg, s_val, \text{side}='left'))$ の結果を代入する。
25. 条件 $j_hi \leq 0$ を判定し、真なら内部処理を行う。
26. $j0, j1$ に 0 の結果を代入する。
27. ws に 0.0 の結果を代入する。
28. 上の条件が偽の場合:
29. 条件 $j_hi \geq \text{len}(sg)$ を判定し、真なら内部処理を行う。
30. $j0, j1$ に $\text{len}(sg) - 1$ の結果を代入する。
31. ws に 0.0 の結果を代入する。
32. 上の条件が偽の場合:
33. $j0$ に $j_hi - 1$ の結果を代入する。
34. $j1$ に j_hi の結果を代入する。
35. denom_s に $\max(\text{float}(sg[j1]) - \text{float}(sg[j0]), 1e-09)$ の結果を代入する。
36. ws に $\text{float}((s_val - \text{float}(sg[j0])) / \text{denom_s})$ の結果を代入する。
37. $v00$ に $\text{float}(\text{matrix}[i0, j0])$ の結果を代入する。
38. $v01$ に $\text{float}(\text{matrix}[i0, j1])$ の結果を代入する。
39. $v10$ に $\text{float}(\text{matrix}[i1, j0])$ の結果を代入する。
40. $v11$ に $\text{float}(\text{matrix}[i1, j1])$ の結果を代入する。
41. $v0$ に $(1.0 - ws) * v00 + ws * v01$ の結果を代入する。
42. $v1$ に $(1.0 - ws) * v10 + ws * v11$ の結果を代入する。
43. $\text{float}((1.0 - wt) * v0 + wt * v1)$ を返す。

代表コード断片

```
def interp_forecast_grid(payload: dict | None, col: str, t_utc: datetime, s_km: float | None, default: float) -> float:
    if not payload:
        return float(default)
    matrix = payload.get('matrices', {}).get(col)
    tg = payload.get('time_ns')
    sg = payload.get('s_grid')
    if matrix is None or tg is None or sg is None or len(tg) == 0 or len(sg) == 0:
        return float(default)
    t_ns = int(np.clip(timestamp_ns(t_utc), int(tg[0]), int(tg[-1])))
    s_val = float(s_km if s_km is not None else sg[0])
    s_val = float(np.clip(s_val, float(sg[0]), float(sg[-1])))

    i_hi = int(np.searchsorted(tg, t_ns, side='left'))
    if i_hi <= 0:
        i0 = i1 = 0
        wt = 0.0
    elif i_hi >= len(tg):
        i0 = i1 = len(tg) - 1
        wt = 0.0
    else:
        i0 = i_hi - 1
        i1 = i_hi
        denom_t = max(int(tg[i1]) - int(tg[i0]), 1)
        wt = float((t_ns - int(tg[i0])) / denom_t)

    j_hi = int(np.searchsorted(sg, s_val, side='left'))
    if j_hi <= 0:
        j0 = j1 = 0
        ws = 0.0
    elif j_hi >= len(sg):
        j0 = j1 = len(sg) - 1
        ws = 0.0
    else:
        j0 = j_hi - 1
        j1 = j_hi

    ...
```

7.20 L523 関数 load_progress_reference_dataframe

- 行範囲: L523–L540
- 所属: module 直下
- 定義: load_progress_reference_dataframe(path:str)->pd.DataFrame|None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: DataFrame, drop_duplicates, dropna, read_csv, reset_index, sort_values, to_datetime, to_numeric
- 戻り値の要点: out.reset_index(drop=True) / None / None / None
- 主なローカル代入名: df, out, t, time_col
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 4、ループ 0、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 not path を判定し、真なら内部処理を行う。
2. None を返す。
3. 例外処理を伴う try ブロックを実行する。
4. df に pd.read_csv(path, low_memory=False) の結果を代入する。
5. Exception を捕捉した場合:
6. None を返す。
7. time_col に 'time_utc' if 'time_utc' in df.columns else 'time' if 'time' in df.columns else '' の結果を代入する。
8. 条件 not time_col or 's_kmh' not in df.columns を判定し、真なら内部処理を行う。
9. None を返す。
10. t に pd.to_datetime(df[time_col], format='mixed', utc=True, errors='coerce') の結果を代入する。
11. out に pd.DataFrame({'time_utc': t, 's_kmh': pd.to_numeric(df['s_kmh'], errors='coerce')}) の結果を代入する。
12. 条件 'speed_kmh' in df.columns を判定し、真なら内部処理を行う。
13. out['speed_kmh'] に pd.to_numeric(df['speed_kmh'], errors='coerce') の結果を代入する。
14. out に out.dropna(subset=['time_utc', 's_kmh']).sort_values('time_utc').drop_duplicates(subset=['time_utc'], keep='last') の結果を代入する。
15. 条件 out.empty を判定し、真なら内部処理を行う。
16. None を返す。
17. out.reset_index(drop=True) を返す。

代表コード断片

```
def load_progress_reference_dataframe(path: str) -> pd.DataFrame | None:
    if not path:
        return None
    try:
        df = pd.read_csv(path, low_memory=False)
    except Exception:
        return None
    time_col = 'time_utc' if 'time_utc' in df.columns else ('time' if 'time' in df.columns
    else '')
    if not time_col or 's_kmh' not in df.columns:
        return None
    t = pd.to_datetime(df[time_col], format='mixed', utc=True, errors='coerce')
    out = pd.DataFrame({'time_utc': t, 's_kmh': pd.to_numeric(df['s_kmh'], errors='coerce')})
    if 'speed_kmh' in df.columns:
        out['speed_kmh'] = pd.to_numeric(df['speed_kmh'], errors='coerce')
    out = out.dropna(subset=['time_utc', 's_kmh']).sort_values('time_utc').drop_duplicates(
    subset=['time_utc'], keep='last')
    if out.empty:
        return None
    return out.reset_index(drop=True)
```

7.21 L543 関数 `ensure_parent_dir`

- 行範囲: L543–L546
- 所属: module 直下
- 定義: `ensure_parent_dir(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dirname`, `makedirs`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `parent`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `parent` に `os.path.dirname(path)` の結果を代入する。
2. 条件 `parent` を判定し、真なら内部処理を行う。
3. `os.makedirs(...)` を実行する。

代表コード断片

```
def ensure_parent_dir(path):  
    parent = os.path.dirname(path)  
    if parent:  
        os.makedirs(parent, exist_ok=True)
```

7.22 L549 クラス `DetailCsvStream`

- 行範囲: L549–L580
- 所属: module 直下
- 定義: `DetailCsvStream(bases=None)`
- docstring: Write the 1 Hz execution trace without retaining the full race in RAM.
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。

上から順の処理

1. docstring または説明文字列を置く。
2. 関数 `__init__` を定義する。
3. 関数 `write` を定義する。
4. 関数 `close` を定義する。
5. 関数 `__enter__` を定義する。
6. 関数 `__exit__` を定義する。

代表コード断片

```
class DetailCsvStream:
    """Write the 1 Hz execution trace without retaining the full race in RAM."""

    def __init__(self, path):
        self.path = str(path)
        self._file = None
        self._writer = None
        self.fieldnames = []
        self.row_count = 0

    def write(self, row):
        if self._writer is None:
            ensure_parent_dir(self.path)
            self.fieldnames = list(row.keys())
            opener = gzip.open if self.path.lower().endswith('.gz') else open
            self._file = opener(self.path, 'wt' if opener is gzip.open else 'w', encoding='utf-8', newline='')
            self._writer = csv.DictWriter(self._file, fieldnames=self.fieldnames, extrasaction='ignore')
            self._writer.writeheader()
            self._writer.writerow(row)
            self.row_count += 1

    def close(self):
        if self._file is not None:
            self._file.flush()
            self._file.close()
            self._file = None

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc, tb):
        self.close()
```

7.23 L552 関数 `DetailCsvStream.__init__`

- 行範囲: L552–L557
- 所属: `DetailCsvStream`
- 定義: `__init__(self, path)`

- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self._file`, `self._writer`, `self.fieldnames`, `self.path`, `self.row_count`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.path` に `str(path)` の結果を代入する。
2. `self._file` に `None` の結果を代入する。
3. `self._writer` に `None` の結果を代入する。
4. `self.fieldnames` に `[]` の結果を代入する。
5. `self.row_count` に `0` の結果を代入する。

代表コード断片

```
def __init__(self, path):
    self.path = str(path)
    self._file = None
    self._writer = None
    self.fieldnames = []
    self.row_count = 0
```

7.24 L559 関数 `DetailCsvStream.write`

- 行範囲: L559–L568
- 所属: `DetailCsvStream`
- 定義: `write(self, row)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DictWriter`, `endswith`, `ensure_parent_dir`, `keys`, `list`, `lower`, `opener`, `writeheader`, `writerow`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `opener`
- 読み取る主なインスタンス属性: `self._file`, `self._writer`, `self.fieldnames`, `self.path`
- 更新する主なインスタンス属性: `self._file`, `self._writer`, `self.fieldnames`, `self.row_count`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 self._writer is None を判定し、真なら内部処理を行う。
2. ensure_parent_dir(...) を実行する。
3. self.fieldnames に list(row.keys()) の結果を代入する。
4. opener に gzip.open if self.path.lower().endswith('.gz') else open の結果を代入する。
5. self._file に opener(self.path, 'wt' if opener is gzip.open else 'w', encoding='utf-8', newline='') の結果を代入する。
6. self._writer に csv.DictWriter(self._file, fieldnames=self.fieldnames, extrasaction='ignore') の結果を代入する。
7. self._writer.writeheader(...) を実行する。
8. self._writer.writerow(...) を実行する。
9. self.row_count を Add で更新する。

代表コード断片

```
def write(self, row):
    if self._writer is None:
        ensure_parent_dir(self.path)
        self.fieldnames = list(row.keys())
        opener = gzip.open if self.path.lower().endswith('.gz') else open
        self._file = opener(self.path, 'wt' if opener is gzip.open else 'w', encoding='utf-8', newline='')
        self._writer = csv.DictWriter(self._file, fieldnames=self.fieldnames, extrasaction='ignore')
        self._writer.writeheader()
        self._writer.writerow(row)
        self.row_count += 1
```

7.25 L570 関数 DetailCsvStream.close

- 行範囲: L570–L574
- 所属: DetailCsvStream
- 定義: close(self)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: close, flush
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self._file
- 更新する主なインスタンス属性: self._file
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `self._file is not None` を判定し、真なら内部処理を行う。
2. `self._file.flush(...)` を実行する。
3. `self._file.close(...)` を実行する。
4. `self._file` に `None` の結果を代入する。

代表コード断片

```
def close(self):
    if self._file is not None:
        self._file.flush()
        self._file.close()
    self._file = None
```

7.26 L576 関数 `DetailCsvStream.__enter__`

- 行範囲: L576–L577
- 所属: `DetailCsvStream`
- 定義: `__enter__(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: `self`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self` を返す。

代表コード断片

```
def __enter__(self):
    return self
```

7.27 L579 関数 `DetailCsvStream.__exit__`

- 行範囲: L579–L580
- 所属: `DetailCsvStream`
- 定義: `__exit__(self, exc_type, exc, tb)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `close`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.close`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.close(...)` を実行する。

代表コード断片

```
def __exit__(self, exc_type, exc, tb):  
    self.close()
```

7.28 L583 関数 `_deep_copy_cfg`

- 行範囲: L583–L584
- 所属: `module` 直下
- 定義: `_deep_copy_cfg(cfg)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `deepcopy`, `isinstance`
- 戻り値の要点: `copy.deepcopy(cfg) if isinstance(cfg, dict) else {}`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `copy.deepcopy(cfg) if isinstance(cfg, dict) else {}` を返す。

代表コード断片

```
def _deep_copy_cfg(cfg):  
    return copy.deepcopy(cfg) if isinstance(cfg, dict) else {}
```

7.29 L587 関数 `_set_nested`

- 行範囲: L587–L598
- 所属: module 直下
- 定義: `_set_nested(cfg, dotted_key:str, value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `get`, `isinstance`, `split`, `str`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `cur`, `next_val`, `part`, `parts`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('override key is empty')`
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `parts` に `[part for part in str(dotted_key).split('.') if part]` の結果を代入する。
2. 条件 `not parts` を判定し、真なら内部処理を行う。
3. `ValueError('override key is empty')` を送出する。
4. `cur` に `cfg` の結果を代入する。
5. `parts[:-1]` を順に走査し、各要素を `part` に入れて処理する。
6. `next_val` に `cur.get(part)` の結果を代入する。
7. 条件 `not isinstance(next_val, dict)` を判定し、真なら内部処理を行う。
8. `next_val` に `{}` の結果を代入する。
9. `cur[part]` に `next_val` の結果を代入する。
10. `cur` に `next_val` の結果を代入する。
11. `cur[parts[-1]]` に `value` の結果を代入する。

代表コード断片

```
def _set_nested(cfg, dotted_key: str, value):
    parts = [part for part in str(dotted_key).split('.') if part]
    if not parts:
        raise ValueError('override key is empty')
    cur = cfg
    for part in parts[:-1]:
        next_val = cur.get(part)
        if not isinstance(next_val, dict):
            next_val = {}
            cur[part] = next_val
        cur = next_val
    cur[parts[-1]] = value
```

7.30 L601 関数 `apply_overrides`

- 行範囲: L601–L615
- 所属: module 直下
- 定義: `apply_overrides(cfg, overrides)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `_deep_copy_cfg`, `_set_nested`, `append`, `safe_load`, `split`, `str`, `strip`
- 戻り値の要点: `(cfg, applied)`
- 主なローカル代入名: `applied`, `cfg`, `item`, `key`, `raw`, `raw_value`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError(f'Invalid override: {item}')`
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `cfg` に `_deep_copy_cfg(cfg)` の結果を代入する。
2. `applied` に `[]` の結果を代入する。
3. `overrides` or `[]` を順に走査し、各要素を `raw` に入れて処理する。
4. `item` に `str(raw).strip()` の結果を代入する。
5. 条件 `not item` を判定し、真なら内部処理を行う。
6. `Continue` 文を実行する。
7. 条件 `'=' not in item` を判定し、真なら内部処理を行う。
8. `ValueError(f'Invalid override: {item}')` を送出する。

9. (key, raw_value) に item.split('=', 1) の結果を代入する。
10. key に key.strip() の結果を代入する。
11. value に yaml.safe_load(raw_value) の結果を代入する。
12. _set_nested(...) を実行する。
13. applied.append(...) を実行する。
14. (cfg, applied) を返す。

代表コード断片

```
def apply_overrides(cfg, overrides):
    cfg = _deep_copy_cfg(cfg)
    applied = []
    for raw in overrides or []:
        item = str(raw).strip()
        if not item:
            continue
        if '=' not in item:
            raise ValueError(f'Invalid override: {item}')
        key, raw_value = item.split('=', 1)
        key = key.strip()
        value = yaml.safe_load(raw_value)
        _set_nested(cfg, key, value)
        applied.append({'key': key, 'value': value})
    return cfg, applied
```

7.31 L618 関数 build_config_tag

- 行範囲: L618–L630
- 所属: module 直下
- 定義: build_config_tag(profile_path:str, profile_cfg:dict, overrides=None)->str
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: encode, fromtimestamp, getmtime, hexdigest, isinstance, now, safe_dump, sha1, strftime
- 戻り値の要点: f'{stamp}_{digest}'
- 主なローカル代入名: canonical, digest, mtime, payload, stamp
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `payload` に `{'profile_cfg': profile_cfg if isinstance(profile_cfg, dict) else {}, 'overrides': overrides or []}` の結果を代入する。
2. `canonical` に `yaml.safe_dump(payload, sort_keys=True, allow_unicode=True)` の結果を代入する。
3. `digest` に `hashlib.sha1(canonical.encode('utf-8')).hexdigest()[:8]` の結果を代入する。
4. 例外処理を伴う `try` ブロックを実行する。
5. `mtime` に `datetime.fromtimestamp(os.path.getmtime(profile_path), timezone.utc)` の結果を代入する。
6. `Exception` を捕捉した場合:
7. `mtime` に `datetime.now(timezone.utc)` の結果を代入する。
8. `stamp` に `mtime.strftime('%Y%m%d_%H%M%S')` の結果を代入する。
9. `f'{stamp}_{digest}'` を返す。

代表コード断片

```
def build_config_tag(profile_path: str, profile_cfg: dict, overrides=None) -> str:
    payload = {
        'profile_cfg': profile_cfg if isinstance(profile_cfg, dict) else {},
        'overrides': overrides or [],
    }
    canonical = yaml.safe_dump(payload, sort_keys=True, allow_unicode=True)
    digest = hashlib.sha1(canonical.encode('utf-8')).hexdigest()[:8]
    try:
        mtime = datetime.fromtimestamp(os.path.getmtime(profile_path), timezone.utc)
    except Exception:
        mtime = datetime.now(timezone.utc)
    stamp = mtime.strftime('%Y%m%d_%H%M%S')
    return f'{stamp}_{digest}'
```

7.32 L633 関数 `tag_output_path`

- 行範囲: L633–L642
- 所属: `module` 直下
- 定義: `tag_output_path(path_value:str, tag:str, default_ext:str='') -> str`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `replace`, `splitext`, `str`, `strip`
- 戻り値の要点: `f'{stem}_{tag}{ext}' / raw / raw.replace('{tag}', tag)`
- 主なローカル代入名: `ext`, `raw`, `stem`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. raw に `str(path_value or "").strip()` の結果を代入する。
2. 条件 `not raw or not tag` を判定し、真なら内部処理を行う。
3. raw を返す。
4. 条件 `'{tag}' in raw` を判定し、真なら内部処理を行う。
5. `raw.replace('{tag}', tag)` を返す。
6. (stem, ext) に `os.path.splitext(raw)` の結果を代入する。
7. 条件 `not ext and default_ext` を判定し、真なら内部処理を行う。
8. ext に `default_ext` の結果を代入する。
9. `f'{stem}_{tag}{ext}'` を返す。

代表コード断片

```
def tag_output_path(path_value: str, tag: str, default_ext: str = '') -> str:
    raw = str(path_value or '').strip()
    if not raw or not tag:
        return raw
    if '{tag}' in raw:
        return raw.replace('{tag}', tag)
    stem, ext = os.path.splitext(raw)
    if not ext and default_ext:
        ext = default_ext
    return f'{stem}_{tag}{ext}'
```

7.33 L645 関数 `apply_profile_cfg_to_args`

- 行範囲: L645–L795
- 所属: module 直下
- 定義: `apply_profile_cfg_to_args(profile_path, profile_cfg, args, *, force_output_defaults=False)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool`, `build_config_tag`, `clip`, `endswith`, `float`, `get`, `get_path`, `get_section`, `getattr`, `int`, `isinstance`, `join`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `auto_version_outputs`, `command_bridge_cfg`, `detail_compression`, `detail_csv`, `exec_cfg`, `latest_manifest_json`, `live_cfg`, `logging_cfg`, `mpc_cfg`, `output_dir`, `output_prefix`, `output_tag`, `plan_csv`, `report_html`, `resolved_yaml`, `runtime_cfg`, `sim_cfg`, `summary_json`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 9、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `not profile_cfg` を判定し、真なら内部処理を行う。
2. を返す。
3. `sim_cfg` に `get_section(profile_cfg, 'simulation')` の結果を代入する。
4. `runtime_cfg` に `get_section(profile_cfg, 'runtime')` の結果を代入する。
5. `logging_cfg` に `get_section(profile_cfg, 'logging')` の結果を代入する。
6. `auto_version_outputs` に `bool(sim_cfg.get('auto_version_outputs', True))` の結果を代入する。
7. `output_tag` に `build_config_tag(profile_path, profile_cfg, getattr(args, 'override', [])) if auto_version_outputs else "` の結果を代入する。
8. `args.params_yaml` に `profile_path` の結果を代入する。
9. `args.profile_path_resolved` に `profile_path` の結果を代入する。
10. `args.auto_version_outputs` に `auto_version_outputs` の結果を代入する。
11. `args.output_tag` に `output_tag` の結果を代入する。
12. `args.forecast_csv` に `get_path(profile_cfg, profile_path, 'forecast_csv', args.forecast_csv)` の結果を代入する。
13. `args.forecast_fill_csv` に `get_path(profile_cfg, profile_path, 'forecast_fill_csv', getattr(args, 'forecast_fill_csv', ""))` の結果を代入する。
14. `args.progress_reference_csv` に `get_path(profile_cfg, profile_path, 'progress_reference_csv', getattr(args, 'progress_reference_csv', ""))` の結果を代入する。
15. `args.route_profile_csv` に `get_path(profile_cfg, profile_path, 'route_profile_csv', args.route_profile_csv)` の結果を代入する。
16. `args.speed_profile_csv` に `get_path(profile_cfg, profile_path, 'speed_profile_csv', args.speed_profile_csv)` の結果を代入する。
17. `args.stop_yaml` に `get_path(profile_cfg, profile_path, 'stop_yaml', args.stop_yaml)` の結果を代入する。
18. `args.drive_schedule_yaml` に `get_path(profile_cfg, profile_path, 'drive_schedule_yaml', args.drive_schedule_yaml)` の結果を代入する。
19. `args.drive_eff_map` に `get_path(profile_cfg, profile_path, 'drive_eff_map', args.drive_eff_map)` の結果を代入する。
20. `args.regen_eff_map` に `get_path(profile_cfg, profile_path, 'regen_eff_map', args.regen_eff_map)` の結果を代入する。
21. `args.rint_map` に `get_path(profile_cfg, profile_path, 'rint_map', args.rint_map)` の結果を代入する。
22. `args.panel_eff_map` に `get_path(profile_cfg, profile_path, 'panel_eff_map', args.panel_eff_map)` の結果を代入する。
23. `args.mppt_eff_map` に `get_path(profile_cfg, profile_path, 'mppt_eff_map', args.mppt_eff_map)` の結果を代入する。
24. `args.ocv_soc_map` に `get_path(profile_cfg, profile_path, 'ocv_soc_map', args.ocv_soc_map)` の結果を代入する。
25. `args.drive_map_eco` に `get_path(profile_cfg, profile_path, 'drive_map_eco', args.drive_map_eco)` の結果を代入する。

26. `args.drive_map_power` に `get_path(profile_cfg, profile_path, 'drive_map_power', args.drive_map_power)` の結果を代入する。
27. `args.regen_map_eco` に `get_path(profile_cfg, profile_path, 'regen_map_eco', args.regen_map_eco)` の結果を代入する。
28. `args.regen_map_power` に `get_path(profile_cfg, profile_path, 'regen_map_power', args.regen_map_power)` の結果を代入する。
29. `args.forecast_time_mode` に `str(sim_cfg.get('forecast_time_mode', runtime_cfg.get('forecast_time_mode', args.forecast_time_mode)))` の結果を代入する。
30. `args.forecast_time_tz` に `str(runtime_cfg.get('forecast_time_tz', args.forecast_time_tz))` の結果を代入する。
31. `args.dt` に `float(get_section(profile_cfg, 'mpc').get('dt', args.dt))` の結果を代入する。
32. `args.horizon_steps` に `int(get_section(profile_cfg, 'mpc').get('horizon_steps', args.horizon_steps))` の結果を代入する。
33. `args.soc0` に `float(sim_cfg.get('soc0', args.soc0))` の結果を代入する。
34. `args.Tb0` に `float(sim_cfg.get('Tb0', args.Tb0))` の結果を代入する。
35. `args.v0_kmh` に `float(sim_cfg.get('v0_kmh', args.v0_kmh))` の結果を代入する。
36. `args.start_utc` に `str(sim_cfg.get('start_utc', args.start_utc or '') or '')` の結果を代入する。
37. `args.forecast_start_time_utc` に `str(sim_cfg.get('forecast_start_time_utc', args.forecast_start_time_utc or '') or '')` の結果を代入する。
38. `args.start_s_km` に `float(sim_cfg.get('start_s_km', args.start_s_km))` の結果を代入する。
39. `args.energy_budget` に `bool(sim_cfg.get('energy_budget', args.energy_budget))` の結果を代入する。
40. `args.solar_gain` に `float(get_section(profile_cfg, 'mpc').get('solar_gain', args.solar_gain))` の結果を代入する。
41. `args.poa_gain_drive` に `float(get_section(profile_cfg, 'mpc').get('poa_gain_drive', args.poa_gain_drive))` の結果を代入する。
42. `args.poa_gain_stop` に `float(get_section(profile_cfg, 'mpc').get('poa_gain_stop', args.poa_gain_stop))` の結果を代入する。
43. `args.stop_tilt_fraction` に `float(np.clip(get_section(profile_cfg, 'mpc').get('stop_tilt_fraction', args.stop_tilt_fraction), 0.0, 1.0))` の結果を代入する。
44. `args.control_stop_tilt_fraction` に `float(np.clip(get_section(profile_cfg, 'mpc').get('control_stop_tilt_fraction', args.control_stop_tilt_fraction), 0.0, 1.0))` の結果を代入する。
45. `mpc_cfg` に `get_section(profile_cfg, 'mpc')` の結果を代入する。
46. `live_cfg` に `get_section(profile_cfg, 'live')` の結果を代入する。
47. `command_bridge_cfg` に `live_cfg.get('command_bridge', {})` if `isinstance(live_cfg.get('command_bridge', {}), dict)` else `{}` の結果を代入する。
48. `exec_cfg` に `sim_cfg.get('execution_model', {})` if `isinstance(sim_cfg.get('execution_model', {}), dict)` else `{}` の結果を代入する。
49. `args.exec_model_enabled` に `bool(exec_cfg.get('enabled', args.exec_model_enabled))` の結果を代入する。
50. `args.exec_inner_dt_sec` に `float(exec_cfg.get('inner_dt_sec', args.exec_inner_dt_sec))` の結果を代入する。
51. `args.detail_rate_hz` に `float(sim_cfg.get('detail_rate_hz', args.detail_rate_hz))` の結果を代入する。

52. args.exec_tau_sec に float(exec_cfg.get('tau_sec', command_bridge_cfg.get('filter_tau_sec', mpc_cfg.get('speed_meas_f', args.exec_tau_sec)))) の結果を代入する。
53. args.exec_accel_limit_kmhps に float(exec_cfg.get('accel_limit_kmhps', command_bridge_cfg.get('accel_limit_kmhps', mpc_cfg.get('lower_ref_accel_limit_kmhps', args.exec_accel_limit_kmhps)))) の結果を代入する。
54. args.exec_decel_limit_kmhps に float(exec_cfg.get('decel_limit_kmhps', command_bridge_cfg.get('decel_limit_kmhps', mpc_cfg.get('lower_ref_decel_limit_kmhps', args.exec_decel_limit_kmhps)))) の結果を代入する。
55. args.exec_deadband_kmh に float(exec_cfg.get('deadband_kmh', command_bridge_cfg.get('speed_deadband_kmh', mpc_cfg.get('lower_ref_deadband_kmh', args.exec_deadband_kmh)))) の結果を代入する。
56. args.exec_quantize_step_kmh に float(exec_cfg.get('quantize_step_kmh', command_bridge_cfg.get('speed_quantize_step_kmh', args.exec_quantize_step_kmh)))) の結果を代入する。
57. args.exec_reaction_delay_sec に float(exec_cfg.get('reaction_delay_sec', args.exec_reaction_delay_sec)) の結果を代入する。
58. output_dir に sim_cfg.get('output_dir', os.path.join('outputs', 'prerace')) の結果を代入する。
59. output_prefix に str(sim_cfg.get('output_prefix', logging_cfg.get('file_prefix', 'solar_prerace'))) の結果を代入する。
60. 条件 force_output_defaults or not args.out_csv or args.out_csv == 'solar_sim_log.csv' を判定し、真なら内部処理を行う。
61. args.out_csv に tag_output_path(os.path.join(output_dir, f'{output_prefix}.csv'), output_tag, '.csv') の結果を代入する。
62. 条件 force_output_defaults or not args.out_detail_csv を判定し、真なら内部処理を行う。
63. detail_csv に str(sim_cfg.get('out_detail_csv', '') or '') の結果を代入する。
64. args.out_detail_csv に tag_output_path(detail_csv, output_tag, '.csv') if detail_csv else tag_output_path(os.path.join(output_dir, f'{output_prefix}_detail.csv'), output_tag, '.csv') の結果を代入する。
65. detail_compression に str(sim_cfg.get('detail_compression', '') or '').strip().lower() の結果を代入する。
66. 条件 detail_compression in ('gzip', 'gz') and (not args.out_detail_csv.lower().endswith('.gz')) を判定し、真なら内部処理を行う。
67. args.out_detail_csv を Add で更新する。
68. 条件 force_output_defaults or not args.out_plan_csv を判定し、真なら内部処理を行う。
69. plan_csv に str(sim_cfg.get('out_plan_csv', '') or '') の結果を代入する。
70. args.out_plan_csv に tag_output_path(plan_csv, output_tag, '.csv') if plan_csv else tag_output_path(os.path.join(output_dir, f'{output_prefix}_upper_plan.csv'), output_tag, '.csv') の結果を代入する。
71. 条件 force_output_defaults or not args.report_html を判定し、真なら内部処理を行う。
72. report_html に str(sim_cfg.get('report_html', '') or '') の結果を代入する。
73. args.report_html に tag_output_path(report_html, output_tag, '.html') if report_html else tag_output_path(os.path.join(output_dir, f'{output_prefix}_report.html'), output_tag, '.html') の結果を代入する。
74. 条件 force_output_defaults or not args.summary_json を判定し、真なら内部処理を行う。
75. summary_json に str(sim_cfg.get('summary_json', '') or '') の結果を代入する。
76. args.summary_json に tag_output_path(summary_json, output_tag, '.json') if summary_json else tag_output_path(os.path.join(output_dir, f'{output_prefix}_summary.json'), output_tag, '.json') の結果を代入する。

77. 条件 `force_output_defaults` or not `args.resolved_yaml` を判定し、真なら内部処理を行う。
78. `resolved_yaml` に `str(sim_cfg.get('resolved_yaml', ''))` or `''` の結果を代入する。
79. `args.resolved_yaml` に `tag_output_path(resolved_yaml, output_tag, '.yaml')` if `resolved_yaml` else `tag_output_path(os.path.join(output_prefix, resolved_yaml), output_tag, '.yaml')` の結果を代入する。
80. `latest_manifest_json` に `str(sim_cfg.get('latest_manifest_json', ''))` or `''` の結果を代入する。

代表コード断片

```
def apply_profile_cfg_to_args(profile_path, profile_cfg, args, *, force_output_defaults=False):
    if not profile_cfg:
        return

    sim_cfg = get_section(profile_cfg, 'simulation')
    runtime_cfg = get_section(profile_cfg, 'runtime')
    logging_cfg = get_section(profile_cfg, 'logging')
    auto_version_outputs = bool(sim_cfg.get('auto_version_outputs', True))
    output_tag = build_config_tag(profile_path, profile_cfg, getattr(args, 'override', []))
    if auto_version_outputs else ''

    args.params_yaml = profile_path
    args.profile_path_resolved = profile_path
    args.auto_version_outputs = auto_version_outputs
    args.output_tag = output_tag
    args.forecast_csv = get_path(profile_cfg, profile_path, 'forecast_csv', args.forecast_csv)
    args.forecast_fill_csv = get_path(profile_cfg, profile_path, 'forecast_fill_csv',
    getattr(args, 'forecast_fill_csv', ''))
    args.progress_reference_csv = get_path(profile_cfg, profile_path, 'progress_reference_csv',
    getattr(args, 'progress_reference_csv', ''))
    args.route_profile_csv = get_path(profile_cfg, profile_path, 'route_profile_csv', args.route_profile_csv)
    args.speed_profile_csv = get_path(profile_cfg, profile_path, 'speed_profile_csv', args.speed_profile_csv)
    args.stop_yaml = get_path(profile_cfg, profile_path, 'stop_yaml', args.stop_yaml)
    args.drive_schedule_yaml = get_path(profile_cfg, profile_path, 'drive_schedule_yaml',
    args.drive_schedule_yaml)
    args.drive_eff_map = get_path(profile_cfg, profile_path, 'drive_eff_map', args.drive_eff_map)
    args.regen_eff_map = get_path(profile_cfg, profile_path, 'regen_eff_map', args.regen_eff_map)
    args.rint_map = get_path(profile_cfg, profile_path, 'rint_map', args.rint_map)
    args.panel_eff_map = get_path(profile_cfg, profile_path, 'panel_eff_map', args.panel_eff_map)
    args.mppt_eff_map = get_path(profile_cfg, profile_path, 'mppt_eff_map', args.mppt_eff_map)
    args.ocv_soc_map = get_path(profile_cfg, profile_path, 'ocv_soc_map', args.ocv_soc_map)
    args.drive_map_eco = get_path(profile_cfg, profile_path, 'drive_map_eco', args.drive_map_eco)
    args.drive_map_power = get_path(profile_cfg, profile_path, 'drive_map_power', args.drive_map_power)
    args.regen_map_eco = get_path(profile_cfg, profile_path, 'regen_map_eco', args.regen_map_eco)
    args.regen_map_power = get_path(profile_cfg, profile_path, 'regen_map_power', args.regen_map_power)

    args.forecast_time_mode = str(sim_cfg.get('forecast_time_mode', runtime_cfg.get('forecast_time_mode',
    args.forecast_time_mode)))
    args.forecast_time_tz = str(runtime_cfg.get('forecast_time_tz', args.forecast_time_tz))
    args.dt = float(get_section(profile_cfg, 'mpc').get('dt', args.dt))
    ...
```

7.34 L798 関数 `apply_profile_defaults`

- 行範囲: L798–L803
- 所属: module 直下
- 定義: `apply_profile_defaults(args)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `apply_profile_cfg_to_args`, `load_workflow_profile`
- 戻り値の要点: `profile_cfg / {}`
- 主なローカル代入名: `profile_cfg`, `profile_path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `not args.profile_yaml` を判定し、真なら内部処理を行う。
2. `{}` を返す。
3. `(profile_path, profile_cfg)` に `load_workflow_profile(args.profile_yaml)` の結果を代入する。
4. `apply_profile_cfg_to_args(...)` を実行する。
5. `profile_cfg` を返す。

代表コード断片

```
def apply_profile_defaults(args):
    if not args.profile_yaml:
        return {}
    profile_path, profile_cfg = load_workflow_profile(args.profile_yaml)
    apply_profile_cfg_to_args(profile_path, profile_cfg, args)
    return profile_cfg
```

7.35 L806 関数 `resolve_config_path`

- 行範囲: L806–L817
- 所属: module 直下
- 定義: `resolve_config_path(profile_path, value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `exists`, `is_absolute`, `resolve`, `str`, `strip`
- 戻り値の要点: `str((ROOT / path).resolve()) / "/" / str(path) / str(candidate)`

- 主なローカル代入名: `candidate`, `path`, `profile_dir`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `value` に `str(value or '')` の結果を代入する。
2. 条件 `not value` を判定し、真なら内部処理を行う。
3. `''` を返す。
4. `path` に `Path(value)` の結果を代入する。
5. 条件 `path.is_absolute()` を判定し、真なら内部処理を行う。
6. `str(path)` を返す。
7. `profile_dir` に `Path(profile_path).resolve().parent if profile_path else ROOT` の結果を代入する。
8. `candidate` に `(profile_dir / path).resolve()` の結果を代入する。
9. 条件 `candidate.exists()` を判定し、真なら内部処理を行う。
10. `str(candidate)` を返す。
11. `str((ROOT / path).resolve())` を返す。

代表コード断片

```
def resolve_config_path(profile_path, value):
    value = str(value or '').strip()
    if not value:
        return ''
    path = Path(value)
    if path.is_absolute():
        return str(path)
    profile_dir = Path(profile_path).resolve().parent if profile_path else ROOT
    candidate = (profile_dir / path).resolve()
    if candidate.exists():
        return str(candidate)
    return str((ROOT / path).resolve())
```

7.36 L820 関数 `evaluate_model_validation_gate`

- 行範囲: L820–L1029
- 所属: `module` 直下
- 定義: `evaluate_model_validation_gate(cfg, profile_path)`
- `docstring`: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: Path, abs, all, bool, dict, exists, float, get, int, isfinite, isinstance, load_yaml
- 戻り値の要点: result / result
- 主なローカル代入名: acceleration_fit, checks, fit_summary_path, gate_cfg, grade_fit, identification, label, metrics, observed, predicted, prefix, result, spread, summary, terminal, terminal_anchor, terminal_anchor_role, terminal_path, terminal_voltage_observed, terminal_voltage_predicted
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 5、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. identification に `cfg.get('identification', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
2. gate_cfg に `identification.get('validation_gate', {})` if `isinstance(identification, dict)` else `{}` の結果を代入する。
3. 条件 `not isinstance(gate_cfg, dict)` を判定し、真なら内部処理を行う。
4. gate_cfg に `{}` の結果を代入する。
5. fit_summary_path に `resolve_config_path(profile_path, identification.get('fit_summary_yaml', ''))` の結果を代入する。
6. terminal_path に `resolve_config_path(profile_path, identification.get('terminal_consistency_yaml', ''))` の結果を代入する。
7. result に `{'available': False, 'fit_summary_yaml': fit_summary_path, 'terminal_consistency_yaml': terminal_path, 'gate_pass': False, 'checks': {}, 'thresholds': {}}` の結果を代入する。
8. 条件 `not fit_summary_path or not Path(fit_summary_path).exists()` を判定し、真なら内部処理を行う。
9. `result['reason']` に `'fit summary is not configured or does not exist'` の結果を代入する。
10. result を返す。
11. summary に `load_yaml(fit_summary_path)` の結果を代入する。
12. metrics に `summary.get('validation_metrics', {})` if `isinstance(summary, dict)` else `{}` の結果を代入する。
13. thresholds に `{'vehicle_power_rmse_max_w': float(gate_cfg.get('vehicle_power_rmse_max_w', 150.0)), 'vehicle_voltage_rmse_max_v': float(gate_cfg.get('vehicle_voltage_rmse_max_v', 1.0)), 'conditional_power_rmse_max_w': float(gate_cfg.get('conditional_power_rmse_max_w', 150.0)), 'conditional_voltage_rmse_max_v': float(gate_cfg.get('conditional_voltage_rmse_max_v', 1.0)), 'end_to_end_power_rmse_max_w': float(gate_cfg.get('end_to_end_power_rmse_max_w', 200.0)), 'end_to_end_voltage_rmse_max_v': float(gate_cfg.get('end_to_end_voltage_rmse_max_v', 2.0)), 'moving_pv_rmse_max_w': float(gate_cfg.get('moving_pv_rmse_max_w', 150.0)), 'pv_lodo_moving_rmse_max_w': float(gate_cfg.get('pv_lodo_moving_rmse_max_w', 150.0)), 'pv_lodo_deployed_stop_rmse_max_w': float(gate_cfg.get('pv_lodo_deployed_stop_rmse_max_w', 200.0)), 'power_residual_mean_120s_rmse_max_w': float(gate_cfg.get('power_residual_mean_120s_rmse_max_w', 150.0)), 'energy_error_25km_rmse_max_wh': float(gate_cfg.get('energy_error_25km_rmse_max_wh', 35.0)), 'terminal_soc_evidence_spread_max':`

float(gate_cfg.get('terminal_soc_evidence_spread_max', 0.05)), 'terminal_replay_soc_error_max': float(gate_cfg.get('terminal_replay_soc_error_max', 0.02)), 'terminal_replay_voltage_error_max_v': float(gate_cfg.get('terminal_replay_voltage_error_max_v', 0.5)), 'vehicle_terminal_soc_error_max': float(gate_cfg.get('vehicle_terminal_soc_error_max', 0.02)), 'vehicle_terminal_voltage_error_max_v': float(gate_cfg.get('vehicle_terminal_voltage_error_max_v', 0.5)), 'end_to_end_terminal_soc_error_max': float(gate_cfg.get('end_to_end_terminal_soc_error_max', 0.03)), 'end_to_end_terminal_voltage_error_max_v': float(gate_cfg.get('end_to_end_terminal_voltage_error_max_v', 1.0)), 'acceleration_validation_rmse_ratio_max': float(gate_cfg.get('acceleration_validation_rmse_ratio_max', 1.02)), 'acceleration_validation_min_samples': int(gate_cfg.get('acceleration_validation_min_samples', 100)), 'grade_validation_rmse_ratio_max': float(gate_cfg.get('grade_validation_rmse_ratio_max', 1.02)), 'grade_validation_min_samples': int(gate_cfg.get('grade_validation_min_samples', 100))} の結果を代入する。

14. values に {'vehicle_power_rmse_w': float(metrics.get('power_rmse_clean_w', math.inf)), 'vehicle_voltage_rmse_v': float(metrics.get('voltage_rmse_clean_v', math.inf)), 'conditional_power_rmse_w': float(metrics.get('battery_conditional_power_rmse_w', math.inf)), 'conditional_voltage_rmse_v': float(metrics.get('battery_conditional_voltage_rmse_clean_v', math.inf)), 'end_to_end_power_rmse_w': float(metrics.get('end_to_end_power_rmse_clean_w', math.inf)), 'end_to_end_voltage_rmse_v': float(metrics.get('end_to_end_voltage_rmse_clean_v', math.inf)), 'moving_pv_rmse_w': float(metrics.get('end_to_end_moving_pv_rmse_w', math.inf)), 'pv_lodo_moving_rmse_w': float(metrics.get('pv_lodo_moving_rmse_w', math.inf)), 'pv_lodo_deployed_stop_rmse_w': float(metrics.get('pv_lodo_deployed_stop_rmse_w', math.inf)), 'power_residual_mean_120s_rmse_w': float(metrics.get('end_to_end_power_residual_mean_120s_rmse_w', math.inf)), 'energy_error_25km_rmse_wh': float(metrics.get('end_to_end_energy_error_25km_rmse_wh', math.inf))} の結果を代入する。
15. terminal に load_yaml(terminal_path) if terminal_path and Path(terminal_path).exists() else {} の結果を代入する。
16. spread に float(terminal.get('evidence_interval_max', math.inf)) - float(terminal.get('evidence_interval_min', -math.inf)) の結果を代入する。
17. values['terminal_soc_evidence_spread'] に spread の結果を代入する。
18. terminal_anchor_role に str((summary.get('fit_plan', {}) or {}).get('terminal_anchor_role', 'unknown')).strip().lower() の結果を代入する。
19. values['terminal_anchor_role'] に terminal_anchor_role の結果を代入する。
20. values['terminal_replay_soc_error'] に abs(float(metrics.get('battery_conditional_retire_anchor_soc_error', math.inf))) の結果を代入する。
21. terminal_voltage_observed に float(metrics.get('battery_conditional_retire_anchor_voltage_obs_v', math.nan)) の結果を代入する。
22. terminal_voltage_predicted に float(metrics.get('battery_conditional_retire_anchor_voltage_pred_v', math.nan)) の結果を代入する。
23. values['terminal_replay_voltage_error_v'] に abs(terminal_voltage_predicted - terminal_voltage_observed) の結果を代入する。
24. (('vehicle', ''), ('end_to_end', 'end_to_end_')) を順に走査し、各要素を (label, prefix) に入れて処理する。
25. values[f'{label}_terminal_soc_error'] に abs(float(metrics.get(f'{prefix}retire_anchor_soc_error', math.inf))) の結果を代入する。
26. observed に float(metrics.get(f'{prefix}retire_anchor_voltage_obs_v', math.nan)) の結果を代入する。
27. predicted に float(metrics.get(f'{prefix}retire_anchor_voltage_pred_v', math.nan)) の結果を代入する。
28. values[f'{label}_terminal_voltage_error_v'] に abs(predicted - observed) の結果を代入する。

29. checks に {'vehicle_power_rmse': values['vehicle_power_rmse_w'] <= thresholds['vehicle_power_rmse_max_w'],
'vehicle_voltage_rmse': values['vehicle_voltage_rmse_v'] <= thresholds['vehicle_voltage_rmse_max_v'],
'conditional_power_rmse': values['conditional_power_rmse_w'] <= thresholds['conditional_power_rmse_max_w'],
'conditional_voltage_rmse': values['conditional_voltage_rmse_v'] <= thresholds['conditional_voltage_rmse_max_v'],
'end_to_end_power_rmse': values['end_to_end_power_rmse_w'] <= thresholds['end_to_end_power_rmse_max_w'],
'end_to_end_voltage_rmse': values['end_to_end_voltage_rmse_v'] <= thresholds['end_to_end_voltage_rmse_max_v'],
'moving_pv_rmse': values['moving_pv_rmse_w'] <= thresholds['moving_pv_rmse_max_w'], 'pv_lodo_moving_rmse':
values['pv_lodo_moving_rmse_w'] <= thresholds['pv_lodo_moving_rmse_max_w'], 'pv_lodo_deployed_stop_rmse':
values['pv_lodo_deployed_stop_rmse_w'] <= thresholds['pv_lodo_deployed_stop_rmse_max_w'], 'power_residual_mean':
values['power_residual_mean_120s_rmse_w'] <= thresholds['power_residual_mean_120s_rmse_max_w'],
'energy_error_25km_rmse': values['energy_error_25km_rmse_wh'] <= thresholds['energy_error_25km_rmse_max_wh'],
'terminal_soc_evidence_spread': spread <= thresholds['terminal_soc_evidence_spread_max'], 'terminal_replay_soc':
math.isfinite(values['terminal_replay_soc_error']) and values['terminal_replay_soc_error']
<= thresholds['terminal_replay_soc_error_max'], 'terminal_replay_voltage': math.isfinite(values['terminal_replay_voltage_error_v'])
and values['terminal_replay_voltage_error_v'] <= thresholds['terminal_replay_voltage_error_max_v'], 'vehicle_terminal_soc':
math.isfinite(values['vehicle_terminal_soc_error']) and values['vehicle_terminal_soc_error']
<= thresholds['vehicle_terminal_soc_error_max'], 'vehicle_terminal_voltage': math.isfinite(values['vehicle_terminal_voltage_error_v'])
and values['vehicle_terminal_voltage_error_v'] <= thresholds['vehicle_terminal_voltage_error_max_v'],
'end_to_end_terminal_soc': math.isfinite(values['end_to_end_terminal_soc_error']) and values['end_to_end_terminal_soc_error']
<= thresholds['end_to_end_terminal_soc_error_max'], 'end_to_end_terminal_voltage': math.isfinite(values['end_to_end_terminal_voltage_error_v'])
and values['end_to_end_terminal_voltage_error_v'] <= thresholds['end_to_end_terminal_voltage_error_max_v'],
'terminal_high_precision_evidence': bool(terminal.get('high_precision_gate_pass', False)), 'terminal_anchor_role_operation':
terminal_anchor_role in {'independent_consensus', 'independent_measurement'}} の結果を代入する。
30. terminal_anchor に dict(summary.get('terminal_anchor', {}) or {}) の結果を代入する。
31. 条件 terminal_anchor を判定し、真なら内部処理を行う。
32. checks['terminal_local_anchor_quality'] に bool(terminal_anchor.get('quality_gate_pass', False)) の結果を代入する。
33. checks['terminal_independent_cross_channel_consistency'] に bool(terminal_anchor.get('weak_channel_cross_consistency_gate_pass', False)) の結果を代入する。
34. acceleration_fit に dict((summary.get('fit_plan', {}) or {}).get('acceleration_observation_fit', {}) or {}) の結果を代入する。
35. 条件 bool(acceleration_fit.get('enabled', False)) を判定し、真なら内部処理を行う。
36. validation_count に int(acceleration_fit.get('validation_sample_count', 0) or 0) の結果を代入する。
37. validation_ratio に float(acceleration_fit.get('validation_rmse_ratio', math.inf)) の結果を代入する。
38. values['acceleration_validation_sample_count'] に validation_count の結果を代入する。
39. values['acceleration_validation_rmse_ratio'] に validation_ratio の結果を代入する。
40. checks['acceleration_timestamp_holdout'] に bool(validation_count >= thresholds['acceleration_validation_min_samples'] and math.isfinite(validation_ratio) and (validation_ratio <= thresholds['acceleration_validation_rmse_ratio_max'])) の結果を代入する。
41. grade_fit に dict((summary.get('fit_plan', {}) or {}).get('grade_observation_fit', {}) or {}) の結果を代入する。
42. 条件 bool(grade_fit.get('enabled', False)) を判定し、真なら内部処理を行う。
43. validation_count に int(grade_fit.get('validation_sample_count', 0) or 0) の結果を代入する。
44. validation_ratio に float(grade_fit.get('validation_rmse_ratio', math.inf)) の結果を代入する。
45. values['grade_validation_sample_count'] に validation_count の結果を代入する。

46. values['grade_validation_rmse_ratio'] に validation_ratio の結果を代入する。
47. values['grade_observation_adopted'] に bool(grade_fit.get('adopted', False)) の結果を代入する。
48. checks['grade_observation_holdout'] に bool(validation_count >= thresholds['grade_validation_min_samples'] and math.isfinite(validation_ratio) and (validation_ratio <= thresholds['grade_validation_rmse_ratio_max'])) の結果を代入する。
49. result.update(...) を実行する。
50. result を返す。

代表コード断片

```
def evaluate_model_validation_gate(cfg, profile_path):
    identification = cfg.get('identification', {}) if isinstance(cfg, dict) else {}
    gate_cfg = identification.get('validation_gate', {}) if isinstance(identification, dict)
    else {}
    if not isinstance(gate_cfg, dict):
        gate_cfg = {}
    fit_summary_path = resolve_config_path(profile_path, identification.get('
fit_summary_yaml', ''))
    terminal_path = resolve_config_path(profile_path, identification.get('
terminal_consistency_yaml', ''))
    result = {
        'available': False,
        'fit_summary_yaml': fit_summary_path,
        'terminal_consistency_yaml': terminal_path,
        'gate_pass': False,
        'checks': {},
        'thresholds': {},
    }
    if not fit_summary_path or not Path(fit_summary_path).exists():
        result['reason'] = 'fit summary is not configured or does not exist'
        return result
    summary = load_yaml(fit_summary_path)
    metrics = summary.get('validation_metrics', {}) if isinstance(summary, dict) else {}
    thresholds = {
        'vehicle_power_rmse_max_w': float(gate_cfg.get('vehicle_power_rmse_max_w', 150.0)),
        'vehicle_voltage_rmse_max_v': float(gate_cfg.get('vehicle_voltage_rmse_max_v', 1.0))
    },
    'conditional_power_rmse_max_w': float(gate_cfg.get('conditional_power_rmse_max_w',
150.0)),
    'conditional_voltage_rmse_max_v': float(gate_cfg.get('conditional_voltage_rmse_max_v
', 1.0)),
    'end_to_end_power_rmse_max_w': float(gate_cfg.get('end_to_end_power_rmse_max_w',
200.0)),
    'end_to_end_voltage_rmse_max_v': float(gate_cfg.get('end_to_end_voltage_rmse_max_v',
2.0)),
    'moving_pv_rmse_max_w': float(gate_cfg.get('moving_pv_rmse_max_w', 150.0)),
    'pv_lodo_moving_rmse_max_w': float(gate_cfg.get('pv_lodo_moving_rmse_max_w', 150.0))
    },
    'pv_lodo_deployed_stop_rmse_max_w': float(
        gate_cfg.get('pv_lodo_deployed_stop_rmse_max_w', 200.0)
    ),
    'power_residual_mean_120s_rmse_max_w': float(
        gate_cfg.get('power_residual_mean_120s_rmse_max_w', 150.0)
    ),
    ...
```

7.37 L1032 関数 get_profile_val

- 行範囲: L1032–L1038

- 所属: module 直下
- 定義: `get_profile_val(df, s_km, field, default=0.0)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `interpolate_profile`, `isfinite`
- 戻り値の要点: `float(val) / float(default) / float(default)`
- 主なローカル代入名: `val`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `df is None or field not in df.columns` を判定し、真なら内部処理を行う。
2. `float(default)` を返す。
3. `val` に `float(interpolate_profile(df, s_km, field, default))` の結果を代入する。
4. 条件 `not math.isfinite(val)` を判定し、真なら内部処理を行う。
5. `float(default)` を返す。
6. `float(val)` を返す。

代表コード断片

```
def get_profile_val(df, s_km, field, default=0.0):
    if df is None or field not in df.columns:
        return float(default)
    val = float(interpolate_profile(df, s_km, field, default))
    if not math.isfinite(val):
        return float(default)
    return float(val)
```

7.38 L1040 関数 `parse_utc_arg`

- 行範囲: L1040–L1049
- 所属: module 直下
- 定義: `parse_utc_arg(raw_value:str)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astimezone`, `endswith`, `fromisoformat`, `replace`, `str`, `strip`
- 戻り値の要点: `dt.astimezone(timezone.utc) / None`

- 主なローカル代入名: `dt`, `text`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `text` に `str(raw_value or '').strip()` の結果を代入する。
2. 条件 `not text` を判定し、真なら内部処理を行う。
3. `None` を返す。
4. 条件 `text.endswith('Z')` を判定し、真なら内部処理を行う。
5. `text` に `text[:-1] + '+00:00'` の結果を代入する。
6. `dt` に `datetime.fromisoformat(text)` の結果を代入する。
7. 条件 `dt.tzinfo is None` を判定し、真なら内部処理を行う。
8. `dt` に `dt.replace(tzinfo=timezone.utc)` の結果を代入する。
9. `dt.astimezone(timezone.utc)` を返す。

代表コード断片

```
def parse_utc_arg(raw_value: str):
    text = str(raw_value or '').strip()
    if not text:
        return None
    if text.endswith('Z'):
        text = text[:-1] + '+00:00'
    dt = datetime.fromisoformat(text)
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=timezone.utc)
    return dt.astimezone(timezone.utc)
```

7.39 L1052 関数 `resolve_forecast_mode`

- 行範囲: L1052–L1061
- 所属: `module` 直下
- 定義: `resolve_forecast_mode(mode:str, df:pd.DataFrame) -> str`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `all`, `isna`, `lower`, `str`, `strip`
- 戻り値の要点: `raw / 'absolute' if has_time else 'relative' / 'relative' / 'relative'`
- 主なローカル代入名: `has_time`, `raw`

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `raw` に `str(mode or 'auto').strip().lower()` の結果を代入する。
2. `has_time` に `'time' in df.columns and (not df['time'].isna().all())` の結果を代入する。
3. 条件 `raw == 'auto'` を判定し、真なら内部処理を行う。
4. `'absolute' if has_time else 'relative'` を返す。
5. 条件 `raw not in ('absolute', 'relative', 'loop')` を判定し、真なら内部処理を行う。
6. `'relative'` を返す。
7. 条件 `raw == 'absolute' and (not has_time)` を判定し、真なら内部処理を行う。
8. `'relative'` を返す。
9. `raw` を返す。

代表コード断片

```
def resolve_forecast_mode(mode: str, df: pd.DataFrame) -> str:
    raw = str(mode or 'auto').strip().lower()
    has_time = ('time' in df.columns) and (not df['time'].isna().all())
    if raw == 'auto':
        return 'absolute' if has_time else 'relative'
    if raw not in ('absolute', 'relative', 'loop'):
        return 'relative'
    if raw == 'absolute' and not has_time:
        return 'relative'
    return raw
```

7.40 L1064 関数 `forecast_row_index`

- 行範囲: L1064–L1079
- 所属: module 直下
- 定義: `forecast_row_index(df:pd.DataFrame,sim_t:datetime,*,dt_sec:float,mode:str,forecast_start_time:datetime)->int`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `all`, `clip`, `int`, `isna`, `len`, `max`, `searchsorted`, `total_seconds`
- 戻り値の要点: `int(np.clip(idx, 0, len(df) - 1)) / 0 / int(idx % len(df)) / int(np.clip(idx, 0, len(df) - 1))`
- 主なローカル代入名: `elapsed`, `idx`, `mode`, `t_max`, `t_min`

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `len(df) == 0` を判定し、真なら内部処理を行う。
2. 0 を返す。
3. 条件 `mode == 'absolute' and 'time' in df.columns and (not df['time'].isna().all())` を判定し、真なら内部処理を行う。
4. `t_min` に `df['time'].iloc[0]` の結果を代入する。
5. `t_max` に `df['time'].iloc[-1]` の結果を代入する。
6. 条件 `t_min <= sim_t <= t_max` を判定し、真なら内部処理を行う。
7. `idx` に `int(df['time'].searchsorted(pd.Timestamp(sim_t), side='right') - 1)` の結果を代入する。
8. `int(np.clip(idx, 0, len(df) - 1))` を返す。
9. `mode` に `'relative'` の結果を代入する。
10. `elapsed` に `max(0.0, (sim_t - forecast_start_time).total_seconds())` の結果を代入する。
11. `idx` に `int(elapsed / max(dt_sec, 0.001))` の結果を代入する。
12. 条件 `mode == 'loop'` を判定し、真なら内部処理を行う。
13. `int(idx % len(df))` を返す。
14. `int(np.clip(idx, 0, len(df) - 1))` を返す。

代表コード断片

```
def forecast_row_index(df: pd.DataFrame, sim_t: datetime, *, dt_sec: float, mode: str,
forecast_start_time: datetime) -> int:
    if len(df) == 0:
        return 0
    if mode == 'absolute' and 'time' in df.columns and not df['time'].isna().all():
        t_min = df['time'].iloc[0]
        t_max = df['time'].iloc[-1]
        if t_min <= sim_t <= t_max:
            idx = int(df['time'].searchsorted(pd.Timestamp(sim_t), side='right') - 1)
            return int(np.clip(idx, 0, len(df) - 1))
        mode = 'relative'

    elapsed = max(0.0, (sim_t - forecast_start_time).total_seconds())
    idx = int(elapsed / max(dt_sec, 1.0e-3))
    if mode == 'loop':
        return int(idx % len(df))
    return int(np.clip(idx, 0, len(df) - 1))
```

7.41 L1082 関数 `format_metric`

- 行範囲: L1082–L1091
- 所属: module 直下
- 定義: `format_metric(value,digits=3,default='--')`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`
- 戻り値の要点: `default / default / f'{fval:.{digits}f}'`
- 主なローカル代入名: `fval`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 1

この定義を読むための Python 構文

- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `value is None` を判定し、真なら内部処理を行う。
2. `default` を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `fval` に `float(value)` の結果を代入する。
5. 条件 `math.isfinite(fval)` を判定し、真なら内部処理を行う。
6. `f'{fval:.{digits}f}'` を返す。
7. `Exception` を捕捉した場合:
8. `Pass` 文を実行する。
9. `default` を返す。

代表コード断片

```
def format_metric(value, digits=3, default='--'):
    if value is None:
        return default
    try:
        fval = float(value)
        if math.isfinite(fval):
            return f'{fval:.{digits}f}'
    except Exception:
        pass
    return default
```

7.42 L1094 関数 `decimate_xy`

- 行範囲: L1094–L1102
- 所属: module 直下
- 定義: `decimate_xy(xs,ys,max_points=600)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `ceil`, `float`, `int`, `isfinite`, `len`, `max`, `zip`
- 戻り値の要点: `reduced` / `pts`
- 主なローカル代入名: `pts`, `reduced`, `step`, `x`, `y`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `pts` に `[(float(x), float(y)) for x, y in zip(xs, ys) if math.isfinite(float(y))]` の結果を代入する。
2. 条件 `len(pts) <= max_points` を判定し、真なら内部処理を行う。
3. `pts` を返す。
4. `step` に `max(1, int(math.ceil(len(pts) / max_points)))` の結果を代入する。
5. `reduced` に `pts[::step]` の結果を代入する。
6. 条件 `reduced[-1] != pts[-1]` を判定し、真なら内部処理を行う。
7. `reduced.append(...)` を実行する。
8. `reduced` を返す。

代表コード断片

```
def decimate_xy(xs, ys, max_points=600):
    pts = [(float(x), float(y)) for x, y in zip(xs, ys) if math.isfinite(float(y))]
    if len(pts) <= max_points:
        return pts
    step = max(1, int(math.ceil(len(pts) / max_points)))
    reduced = pts[::step]
    if reduced[-1] != pts[-1]:
        reduced.append(pts[-1])
    return reduced
```

7.43 L1105 関数 build_svg_chart

- 行範囲: L1105–L1143
- 所属: module 直下
- 定義: `build_svg_chart(xs,ys,*,color='#135d66',width=920,height=220,pad=26,label='')`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `append`, `decimate_xy`, `escape`, `format_metric`, `join`, `max`, `min`
- 戻り値の要点: `f'<svg viewBox="0 0 {width} {height}" class="chart-svg" role="img" aria-label="{html.escape(label)}">x="0" y="0" width="{width}" height="{height}" rx="18" ry="18" fill="#ffdf8" stroke="#d8d1c4" /><line x1="{pad}" y1="{pad}" x2="{pad}" y2="{height - pad}" stroke="#a9a293" stroke-width="1" /><line x1="{pad}" y1="{height - pad}" x2="{width - pad}" y2="{height - pad}" stroke="#a9a293" stroke-width="1" /><polyline fill="none" stroke="{color}" stroke-width="2.6" points="{polyline}" /><text x="{pad}" y="18" font-size="12" fill="#50483f">{html.escape(label)}</text><text x="{width - pad}" y="18" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(y_max, 2)}</text><text x="{width - pad}" y="{height - 8}" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(x_max, 1)}</text><text x="{pad}" y="{height - 8}" font-size="11" fill="#6f665b">{format_metric(x_min, 1)}</text><text x="{width - pad}" y="{height / 2:.1f}" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(y_mid, 2)}</text><text x="{width - pad}" y="{height - pad + 16}" text-anchor="end" font-size="11" fill="#6f665b">x</text></svg>' /><div class="chart-empty">no data</div>`
- 主なローカル代入名: `coords`, `inner_h`, `inner_w`, `p`, `polyline`, `pts`, `span`, `sx`, `sy`, `x_max`, `x_min`, `x_val`, `x_vals`, `y_max`, `y_mid`, `y_min`, `y_val`, `y_vals`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、`try 0`

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `pts` に `decimate_xy(xs, ys, max_points=700)` の結果を代入する。
2. 条件 `not pts` を判定し、真なら内部処理を行う。
3. `'<div class="chart-empty">no data</div>'` を返す。
4. `x_vals` に `[p[0] for p in pts]` の結果を代入する。
5. `y_vals` に `[p[1] for p in pts]` の結果を代入する。
6. `x_min` に `min(x_vals)` の結果を代入する。
7. `x_max` に `max(x_vals)` の結果を代入する。
8. `y_min` に `min(y_vals)` の結果を代入する。
9. `y_max` に `max(y_vals)` の結果を代入する。

10. 条件 $x_max \leq x_min$ を判定し、真なら内部処理を行う。
11. x_max に $x_min + 1.0$ の結果を代入する。
12. 条件 $y_max \leq y_min$ を判定し、真なら内部処理を行う。
13. $span$ に 1.0 if $abs(y_max) < 1.0$ else $abs(y_max) * 0.1$ の結果を代入する。
14. y_min を Sub で更新する。
15. y_max を Add で更新する。
16. $inner_w$ に $\max(10.0, width - 2 * pad)$ の結果を代入する。
17. $inner_h$ に $\max(10.0, height - 2 * pad)$ の結果を代入する。
18. $coords$ に [] の結果を代入する。
19. pts を順に走査し、各要素を (x_val, y_val) に入れて処理する。
20. sx に $pad + (x_val - x_min) / (x_max - x_min) * inner_w$ の結果を代入する。
21. sy に $height - pad - (y_val - y_min) / (y_max - y_min) * inner_h$ の結果を代入する。
22. $coords.append(...)$ を実行する。
23. $polyline$ に `' '.join(coords)` の結果を代入する。
24. y_mid に $0.5 * (y_min + y_max)$ の結果を代入する。
25. `f<svg viewBox="0 0 {width} {height}" class="chart-svg" role="img" aria-label="{html.escape(label)}"><rect x="0" y="0" width="{width}" height="{height}" rx="18" ry="18" fill="#fffd8" stroke="#d8d1c4" /><line x1="{pad}" y1="{pad}" x2="{pad}" y2="{height - pad}" stroke="#a9a293" stroke-width="1" /><line x1="{pad}" y1="{height - pad}" x2="{width - pad}" y2="{height - pad}" stroke="#a9a293" stroke-width="1" /><polyline fill="none" stroke="{color}" stroke-width="2.6" points="{polyline}" /><text x="{pad}" y="18" font-size="12" fill="#50483f">{html.escape(label)}</text><text x="{width - pad}" y="18" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(y_max, 2)}</text><text x="{width - pad}" y="{height - 8}" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(x_max, 1)}</text><text x="{pad}" y="{height - 8}" font-size="11" fill="#6f665b">{format_metric(x_min, 1)}</text><text x="{width - pad}" y="{height / 2:.1f}" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(y_mid, 2)}</text><text x="{width - pad}" y="{height - pad + 16}" text-anchor="end" font-size="11" fill="#6f665b">x</text></svg>` を返す。

代表コード断片

```
def build_svg_chart(xs, ys, *, color='#135d66', width=920, height=220, pad=26, label=''):
    pts = decimate_xy(xs, ys, max_points=700)
    if not pts:
        return '<div class="chart-empty">no data</div>'
    x_vals = [p[0] for p in pts]
    y_vals = [p[1] for p in pts]
    x_min = min(x_vals)
    x_max = max(x_vals)
    y_min = min(y_vals)
    y_max = max(y_vals)
    if x_max <= x_min:
        x_max = x_min + 1.0
    if y_max <= y_min:
        span = 1.0 if abs(y_max) < 1.0 else abs(y_max) * 0.1
        y_min -= span
        y_max += span
    inner_w = max(10.0, width - 2 * pad)
    inner_h = max(10.0, height - 2 * pad)
    coords = []
```

```

for x_val, y_val in pts:
    sx = pad + (x_val - x_min) / (x_max - x_min) * inner_w
    sy = height - pad - (y_val - y_min) / (y_max - y_min) * inner_h
    coords.append(f'{sx:.2f},{sy:.2f}')
polyline = ' '.join(coords)
y_mid = 0.5 * (y_min + y_max)
return (
    f'<svg viewBox="0 0 {width} {height}" class="chart-svg" role="img" aria-label="{html
.escape(label)}">'
    f'<rect x="0" y="0" width="{width}" height="{height}" rx="18" ry="18" fill="#fffd8"
stroke="#d8d1c4" />'
    f'<line x1="{pad}" y1="{pad}" x2="{pad}" y2="{height - pad}" stroke="#a9a293" stroke
-width="1" />'
    f'<line x1="{pad}" y1="{height - pad}" x2="{width - pad}" y2="{height - pad}" stroke
="#a9a293" stroke-width="1" />'
    f'<polyline fill="none" stroke="{color}" stroke-width="2.6" points="{polyline}" />'
    f'<text x="{pad}" y="18" font-size="12" fill="#50483f">{html.escape(label)}</text>'
    f'<text x="{width - pad}" y="18" text-anchor="end" font-size="11" fill="#6f665b">{
format_metric(y_max, 2)}</text>'
    f'<text x="{width - pad}" y="{height - 8}" text-anchor="end" font-size="11" fill="#6
f665b">{format_metric(x_max, 1)}</text>'
    f'<text x="{pad}" y="{height - 8}" font-size="11" fill="#6f665b">{format_metric(
x_min, 1)}</text>'
    ...

```

7.44 L1146 関数 `flatten_params_for_report`

- 行範囲: L1146–L1163
- 所属: module 直下
- 定義: `flatten_params_for_report(cfg)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `get`, `isinstance`, `items`, `str`, `visit`
- 戻り値の要点: `rows / rows`
- 主なローカル代入名: `child`, `child_prefix`, `key`, `rows`, `section_name`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 2、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `rows` に `[]` の結果を代入する。
2. 関数 `visit` を定義する。
3. 条件 `not isinstance(cfg, dict)` を判定し、真なら内部処理を行う。

4. rows を返す。
5. ('simulation', 'model', 'mpc', 'runtime') を順に走査し、各要素を section_name に入れて処理する。
6. visit(...) を実行する。
7. rows を返す。

代表コード断片

```
def flatten_params_for_report(cfg):
    rows = []

    def visit(prefix, value):
        if isinstance(value, dict):
            for key, child in value.items():
                child_prefix = f'{prefix}.{key}' if prefix else str(key)
                visit(child_prefix, child)
            return
        if isinstance(value, list):
            return
        rows.append((prefix, value))

    if not isinstance(cfg, dict):
        return rows
    for section_name in ('simulation', 'model', 'mpc', 'runtime'):
        visit(section_name, cfg.get(section_name, {}))
    return rows
```

7.45 L1149 関数 flatten_params_for_report.visit

- 行範囲: L1149–L1157
- 所属: flatten_params_for_report
- 定義: visit(prefix, value)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: append, isinstance, items, str, visit
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: child, child_prefix, key
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `isinstance(value, dict)` を判定し、真なら内部処理を行う。
2. `value.items()` を順に走査し、各要素を `(key, child)` に入れて処理する。
3. `child_prefix` に `f'{prefix}.{key}' if prefix else str(key)` の結果を代入する。
4. `visit(...)` を実行する。
5. を返す。
6. 条件 `isinstance(value, list)` を判定し、真なら内部処理を行う。
7. を返す。
8. `rows.append(...)` を実行する。

代表コード断片

```
def visit(prefix, value):
    if isinstance(value, dict):
        for key, child in value.items():
            child_prefix = f'{prefix}.{key}' if prefix else str(key)
            visit(child_prefix, child)
        return
    if isinstance(value, list):
        return
    rows.append((prefix, value))
```

7.46 L1166 関数 `write_simulation_report`

- 行範囲: L1166–L1286
- 所属: module 直下
- 定義: `write_simulation_report(path, summary, detail_df, params_rows)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Series`, `append`, `build_svg_chart`, `dumps`, `ensure_parent_dir`, `escape`, `extend`, `format_metric`, `get`, `join`, `len`, `list`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `cards_html`, `charts`, `charts_html`, `f`, `html_text`, `item`, `key`, `label`, `overrides_html`, `pack_svg`, `params_html`, `slope_svg`, `soc_svg`, `solar_svg`, `speed_cmd_svg`, `speed_exec_present`, `speed_series`, `speed_svg`, `summary_cards`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `ensure_parent_dir(...)` を実行する。
2. `x_index` に `list(range(len(detail_df)))` の結果を代入する。
3. `speed_exec_present` に `'v_exec_kmh' in detail_df.columns` の結果を代入する。
4. `speed_series` に `detail_df.get('v_exec_kmh', detail_df.get('v_cmd_kmh', pd.Series(dtype=float)))` の結果を代入する。
5. `speed_svg` に `build_svg_chart(x_index, speed_series, color='#0f766e', label='speed exec [km/h]' if speed_exec_present else 'speed cmd [km/h]')` の結果を代入する。
6. `speed_cmd_svg` に” の結果を代入する。
7. 条件 `speed_exec_present` を判定し、真なら内部処理を行う。
8. `speed_cmd_svg` に `build_svg_chart(x_index, detail_df.get('v_cmd_kmh', pd.Series(dtype=float)), color='#475569', label='speed cmd [km/h]')` の結果を代入する。
9. `soc_svg` に `build_svg_chart(x_index, detail_df.get('soc', pd.Series(dtype=float)), color='#b45309', label='soc [-]')` の結果を代入する。
10. `pack_svg` に `build_svg_chart(x_index, detail_df.get('P_pack', pd.Series(dtype=float)), color='#b91c1c', label='pack power [W]')` の結果を代入する。
11. `solar_svg` に `build_svg_chart(x_index, detail_df.get('P_pv', pd.Series(dtype=float)), color='#2563eb', label='pv power [W]')` の結果を代入する。
12. `wind_svg` に `build_svg_chart(x_index, detail_df.get('headwind_ms', pd.Series(dtype=float)), color='#7c3aed', label='headwind [m/s]')` の結果を代入する。
13. `slope_svg` に `build_svg_chart(x_index, detail_df.get('slope_pct', pd.Series(dtype=float)), color='#57534e', label='slope [%]')` の結果を代入する。
14. `summary_cards` に `[('Finish reached', 'yes' if summary.get('finish_reached') else 'no'), ('Final distance [km]', format_metric(summary.get('final_distance_kmh'), 1)), ('Race progress [%]', format_metric(summary.get('race_progress'), 1)), ('Final SoC [-]', format_metric(summary.get('final_soc'), 3)), ('Min SoC [-]', format_metric(summary.get('min_soc'), 3)), ('Avg speed [km/h]', format_metric(summary.get('avg_speed_kmh'), 1)), ('Elapsed [h]', format_metric(summary.get('elapsed_h'), 2)), ('Exec model', 'on' if summary.get('execution_model_enabled') else 'off'), ('Mean |v_cmd-v_exec| [km/h]', format_metric(summary.get('mean_tracking_error_kmh'), 2)), ('P95 |v_cmd-v_exec| [km/h]', format_metric(summary.get('p95_tracking_error_kmh'), 2)), ('Forecast mode', str(summary.get('forecast_mode', '-'))), ('Overrides', str(summary.get('override_count', 0)))]` の結果を代入する。
15. `overrides_html` に `”.join((f’<tr><td>{html.escape(str(item.get(’key’)))}</td><td><code>{html.escape(json.dumps(item.get(’overrides’, [])))}</code></td></tr>’ for item in summary.get(’overrides’, [])]) or ’<tr><td colspan=`”2”>none</td></tr>” の結果を代入する。
16. `params_html` に `”.join((f’<tr><td>{html.escape(str(key))}</td><td>{html.escape(str(value))}</td></tr>’ for key, value in params_rows))` の結果を代入する。
17. `cards_html` に `”.join((f’<div class=`”metric”><div class=”metric-label”>{html.escape(label)}</div><div class=”metric-value”>{html.escape(str(value))}</div></div>’ for label, value in summary_cards)) の結果を代入する。
18. `charts` に `[speed_svg]` の結果を代入する。
19. 条件 `speed_cmd_svg` を判定し、真なら内部処理を行う。
20. `charts.append(...)` を実行する。

21. charts.extend(...) を実行する。
22. charts_html に".join(charts) の結果を代入する。
23. warning_text に html.escape(str(summary.get('warning', ''))) の結果を代入する。
24. html_text に f"""<!doctype html>\n<html lang="en">\n<head>\n <meta charset="utf-8">\n <title>Solar Simulation Report</title>\n <style>\n body {{ margin: 0; font-family: "Segoe UI", sans-serif; background: #f5efe3; color: #1f2933; }}\n .wrap {{ max-width: 1180px; margin: 0 auto; padding: 24px; }}\n .hero {{ background: linear-gradient(135deg, #ffffa0, #efe6d2); border: 1px solid #d9cfbd; border-radius: 24px; padding: 22px; }}\n h1, h2 {{ margin: 0 0 12px; }}\n p {{ margin: 8px 0; }}\n .muted {{ color: #625a4e; }}\n .grid {{ display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 12px; margin-top: 18px; }}\n .metric {{ background: #fffd8; border: 1px solid #ddd2be; border-radius: 18px; padding: 14px; }}\n .metric-label {{ font-size: 12px; color: #6f665b; text-transform: uppercase; letter-spacing: 0.06em; }}\n .metric-value {{ margin-top: 6px; font-size: 24px; font-weight: 700; }}\n .section {{ margin-top: 20px; background: #fffaf2; border: 1px solid #ddd2be; border-radius: 24px; padding: 18px; }}\n .charts {{ display: grid; grid-template-columns: 1fr; gap: 16px; }}\n .chart-svg {{ width: 100%; height: auto; display: block; }}\n table {{ width: 100%; border-collapse: collapse; }}\n th, td {{ border-bottom: 1px solid #e6ddcf; padding: 8px 10px; text-align: left; vertical-align: top; }}\n th {{ font-size: 12px; color: #6f665b; text-transform: uppercase; letter-spacing: 0.06em; }}\n code {{ font-family: Consolas, monospace; font-size: 12px; }}\n .warning {{ margin-top: 10px; padding: 12px; border-radius: 14px; background: #fff0e1; color: #8a3b12; }}\n </style>\n</head>\n<body>\n <div class="wrap">\n <div class="hero">\n <h1>Solar Simulation Report</h1>\n <p class="muted">profile: {html.escape(str(summary.get('profile_name', '-')))} | generated: {html.escape(str(summary.get('generated_at_utc', '-')))}</p>\n <p class="muted">csv: {html.escape(str(summary.get('out_csv', '-')))} | detail: {html.escape(str(summary.get('detail_csv', '-')))}</p>\n {f"<div class="warning">{warning_text}</div>" if warning_text else ""}\n <div class="grid">{cards_html}</div>\n <div class="section">\n <h2>Key Charts</h2>\n <div class="charts">\n {charts_html}\n </div>\n </div>\n <div class="section">\n <h2>Overrides</h2>\n <table>\n <thead><tr><th>key</th><th>value</th></tr></thead>\n <tbody>{overrides_html}</tbody>\n </table>\n </div>\n <div class="section">\n <h2>Resolved Parameters</h2>\n <table>\n <thead><tr><th>parameter</th><th>value</th></tr></thead>\n <tbody>{params_html}</tbody>\n </table>\n </div>\n </div>\n</body>\n</html>\n""" の結果を代入する。
25. with 文で open(path, 'w', encoding='utf-8') を管理しながら処理する。
26. f.write(...) を実行する。

代表コード断片

```
def write_simulation_report(path, summary, detail_df, params_rows):
    ensure_parent_dir(path)
    x_index = list(range(len(detail_df)))
    speed_exec_present = 'v_exec_kmh' in detail_df.columns
    speed_series = detail_df.get('v_exec_kmh', detail_df.get('v_cmd_kmh', pd.Series(dtype=float)))
    speed_svg = build_svg_chart(
        x_index,
        speed_series,
        color='#0f766e',
        label='speed exec [km/h]' if speed_exec_present else 'speed cmd [km/h]',
    )
    speed_cmd_svg = ''
    if speed_exec_present:
        speed_cmd_svg = build_svg_chart(
            x_index,
            detail_df.get('v_cmd_kmh', pd.Series(dtype=float)),
            color='#475569',
            label='speed cmd [km/h]',
        )
    soc_svg = build_svg_chart(x_index, detail_df.get('soc', pd.Series(dtype=float)), color='#b45309', label='soc [-]')
```

```

pack_svg = build_svg_chart(x_index, detail_df.get('P_pack', pd.Series(dtype=float)),
color='#b91c1c', label='pack power [W]')
solar_svg = build_svg_chart(x_index, detail_df.get('P_pv', pd.Series(dtype=float)),
color='#2563eb', label='pv power [W]')
wind_svg = build_svg_chart(x_index, detail_df.get('headwind_ms', pd.Series(dtype=float))
, color='#7c3aed', label='headwind [m/s]')
slope_svg = build_svg_chart(x_index, detail_df.get('slope_pct', pd.Series(dtype=float)),
color='#57534e', label='slope [%]')

summary_cards = [
    ('Finish reached', 'yes' if summary.get('finish_reached') else 'no'),
    ('Final distance [km]', format_metric(summary.get('final_distance_km'), 1)),
    ('Race progress [%]', format_metric(summary.get('race_progress_pct'), 1)),
    ('Final SoC [-]', format_metric(summary.get('final_soc'), 3)),
    ('Min SoC [-]', format_metric(summary.get('min_soc'), 3)),
    ('Avg speed [km/h]', format_metric(summary.get('avg_speed_kmh'), 1)),
    ('Elapsed [h]', format_metric(summary.get('elapsed_hours'), 2)),
    ('Exec model', 'on' if summary.get('execution_model_enabled') else 'off'),
    ('Mean |v_cmd-v_exec| [km/h]', format_metric(summary.get('mean_tracking_error_kmh'),
2)),
...

```

7.47 L1288 関数 mpc_solve

- 行範囲: L1288–L1399
- 所属: module 直下
- 定義: `mpc_solve(model, data, z0, Tb0, s0_kmh, v0_kmh, v_max_kmh, term_soc_min, w_dv, w_dv_limit, dv_max_kmhps, w_T, w_speed_limit, w_current, speed_profile, soc_target, soc_band, w_soc_target, w_soc_band, schedule, soc_day_end_max, w_soc_day_max, soc_finish_target, soc_finish_tol, w_soc_progress, w_soc_terminal, race_kmh, soc_day_end_target, soc_day_end_tol, w_soc_day_track)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `array`, `clip`, `current_drive_window`, `dict`, `electrical_balance`, `float`, `get`, `get_profile_val`, `is_drive_time`, `len`, `list`
- 戻り値の要点: `float(np.clip(v0_kmh, 0.0, v_max_kmh)) / v0_kmh / x * x / J`
- 主なローカル代入名: `I`, `J`, `Np`, `P_pack`, `Tb`, `Tb_next`, `V`, `bounds`, `d`, `dv`, `dv_max_msps`, `inertial_power_w`, `k`, `lb`, `loss_int`, `out`, `p`, `prog`, `res`, `s_kmh`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 14、ループ 1、`try 0`

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `p` に `model.p` の結果を代入する。
2. `Np` に `len(data)` の結果を代入する。
3. 条件 `Np <= 0` を判定し、真なら内部処理を行う。
4. `v0_kmh` を返す。
5. `v0_ms` に `v0_kmh / 3.6` の結果を代入する。
6. `x0` に `np.array([v0_ms] * Np, dtype=float)` の結果を代入する。
7. `lb` に `np.zeros(Np, dtype=float)` の結果を代入する。
8. `ub` に `np.ones(Np, dtype=float) * (v_max_kmh / 3.6)` の結果を代入する。
9. `dv_max_msps` に `dv_max_kmhps / 3.6` の結果を代入する。
10. 関数 `quad_penalty` を定義する。
11. 関数 `cost` を定義する。
12. `ImportFrom` 文を実行する。
13. `bounds` に `list(zip(lb, ub))` の結果を代入する。
14. `res` に `minimize(cost, x0, method='L-BFGS-B', bounds=bounds, options=dict(maxiter=150))` の結果を代入する。
15. `x_best` に `select_optimized_vector(res, x0, label='lower solve')` の結果を代入する。
16. `v0_kmh` に `float(x_best[0]) * 3.6` の結果を代入する。
17. `float(np.clip(v0_kmh, 0.0, v_max_kmh))` を返す。

代表コード断片

```
def mpc_solve(model, data, z0, Tb0, s0_kmh, v0_kmh, v_max_kmh, term_soc_min,
              w_dv, w_dv_limit, dv_max_kmhps, w_T, w_speed_limit, w_current, speed_profile,
              soc_target, soc_band, w_soc_target, w_soc_band, schedule, soc_day_end_max,
              w_soc_day_max,
              soc_finish_target, soc_finish_tol, w_soc_progress, w_soc_terminal, race_kmh,
              soc_day_end_target, soc_day_end_tol, w_soc_day_track):
    p = model.p
    Np = len(data)
    if Np <= 0:
        return v0_kmh

    v0_ms = v0_kmh / 3.6
    x0 = np.array([v0_ms] * Np, dtype=float)
    lb = np.zeros(Np, dtype=float)
    ub = np.ones(Np, dtype=float) * (v_max_kmh / 3.6)

    dv_max_msps = dv_max_kmhps / 3.6

    def quad_penalty(x, cap=1.0e3):
        if x <= 0.0:
            return 0.0
        if x > cap:
            x = cap
        return x * x

    def cost(v):
```

```

z = float(z0)
Tb = float(Tb0)
s_km = float(s0_km)
v_prev = float(v0_ms)
J = 0.0
for k in range(Np):
    d = data[k]
    v_k = float(v[k])
    inertial_power_w = 0.5 * p.m * (v_k * v_k - v_prev * v_prev) / max(p.dt, 1.0e-9)
    out = model.electrical_balance(
...

```

7.48 L1305 関数 mpc_solve.quad_penalty

- 行範囲: L1305–L1310
- 所属: mpc_solve
- 定義: quad_penalty(x, cap=1000.0)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: $x * x / 0.0$
- 主なローカル代入名: x
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 $x \leq 0.0$ を判定し、真なら内部処理を行う。
2. 0.0 を返す。
3. 条件 $x > \text{cap}$ を判定し、真なら内部処理を行う。
4. x に cap の結果を代入する。
5. $x * x$ を返す。

代表コード断片

```

def quad_penalty(x, cap=1.0e3):
    if x <= 0.0:
        return 0.0
    if x > cap:
        x = cap
    return x * x

```

7.49 L1312 関数 `mpc_solve.cost`

- 行範囲: L1312–L1392
- 所属: `mpc_solve`
- 定義: `cost(v)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `current_drive_window`, `electrical_balance`, `float`, `get`, `get_profile_val`, `is_drive_time`, `max`, `min`, `quad_penalty`, `range`, `soc_step`
- 戻り値の要点: J
- 主なローカル代入名: I, J, P_pack, Tb, Tb_next, V, d, dv, inertial_power_w, k, loss_int, out, prog, s_km, soc_line, t_end, t_next, t_start, v_k, v_prev
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 11、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. z に `float(z0)` の結果を代入する。
2. Tb に `float(Tb0)` の結果を代入する。
3. s_km に `float(s0_km)` の結果を代入する。
4. v_prev に `float(v0_ms)` の結果を代入する。
5. J に 0.0 の結果を代入する。
6. `range(Np)` を順に走査し、各要素を k に入れて処理する。
7. d に `data[k]` の結果を代入する。
8. v_k に `float(v[k])` の結果を代入する。
9. `inertial_power_w` に `0.5 * p.m * (v_k * v_k - v_prev * v_prev) / max(p.dt, 1e-09)` の結果を代入する。
10. out に `model.electrical_balance(v_k, d['slope_pct'], z, Tb, d['G_poa'], d['Tcell_C'], headwind_ms=d.get('headwind_ms', 0.0), inertial_power_w=inertial_power_w, ambient_temp_c=d.get('Tamb_C'), elevation_m=d.get('elevation_m', 0.0))` の結果を代入する。
11. I に `float(out['I'])` の結果を代入する。
12. V に `float(out['V'])` の結果を代入する。
13. P_pack に `float(out['P_pack'])` の結果を代入する。
14. loss_int に `float(out['losses_int'])` の結果を代入する。
15. z_next に `model.soc_step(z, P_pack, p.dt, current_a=I, Tbat_C=Tb)` の結果を代入する。

16. Tb_next に $Tb + p.dt / 1800.0 * (d['Tamb_C'] - Tb) + loss_int * p.dt / 50000.0$ の結果を代入する。
17. s_km に $s_km + v_k * (p.dt / 1000.0)$ の結果を代入する。
18. J を Add で更新する。
19. J を Add で更新する。
20. 条件 `schedule is not None and soc_day_end_target > 0.0 and ('t_utc' in d)` を判定し、真なら内部処理を行う。
21. win に `schedule.current_drive_window(d['t_utc'])` の結果を代入する。
22. 条件 `win is not None` を判定し、真なら内部処理を行う。
23. (t_start, t_end) に win の結果を代入する。
24. 条件 $t_end > t_start$ を判定し、真なら内部処理を行う。
25. 条件 `schedule is not None and soc_day_end_max > 0.0 and ('t_utc' in d)` を判定し、真なら内部処理を行う。
26. t_next に $d['t_utc'] + timedelta(seconds=p.dt)$ の結果を代入する。
27. 条件 `schedule.is_drive_time(d['t_utc']) and (not schedule.is_drive_time(t_next))` を判定し、真なら内部処理を行う。
28. J を Add で更新する。
29. dv に $(v_k - v_prev) / \max(p.dt, 0.001)$ の結果を代入する。
30. 条件 $dv_max_msps > 0.0$ を判定し、真なら内部処理を行う。
31. J を Add で更新する。
32. $vmax_local$ に v_max_kmh の結果を代入する。
33. 条件 `speed_profile is not None` を判定し、真なら内部処理を行う。
34. $vmax_local$ に `get_profile_val(speed_profile, s_km, 'v_max_kmh', v_max_kmh)` の結果を代入する。
35. 条件 $vmax_local < v_max_kmh$ を判定し、真なら内部処理を行う。
36. J を Add で更新する。
37. J を Add で更新する。
38. J を Add で更新する。
39. J を Add で更新する。
40. J を Add で更新する。
41. J を Add で更新する。
42. J を Add で更新する。
43. J を Add で更新する。
44. J を Add で更新する。
45. (z, Tb) に (z_next, Tb_next) の結果を代入する。
46. v_prev に v_k の結果を代入する。
47. J を Add で更新する。
48. 条件 $soc_finish_target > 0.0$ を判定し、真なら内部処理を行う。
49. J を Add で更新する。
50. J を返す。

代表コード断片

```
def cost(v):
    z = float(z0)
    Tb = float(Tb0)
    s_km = float(s0_km)
    v_prev = float(v0_ms)
    J = 0.0
    for k in range(Np):
        d = data[k]
        v_k = float(v[k])
        inertial_power_w = 0.5 * p.m * (v_k * v_k - v_prev * v_prev) / max(p.dt, 1.0e-9)
        out = model.electrical_balance(
            v_k,
            d['slope_pct'],
            z,
            Tb,
            d['G_poa'],
            d['Tcell_C'],
            headwind_ms=d.get('headwind_ms', 0.0),
            inertial_power_w=inertial_power_w,
            ambient_temp_c=d.get('Tamb_C'),
            elevation_m=d.get('elevation_m', 0.0),
        )
        I = float(out['I'])
        V = float(out['V'])
        P_pack = float(out['P_pack'])
        loss_int = float(out['losses_int'])

        z_next = model.soc_step(
            z,
            P_pack,
            p.dt,
            current_a=I,
            Tbat_C=Tb,
        )
        Tb_next = Tb + (p.dt / 1800.0) * (d['Tamb_C'] - Tb) + (loss_int * p.dt) /
50000.0
    ...
```

7.50 L1402 関数 soc_guard_speed

- 行範囲: L1402–L1448
- 所属: module 直下
- 定義: `soc_guard_speed(model, v_kmh, z, Tb, s_km, d0, route_profile, mode, soc_guard)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `electrical_balance`, `float`, `get`, `get_profile_val`, `lower`, `max`, `range`, `soc_step`, `str`, `z_next_for`
- 戻り値の要点: `float(lo) / model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb) / float(lo) / v_kmh`
- 主なローカル代入名: `P_pack`, `_`, `elevation_m`, `headwind_ms`, `hi`, `lo`, `mid`, `mode`, `out`, `slope_pct`, `target`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 2、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. mode に `str(mode).lower()` の結果を代入する。
2. target に `model.p.soc_min + soc_guard` の結果を代入する。
3. slope_pct に `get_profile_val(route_profile, s_km, 'slope_pct', d0.get('slope_pct', 0.0))` の結果を代入する。
4. headwind_ms に `d0.get('headwind_ms', 0.0)` の結果を代入する。
5. elevation_m に `get_profile_val(route_profile, s_km, 'elev_m', d0.get('elevation_m', 0.0))` の結果を代入する。
6. 関数 `z_next_for` を定義する。
7. 条件 `z <= target` を判定し、真なら内部処理を行う。
8. 条件 `mode == 'stop'` を判定し、真なら内部処理を行う。
9. 0.0 を返す。
10. 条件 `mode != 'pv_only'` を判定し、真なら内部処理を行う。
11. `v_kmh` を返す。
12. lo に 0.0 の結果を代入する。
13. hi に `max(0.0, float(v_kmh))` の結果を代入する。
14. `range(20)` を順に走査し、各要素を `_` に入れて処理する。
15. mid に `0.5 * (lo + hi)` の結果を代入する。
16. 条件 `z_next_for(mid) < z` を判定し、真なら内部処理を行う。
17. hi に mid の結果を代入する。
18. 上の条件が偽の場合:
19. lo に mid の結果を代入する。
20. `float(lo)` を返す。
21. 条件 `z_next_for(v_kmh) >= target` を判定し、真なら内部処理を行う。
22. `v_kmh` を返す。
23. lo に 0.0 の結果を代入する。
24. hi に `max(0.0, float(v_kmh))` の結果を代入する。
25. `range(25)` を順に走査し、各要素を `_` に入れて処理する。
26. mid に `0.5 * (lo + hi)` の結果を代入する。
27. 条件 `z_next_for(mid) < target` を判定し、真なら内部処理を行う。
28. hi に mid の結果を代入する。
29. 上の条件が偽の場合:
30. lo に mid の結果を代入する。
31. `float(lo)` を返す。

代表コード断片

```
def soc_guard_speed(model, v_kmh, z, Tb, s_km, d0, route_profile, mode, soc_guard):
    mode = str(mode).lower()
    target = model.p.soc_min + soc_guard
    slope_pct = get_profile_val(route_profile, s_km, 'slope_pct', d0.get('slope_pct', 0.0))
    headwind_ms = d0.get('headwind_ms', 0.0)
    elevation_m = get_profile_val(route_profile, s_km, 'elev_m', d0.get('elevation_m', 0.0))

    def z_next_for(v_kmh_local):
        out = model.electrical_balance(
            v_kmh_local / 3.6, slope_pct, z, Tb, d0['G_poa'], d0['Tcell_C'],
            headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=
elevation_m,
        )
        P_pack = float(out['P_pack'])
        return model.soc_step(
            z,
            P_pack,
            model.p.dt,
            current_a=float(out['I']),
            Tbat_C=Tb,
        )

    if z <= target:
        if mode == 'stop':
            return 0.0
        if mode != 'pv_only':
            return v_kmh
        lo = 0.0
        hi = max(0.0, float(v_kmh))
        for _ in range(20):
            mid = 0.5 * (lo + hi)
            if z_next_for(mid) < z:
                hi = mid
            else:
                lo = mid
        return float(lo)
...
```

7.51 L1409 関数 soc_guard_speed.z_next_for

- 行範囲: L1409–L1421
- 所属: soc_guard_speed
- 定義: z_next_for(v_kmh_local)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: electrical_balance, float, get, soc_step
- 戻り値の要点: model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb)
- 主なローカル代入名: P_pack, out
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. out に `model.electrical_balance(v_kmh_local / 3.6, slope_pct, z, Tb, d0['G_poa'], d0['Tcell_C'], headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=elevation_m)` の結果を代入する。
2. P_pack に `float(out['P_pack'])` の結果を代入する。
3. `model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb)` を返す。

代表コード断片

```
def z_next_for(v_kmh_local):
    out = model.electrical_balance(
        v_kmh_local / 3.6, slope_pct, z, Tb, d0['G_poa'], d0['Tcell_C'],
        headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=
    elevation_m,
    )
    P_pack = float(out['P_pack'])
    return model.soc_step(
        z,
        P_pack,
        model.p.dt,
        current_a=float(out['I']),
        Tbat_C=Tb,
    )
```

7.52 L1451 関数 soc_day_guard_speed

- 行範囲: L1451–L1482
- 所属: module 直下
- 定義: `soc_day_guard_speed(model, v_kmh, z, Tb, s_kmh, d0, route_profile, target_soc, tol)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `electrical_balance`, `float`, `get`, `get_profile_val`, `max`, `range`, `soc_step`, `z_next_for`
- 戻り値の要点: `float(lo) / v_kmh / model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb) / v_kmh`
- 主なローカル代入名: `P_pack`, `_`, `elevation_m`, `headwind_ms`, `hi`, `lo`, `mid`, `out`, `slope_pct`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `target_soc <= 0.0` を判定し、真なら内部処理を行う。
2. `v_kmh` を返す。
3. `slope_pct` に `get_profile_val(route_profile, s_kmh, 'slope_pct', d0.get('slope_pct', 0.0))` の結果を代入する。
4. `headwind_ms` に `d0.get('headwind_ms', 0.0)` の結果を代入する。
5. `elevation_m` に `get_profile_val(route_profile, s_kmh, 'elev_m', d0.get('elevation_m', 0.0))` の結果を代入する。
6. 関数 `z_next_for` を定義する。
7. 条件 `z_next_for(v_kmh) >= target_soc - tol` を判定し、真なら内部処理を行う。
8. `v_kmh` を返す。
9. `lo` に `0.0` の結果を代入する。
10. `hi` に `max(0.0, float(v_kmh))` の結果を代入する。
11. `range(25)` を順に走査し、各要素を `_` に入れて処理する。
12. `mid` に `0.5 * (lo + hi)` の結果を代入する。
13. 条件 `z_next_for(mid) < target_soc - tol` を判定し、真なら内部処理を行う。
14. `hi` に `mid` の結果を代入する。
15. 上の条件が偽の場合:
16. `lo` に `mid` の結果を代入する。
17. `float(lo)` を返す。

代表コード断片

```
def soc_day_guard_speed(model, v_kmh, z, Tb, s_kmh, d0, route_profile, target_soc, tol):
    if target_soc <= 0.0:
        return v_kmh
    slope_pct = get_profile_val(route_profile, s_kmh, 'slope_pct', d0.get('slope_pct', 0.0))
    headwind_ms = d0.get('headwind_ms', 0.0)
    elevation_m = get_profile_val(route_profile, s_kmh, 'elev_m', d0.get('elevation_m', 0.0))

    def z_next_for(v_kmh_local):
        out = model.electrical_balance(
            v_kmh_local / 3.6, slope_pct, z, Tb, d0['G_poa'], d0['Tcell_C'],
            headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=
elevation_m,
        )
        P_pack = float(out['P_pack'])
        return model.soc_step(
            z,
            P_pack,
            model.p.dt,
            current_a=float(out['I']),
            Tbat_C=Tb,
        )

    if z_next_for(v_kmh) >= (target_soc - tol):
        return v_kmh
    lo = 0.0
    hi = max(0.0, float(v_kmh))
```

```

for _ in range(25):
    mid = 0.5 * (lo + hi)
    if z_next_for(mid) < (target_soc - tol):
        hi = mid
    else:
        lo = mid
return float(lo)

```

7.53 L1458 関数 soc_day_guard_speed.z_next_for

- 行範囲: L1458–L1470
- 所属: soc_day_guard_speed
- 定義: z_next_for(v_kmh_local)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: electrical_balance, float, get, soc_step
- 戻り値の要点: model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb)
- 主なローカル代入名: P_pack, out
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. out に model.electrical_balance(v_kmh_local / 3.6, slope_pct, z, Tb, d0['G_poa'], d0['Tcell_C'], headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=elevation_m) の結果を代入する。
2. P_pack に float(out['P_pack']) の結果を代入する。
3. model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb) を返す。

代表コード断片

```

def z_next_for(v_kmh_local):
    out = model.electrical_balance(
        v_kmh_local / 3.6, slope_pct, z, Tb, d0['G_poa'], d0['Tcell_C'],
        headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=
elevation_m,
    )
    P_pack = float(out['P_pack'])
    return model.soc_step(
        z,
        P_pack,
        model.p.dt,
        current_a=float(out['I']),
        Tbat_C=Tb,
    )

```

7.54 L1485 関数 main

- 行範囲: L1485–L4016
- 所属: module 直下
- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `DataFrame`, `DetailCsvStream`, `Params`, `Path`, `SmoothRateLimiter`, `SolarCarModel`, `Timestamp`, `ValueError`, `ZoneInfo`, `_deep_copy_cfg`, `abs`
- 戻り値の要点: `choose_integration_step_seconds(args.dt, forecast_native_dt_sec)/t_utc >= forecast_start_time / total_sec / s0 + alpha * (s1 - s0)`
- 主なローカル代入名: `E_batt`, `E_pv`, `G_control_stop_poa`, `G_drive_poa`, `G_raw`, `G_stop_poa`, `I`, `J`, `Nc`, `Np`, `P_allow`, `P_mech_wheel`, `P_pack`, `P_pv`, `Tb`, `Tb_before`, `Tb_cur`, `Tb_next`, `Tb_seed`, `Tb_step`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('Either --profile_yaml or --params_yaml is required.')`, `ValueError('forecast_csv, route_profile_csv, and stop_yaml must be provided.')`, `ValueError('mpc.upper_lock_initial_policy requires paths.initial_upper_policy_csv')`
- 制御構造の規模: 条件分岐 207、ループ 23、try 10

この定義を読むための Python 構文

- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `ap` に `argparse.ArgumentParser()` の結果を代入する。
2. `ap.add_argument(...)` を実行する。
3. `ap.add_argument(...)` を実行する。
4. `ap.add_argument(...)` を実行する。
5. `ap.add_argument(...)` を実行する。
6. `ap.add_argument(...)` を実行する。
7. `ap.add_argument(...)` を実行する。
8. `ap.add_argument(...)` を実行する。
9. `ap.add_argument(...)` を実行する。
10. `ap.add_argument(...)` を実行する。
11. `ap.add_argument(...)` を実行する。

12. `ap.add_argument(...)` を実行する。
13. `ap.add_argument(...)` を実行する。
14. `ap.add_argument(...)` を実行する。
15. `ap.add_argument(...)` を実行する。
16. `ap.add_argument(...)` を実行する。
17. `ap.add_argument(...)` を実行する。
18. `ap.add_argument(...)` を実行する。
19. `ap.add_argument(...)` を実行する。
20. `ap.add_argument(...)` を実行する。
21. `ap.add_argument(...)` を実行する。
22. `ap.add_argument(...)` を実行する。
23. `ap.add_argument(...)` を実行する。
24. `ap.add_argument(...)` を実行する。
25. `ap.add_argument(...)` を実行する。
26. `ap.add_argument(...)` を実行する。
27. `ap.add_argument(...)` を実行する。
28. `ap.add_argument(...)` を実行する。
29. `ap.add_argument(...)` を実行する。
30. `ap.add_argument(...)` を実行する。
31. `ap.add_argument(...)` を実行する。
32. `ap.add_argument(...)` を実行する。
33. `ap.add_argument(...)` を実行する。
34. `ap.add_argument(...)` を実行する。
35. `ap.add_argument(...)` を実行する。
36. `ap.add_argument(...)` を実行する。
37. `ap.add_argument(...)` を実行する。
38. `ap.add_argument(...)` を実行する。
39. `ap.add_argument(...)` を実行する。
40. `ap.add_argument(...)` を実行する。
41. `ap.add_argument(...)` を実行する。
42. `ap.add_argument(...)` を実行する。
43. `ap.add_argument(...)` を実行する。
44. `ap.add_argument(...)` を実行する。
45. `ap.add_argument(...)` を実行する。

46. ap.add_argument(...) を実行する。
47. ap.add_argument(...) を実行する。
48. ap.add_argument(...) を実行する。
49. ap.add_argument(...) を実行する。
50. ap.add_argument(...) を実行する。
51. ap.add_argument(...) を実行する。
52. ap.add_argument(...) を実行する。
53. ap.add_argument(...) を実行する。
54. ap.add_argument(...) を実行する。
55. ap.add_argument(...) を実行する。
56. ap.add_argument(...) を実行する。
57. ap.add_argument(...) を実行する。
58. ap.add_argument(...) を実行する。
59. ap.add_argument(...) を実行する。
60. ap.add_argument(...) を実行する。
61. ap.add_argument(...) を実行する。
62. ap.add_argument(...) を実行する。
63. ap.add_argument(...) を実行する。
64. ap.add_argument(...) を実行する。
65. ap.add_argument(...) を実行する。
66. ap.add_argument(...) を実行する。
67. ap.add_argument(...) を実行する。
68. ap.add_argument(...) を実行する。
69. ap.add_argument(...) を実行する。
70. ap.add_argument(...) を実行する。
71. ap.add_argument(...) を実行する。
72. ap.add_argument(...) を実行する。
73. ap.add_argument(...) を実行する。
74. ap.add_argument(...) を実行する。
75. ap.add_argument(...) を実行する。
76. ap.add_argument(...) を実行する。
77. ap.add_argument(...) を実行する。
78. ap.add_argument(...) を実行する。
79. ap.add_argument(...) を実行する。
80. ap.add_argument(...) を実行する。

代表コード断片

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--profile_yaml', default='')
    ap.add_argument('--forecast_csv', default='')
    ap.add_argument('--forecast_fill_csv', default='')
    ap.add_argument('--progress_reference_csv', default='')
    ap.add_argument('--forecast_time_mode', default='auto')
    ap.add_argument('--forecast_time_tz', default='UTC')
    ap.add_argument('--route_profile_csv', default='')
    ap.add_argument('--speed_profile_csv', required=False, default='')
    ap.add_argument('--params_yaml', default='')
    ap.add_argument('--stop_yaml', default='')
    ap.add_argument('--drive_schedule_yaml', required=False, default='')
    ap.add_argument('--panel_eff_map', default='')
    ap.add_argument('--mppt_eff_map', default='')
    ap.add_argument('--ocv_soc_map', default='')
    ap.add_argument('--drive_eff_map', default='')
    ap.add_argument('--regen_eff_map', default='')
    ap.add_argument('--rint_map', default='')
    ap.add_argument('--drive_map_eco', default='')
    ap.add_argument('--drive_map_power', default='')
    ap.add_argument('--regen_map_eco', default='')
    ap.add_argument('--regen_map_power', default='')
    ap.add_argument('--dt', type=float, default=600.0)
    ap.add_argument('--horizon_steps', type=int, default=9)
    ap.add_argument('--soc0', type=float, default=0.99)
    ap.add_argument('--Tb0', type=float, default=30.0)
    ap.add_argument('--v0_kmh', type=float, default=40.0)
    ap.add_argument('--start_utc', default='')
    ap.add_argument('--forecast_start_time_utc', default='')
    ap.add_argument('--start_index', type=int, default=-1)
    ap.add_argument('--start_s_kmh', type=float, default=-1.0)
    ap.add_argument('--resume_csv', default='')
    ap.add_argument('--resume_s_kmh', type=float, default=-1.0)
    ap.add_argument('--out_csv', default='solar_sim_log.csv')
    ...
```

7.55 L1852 関数 main.integration_step_seconds

- 行範囲: L1852–L1857
- 所属: main
- 定義: integration_step_seconds()
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: choose_integration_step_seconds
- 戻り値の要点: choose_integration_step_seconds(args.dt, forecast_native_dt_sec)
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `choose_integration_step_seconds(args.dt, forecast_native_dt_sec)` を返す。

代表コード断片

```
def integration_step_seconds():  
    # Prediction must not integrate more coarsely than either the execution  
    # state update or the available weather samples. Using max() here made  
    # hourly weather grids turn the planner into an hourly nonlinear state  
    # integrator while execution still advanced every 10 minutes.  
    return choose_integration_step_seconds(args.dt, forecast_native_dt_sec)
```

7.56 L1859 関数 `main.forecast_has_coverage`

- 行範囲: L1859–L1866
- 所属: `main`
- 定義: `forecast_has_coverage(t_utc:datetime)->bool`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `len`
- 戻り値の要点: `t_utc >= forecast_start_time / False / False / t_utc >= forecast_start_data_utc`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `len(df) == 0` を判定し、真なら内部処理を行う。
2. `False` を返す。
3. 条件 `forecast_coverage_end_utc is not None and t_utc > forecast_coverage_end_utc` を判定し、真なら内部処理を行う。
4. `False` を返す。
5. 条件 `forecast_mode == 'absolute' and forecast_start_data_utc is not None` を判定し、真なら内部処理を行う。
6. `t_utc >= forecast_start_data_utc` を返す。
7. `t_utc >= forecast_start_time` を返す。

代表コード断片

```
def forecast_has_coverage(t_utc: datetime) -> bool:
    if len(df) == 0:
        return False
    if forecast_coverage_end_utc is not None and t_utc > forecast_coverage_end_utc:
        return False
    if forecast_mode == 'absolute' and forecast_start_data_utc is not None:
        return t_utc >= forecast_start_data_utc
    return t_utc >= forecast_start_time
```

7.57 L1868 関数 main.integrate_drive_time_between

- 行範囲: L1868–L1883
- 所属: main
- 定義: `integrate_drive_time_between(t_start_utc:datetime,t_end_utc:datetime)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `integration_step_seconds`, `is_drive_time`, `max`, `min`, `timedelta`, `total_seconds`
- 戻り値の要点: `total_sec / 0.0 / (t_end_utc - t_start_utc).total_seconds()`
- 主なローカル代入名: `dt_local`, `dt_sec`, `t_cursor`, `total_sec`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `t_end_utc <= t_start_utc` を判定し、真なら内部処理を行う。
2. 0.0 を返す。
3. 条件 `schedule is None` を判定し、真なら内部処理を行う。
4. `(t_end_utc - t_start_utc).total_seconds()` を返す。
5. `dt_sec` に `integration_step_seconds()` の結果を代入する。
6. `total_sec` に 0.0 の結果を代入する。
7. `t_cursor` に `t_start_utc` の結果を代入する。
8. 条件 `t_cursor < t_end_utc` が成り立つ間くり返す。
9. `dt_local` に `min(dt_sec, max(0.0, (t_end_utc - t_cursor).total_seconds()))` の結果を代入する。
10. 条件 `dt_local <= 0.0` を判定し、真なら内部処理を行う。

11. Break 文を実行する。
12. 条件 `schedule.is_drive_time(t_cursor)` を判定し、真なら内部処理を行う。
13. `total_sec` を Add で更新する。
14. `t_cursor` を Add で更新する。
15. `total_sec` を返す。

代表コード断片

```
def integrate_drive_time_between(t_start_utc: datetime, t_end_utc: datetime) -> float:
    if t_end_utc <= t_start_utc:
        return 0.0
    if schedule is None:
        return (t_end_utc - t_start_utc).total_seconds()
    dt_sec = integration_step_seconds()
    total_sec = 0.0
    t_cursor = t_start_utc
    while t_cursor < t_end_utc:
        dt_local = min(dt_sec, max(0.0, (t_end_utc - t_cursor).total_seconds()))
        if dt_local <= 0.0:
            break
        if schedule.is_drive_time(t_cursor):
            total_sec += dt_local
        t_cursor += timedelta(seconds=dt_local)
    return total_sec
```

7.58 L1885 関数 `main.reference_value_at_time`

- 行範囲: L1885–L1905
- 所属: `main`
- 定義: `reference_value_at_time(t_utc:datetime, field:str)->float|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `int`, `len`, `max`, `min`, `searchsorted`, `timestamp_ns`
- 戻り値の要点: $s0 + \alpha * (s1 - s0)$ / `None` / `None` / `float(progress_ref_df[field].iloc[0])`
- 主なローカル代入名: `alpha`, `idx_hi`, `idx_lo`, `s0`, `s1`, `t0_ns`, `t1_ns`, `ts_ns`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `progress_ref_df is None or progress_ref_time_ns is None or len(progress_ref_df) == 0` を判定し、真なら内部処理を行う。
2. `None` を返す。
3. 条件 `field not in progress_ref_df.columns` を判定し、真なら内部処理を行う。
4. `None` を返す。
5. `ts_ns` に `timestamp_ns(t_utc)` の結果を代入する。
6. `idx_hi` に `int(np.searchsorted(progress_ref_time_ns, ts_ns, side='left'))` の結果を代入する。
7. 条件 `idx_hi <= 0` を判定し、真なら内部処理を行う。
8. `float(progress_ref_df[field].iloc[0])` を返す。
9. 条件 `idx_hi >= len(progress_ref_df)` を判定し、真なら内部処理を行う。
10. `float(progress_ref_df[field].iloc[-1])` を返す。
11. `idx_lo` に `idx_hi - 1` の結果を代入する。
12. `t0_ns` に `int(progress_ref_time_ns[idx_lo])` の結果を代入する。
13. `t1_ns` に `int(progress_ref_time_ns[idx_hi])` の結果を代入する。
14. `s0` に `float(progress_ref_df[field].iloc[idx_lo])` の結果を代入する。
15. `s1` に `float(progress_ref_df[field].iloc[idx_hi])` の結果を代入する。
16. 条件 `t1_ns <= t0_ns` を判定し、真なら内部処理を行う。
17. `s1` を返す。
18. `alpha` に `float((ts_ns - t0_ns) / max(t1_ns - t0_ns, 1))` の結果を代入する。
19. `alpha` に `max(0.0, min(1.0, alpha))` の結果を代入する。
20. `s0 + alpha * (s1 - s0)` を返す。

代表コード断片

```
def reference_value_at_time(t_utc: datetime, field: str) -> float | None:
    if progress_ref_df is None or progress_ref_time_ns is None or len(progress_ref_df) == 0:
        return None
    if field not in progress_ref_df.columns:
        return None
    ts_ns = timestamp_ns(t_utc)
    idx_hi = int(np.searchsorted(progress_ref_time_ns, ts_ns, side='left'))
    if idx_hi <= 0:
        return float(progress_ref_df[field].iloc[0])
    if idx_hi >= len(progress_ref_df):
        return float(progress_ref_df[field].iloc[-1])
    idx_lo = idx_hi - 1
    t0_ns = int(progress_ref_time_ns[idx_lo])
    t1_ns = int(progress_ref_time_ns[idx_hi])
    s0 = float(progress_ref_df[field].iloc[idx_lo])
    s1 = float(progress_ref_df[field].iloc[idx_hi])
    if t1_ns <= t0_ns:
        return s1
    alpha = float((ts_ns - t0_ns) / max(t1_ns - t0_ns, 1))
    alpha = max(0.0, min(1.0, alpha))
    return s0 + alpha * (s1 - s0)
```

7.59 L1907 関数 `main.reference_distance_at_time`

- 行範囲: L1907–L1908
- 所属: `main`
- 定義: `reference_distance_at_time(t_utc:datetime)->float|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `reference_value_at_time`
- 戻り値の要点: `reference_value_at_time(t_utc, 's_km')`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `reference_value_at_time(t_utc, 's_km')` を返す。

代表コード断片

```
def reference_distance_at_time(t_utc: datetime) -> float | None:
    return reference_value_at_time(t_utc, 's_km')
```

7.60 L1958 関数 `main.time_to_index`

- 行範囲: L1958–L1969
- 所属: `main`
- 定義: `time_to_index(dt_utc)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `all`, `astimezone`, `clip`, `datetime64`, `int`, `isna`, `len`, `replace`, `searchsorted`
- 戻り値の要点: `int(np.clip(idx, 0, len(df) - 1)) / 0`
- 主なローカル代入名: `idx`, `lookup_naive`, `lookup_utc`, `t_series`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 'time' not in df.columns or df['time'].isna().all() を判定し、真なら内部処理を行う。
2. 0 を返す。
3. t_series に df['time'].values の結果を代入する。
4. lookup_utc に dt_utc.replace(tzinfo=timezone.utc) if dt_utc.tzinfo is None else dt_utc.astimezone(timezone.utc) の結果を代入する。
5. lookup_naive に lookup_utc.replace(tzinfo=None) の結果を代入する。
6. idx に int(np.searchsorted(t_series, np.datetime64(lookup_naive, 'ns')) - 1) の結果を代入する。
7. int(np.clip(idx, 0, len(df) - 1)) を返す。

代表コード断片

```
def time_to_index(dt_utc):
    if 'time' not in df.columns or df['time'].isna().all():
        return 0
    t_series = df['time'].values
    lookup_utc = (
        dt_utc.replace(tzinfo=timezone.utc)
        if dt_utc.tzinfo is None
        else dt_utc.astimezone(timezone.utc)
    )
    lookup_naive = lookup_utc.replace(tzinfo=None)
    idx = int(np.searchsorted(t_series, np.datetime64(lookup_naive, 'ns')) - 1)
    return int(np.clip(idx, 0, len(df) - 1))
```

7.61 L2095 関数 main.remaining_day_budget

- 行範囲: L2095–L2115
- 所属: main
- 定義: remaining_day_budget(sim_t_local, _k_idx)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: current_drive_window, float, forecast_at_time, forecast_has_coverage, integration_step_seconds, is_drive_time, max, min, pv_power_mppt, timedelta, total_seconds
- 戻り値の要点: (E_pv, t_remain) / (None, None) / (None, None)
- 主なローカル代入名: E_pv, P_pv, _, dt_local, dt_sample, env, t, t_end, t_remain, win
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 3、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `schedule is None or not schedule.is_drive_time(sim_t_local)` を判定し、真なら内部処理を行う。
2. `(None, None)` を返す。
3. `win` に `schedule.current_drive_window(sim_t_local)` の結果を代入する。
4. 条件 `win is None` を判定し、真なら内部処理を行う。
5. `(None, None)` を返す。
6. `(_, t_end)` に `win` の結果を代入する。
7. `dt_sample` に `integration_step_seconds()` の結果を代入する。
8. `t` に `sim_t_local` の結果を代入する。
9. `E_pv` に `0.0` の結果を代入する。
10. `t_remain` に `0.0` の結果を代入する。
11. 条件 `t < t_end and forecast_has_coverage(t)` が成り立つ間くり返す。
12. `dt_local` に `min(dt_sample, max(0.0, (t_end - t).total_seconds()))` の結果を代入する。
13. 条件 `dt_local <= 0.0` を判定し、真なら内部処理を行う。
14. `Break` 文を実行する。
15. `env` に `forecast_at_time(t, s_km, drive=True)` の結果を代入する。
16. `P_pv` に `float(model.pv_power_mppt(env['G_poa'], env['Tcell_C']))` の結果を代入する。
17. `E_pv` を `Add` で更新する。
18. `t_remain` を `Add` で更新する。
19. `t` を `Add` で更新する。
20. `(E_pv, t_remain)` を返す。

代表コード断片

```
def remaining_day_budget(sim_t_local, _k_idx):
    if schedule is None or not schedule.is_drive_time(sim_t_local):
        return None, None
    win = schedule.current_drive_window(sim_t_local)
    if win is None:
        return None, None
    _, t_end = win
    dt_sample = integration_step_seconds()
    t = sim_t_local
    E_pv = 0.0
    t_remain = 0.0
    while t < t_end and forecast_has_coverage(t):
        dt_local = min(dt_sample, max(0.0, (t_end - t).total_seconds()))
        if dt_local <= 0.0:
            break
        env = forecast_at_time(t, s_km, drive=True)
        P_pv = float(model.pv_power_mppt(env['G_poa'], env['Tcell_C']))
```

```

        E_pv += P_pv * (dt_local / 3600.0)
        t_remain += dt_local
        t += timedelta(seconds=dt_local)
    return E_pv, t_remain

```

7.62 L2117 関数 main.budget_speed_limit

- 行範囲: L2117–L2134
- 所属: main
- 定義: budget_speed_limit(P_allow, d0)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: electrical_balance, float, get, range
- 戻り値の要点: float(lo) / None
- 主なローカル代入名: P_pack, _, hi, lo, mid, out
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 P_allow is None を判定し、真なら内部処理を行う。
2. None を返す。
3. lo に 0.0 の結果を代入する。
4. hi に v_max_kmh の結果を代入する。
5. range(24) を順に走査し、各要素を _ に入れて処理する。
6. mid に 0.5 * (lo + hi) の結果を代入する。
7. out に model.electrical_balance(mid / 3.6, d0['slope_pct'], z, Tb, d0['G_poa'], d0['Tcell_C'], headwind_ms=d0['headwind_ms'], ambient_temp_c=d0.get('Tamb_C'), elevation_m=d0.get('elevation_m', 0.0)) の結果を代入する。
8. P_pack に float(out['P_pack']) の結果を代入する。
9. 条件 P_pack > P_allow を判定し、真なら内部処理を行う。
10. hi に mid の結果を代入する。
11. 上の条件が偽の場合:
12. lo に mid の結果を代入する。
13. float(lo) を返す。

代表コード断片

```
def budget_speed_limit(P_allow, d0):
    if P_allow is None:
        return None
    lo = 0.0
    hi = v_max_kmh
    for _ in range(24):
        mid = 0.5 * (lo + hi)
        out = model.electrical_balance(
            mid / 3.6, d0['slope_pct'], z, Tb, d0['G_poa'], d0['Tcell_C'],
            headwind_ms=d0['headwind_ms'], ambient_temp_c=d0.get('Tamb_C'),
            elevation_m=d0.get('elevation_m', 0.0),
        )
        P_pack = float(out['P_pack'])
        if P_pack > P_allow:
            hi = mid
        else:
            lo = mid
    return float(lo)
```

7.63 L2136 関数 main.forecast_at_time

- 行範囲: L2136–L2234
- 所属: main
- 定義: `forecast_at_time(t_utc, s_query_km=None, drive=True)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `dict`, `float`, `get`, `get_profile_val`, `int`, `interp_forecast_grid`, `len`, `max`, `searchsorted`, `timestamp_ns`, `total_seconds`
- 戻り値の要点: `dict(G_raw=G_raw, G_drive_poa=G_drive_poa, G_stop_poa=G_stop_poa, G_control_stop_poa=G_control_stop_poa, G_poa=G_drive_poa if drive else G_stop_poa, Tcell_drive_C=tcell_drive, Tcell_stop_C=tcell_stop, Tcell_control_stop_C=tcell_control_stop, Tcell_C=tcell_drive if drive else tcell_stop, Tamb_C=float(row.get('Tamb_C', 30.0)), headwind_ms=float(row.get('headwind_ms', 0.0)), elevation_m=get_profile_val(route_profile, float(s_query_km or 0.0), 'elev_m', 0.0)) / dict(G_raw=0.0, G_drive_poa=0.0, G_stop_poa=0.0, G_control_stop_poa=0.0, G_poa=0.0, Tcell_drive_C=40.0, Tcell_stop_C=40.0, Tcell_control_stop_C=40.0, Tcell_C=40.0, Tamb_C=30.0, headwind_ms=0.0, elevation_m=0.0) / dict(G_raw=G_raw, G_drive_poa=G_drive_poa, G_stop_poa=G_stop_poa, G_control_stop_poa=G_control_stop_poa, G_poa=G_drive_poa if drive else G_stop_poa, Tcell_drive_C=float(tcell_drive), Tcell_stop_C=float(tcell_stop), Tcell_control_stop_C=float(tcell_control_stop), Tcell_C=float(tcell_drive if drive else tcell_stop), Tamb_C=float(tamb), headwind_ms=float(headwind), elevation_m=get_profile_val(route_profile, float(s_query_km or 0.0), 'elev_m', 0.0))`
- 主なローカル代入名: `G_control_stop_poa`, `G_drive_poa`, `G_raw`, `G_stop_poa`, `elapsed`, `ghi`, `headwind`, `idx`, `poa_control_stop_raw`, `poa_drive_raw`, `poa_stop_ideal_raw`, `poa_stop_raw`, `row`, `tamb`, `tcell_control_stop`, `tcell_drive`, `tcell_stop`, `tcell_stop_ideal`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `len(df) == 0` を判定し、真なら内部処理を行う。
2. `dict(G_raw=0.0, G_drive_poa=0.0, G_stop_poa=0.0, G_control_stop_poa=0.0, G_poa=0.0, Tcell_drive_C=40.0, Tcell_stop_C=40.0, Tcell_control_stop_C=40.0, Tcell_C=40.0, Tamb_C=30.0, headwind_ms=0.0, elevation_m=0.0)` を返す。
3. 条件 `forecast_grid is not None` を判定し、真なら内部処理を行う。
4. `ghi` に `interp_forecast_grid(forecast_grid, 'GHI', t_utc, s_query_km, 0.0)` の結果を代入する。
5. `poa_drive_raw` に `interp_forecast_grid(forecast_grid, 'POA_drive', t_utc, s_query_km, ghi)` の結果を代入する。
6. `poa_stop_ideal_raw` に `interp_forecast_grid(forecast_grid, 'POA_stop_ideal', t_utc, s_query_km, poa_drive_raw)` の結果を代入する。
7. `poa_stop_raw` に `poa_drive_raw + args.stop_tilt_fraction * (poa_stop_ideal_raw - poa_drive_raw)` の結果を代入する。
8. `poa_control_stop_raw` に `poa_drive_raw + args.control_stop_tilt_fraction * (poa_stop_ideal_raw - poa_drive_raw)` の結果を代入する。
9. `tamb` に `interp_forecast_grid(forecast_grid, 'Tamb_C', t_utc, s_query_km, 30.0)` の結果を代入する。
10. `tcell_drive` に `interp_forecast_grid(forecast_grid, 'Tcell_drive_C', t_utc, s_query_km, interp_forecast_grid(forecast_grid, 'Tcell_C', t_utc, s_query_km, tamb))` の結果を代入する。
11. `tcell_stop_ideal` に `interp_forecast_grid(forecast_grid, 'Tcell_stop_ideal_C', t_utc, s_query_km, tcell_drive)` の結果を代入する。
12. `tcell_stop` に `tcell_drive + args.stop_tilt_fraction * (tcell_stop_ideal - tcell_drive)` の結果を代入する。
13. `tcell_control_stop` に `tcell_drive + args.control_stop_tilt_fraction * (tcell_stop_ideal - tcell_drive)` の結果を代入する。
14. `headwind` に `interp_forecast_grid(forecast_grid, 'headwind_ms', t_utc, s_query_km, 0.0)` の結果を代入する。
15. `G_raw` に `float(ghi) * args.solar_gain` の結果を代入する。
16. `G_drive_poa` に `float(poa_drive_raw) * args.solar_gain * args.poa_gain_drive` の結果を代入する。
17. `G_stop_poa` に `float(poa_stop_raw) * args.solar_gain * args.poa_gain_stop` の結果を代入する。
18. `G_control_stop_poa` に `float(poa_control_stop_raw) * args.solar_gain * args.poa_gain_stop` の結果を代入する。
19. `dict(G_raw=G_raw, G_drive_poa=G_drive_poa, G_stop_poa=G_stop_poa, G_control_stop_poa=G_control_stop_poa, G_poa=G_drive_poa if drive else G_stop_poa, Tcell_drive_C=float(tcell_drive), Tcell_stop_C=float(tcell_stop), Tcell_control_stop_C=float(tcell_control_stop), Tcell_C=float(tcell_drive if drive else tcell_stop), Tamb_C=float(tamb), headwind_ms=float(headwind), elevation_m=get_profile_val(route_profile, float(s_query_km or 0.0), 'elev_m', 0.0))` を返す。
20. 条件 `forecast_time_ns is not None` を判定し、真なら内部処理を行う。
21. `idx` に `int(np.searchsorted(forecast_time_ns, timestamp_ns(t_utc)) - 1)` の結果を代入する。
22. `idx` に `int(np.clip(idx, 0, len(df) - 1))` の結果を代入する。
23. 上の条件が偽の場合:
24. `elapsed` に `(t_utc - start_utc).total_seconds()` の結果を代入する。

25. idx に `int(np.clip(elapsed / max(args.dt, 0.001), 0, len(df) - 1))` の結果を代入する。
26. row に `df.iloc[idx]` の結果を代入する。
27. G_raw に `float(row.get('GHI', 0.0)) * args.solar_gain` の結果を代入する。
28. poa_drive_raw に `float(row.get('POA_drive', row.get('GHI', 0.0)))` の結果を代入する。
29. poa_stop_ideal_raw に `float(row.get('POA_stop_ideal', poa_drive_raw))` の結果を代入する。
30. poa_stop_raw に `poa_drive_raw + args.stop_tilt_fraction * (poa_stop_ideal_raw - poa_drive_raw)` の結果を代入する。
31. poa_control_stop_raw に `poa_drive_raw + args.control_stop_tilt_fraction * (poa_stop_ideal_raw - poa_drive_raw)` の結果を代入する。
32. G_drive_poa に `poa_drive_raw * args.solar_gain * args.poa_gain_drive` の結果を代入する。
33. G_stop_poa に `poa_stop_raw * args.solar_gain * args.poa_gain_stop` の結果を代入する。
34. G_control_stop_poa に `poa_control_stop_raw * args.solar_gain * args.poa_gain_stop` の結果を代入する。
35. tcell_drive に `float(row.get('Tcell_drive_C', row.get('Tcell_C', 40.0)))` の結果を代入する。
36. tcell_stop_ideal に `float(row.get('Tcell_stop_ideal_C', tcell_drive))` の結果を代入する。
37. tcell_stop に `tcell_drive + args.stop_tilt_fraction * (tcell_stop_ideal - tcell_drive)` の結果を代入する。
38. tcell_control_stop に `tcell_drive + args.control_stop_tilt_fraction * (tcell_stop_ideal - tcell_drive)` の結果を代入する。
39. `dict(G_raw=G_raw, G_drive_poa=G_drive_poa, G_stop_poa=G_stop_poa, G_control_stop_poa=G_control_stop_poa, G_poa=G_drive_poa if drive else G_stop_poa, Tcell_drive_C=tcell_drive, Tcell_stop_C=tcell_stop, Tcell_control_stop_C=tcell_control_stop, Tcell_C=tcell_drive if drive else tcell_stop, Tamb_C=float(row.get('Tamb_C', 30.0)), headwind_ms=float(row.get('headwind_ms', 0.0)), elevation_m=get_profile_val(route_profile, float(s_query_km or 0.0), 'elev_m', 0.0))` を返す。

代表コード断片

```
def forecast_at_time(t_utc, s_query_km=None, drive=True):
    if len(df) == 0:
        return dict(
            G_raw=0.0,
            G_drive_poa=0.0,
            G_stop_poa=0.0,
            G_control_stop_poa=0.0,
            G_poa=0.0,
            Tcell_drive_C=40.0,
            Tcell_stop_C=40.0,
            Tcell_control_stop_C=40.0,
            Tcell_C=40.0,
            Tamb_C=30.0,
            headwind_ms=0.0,
            elevation_m=0.0,
        )
    if forecast_grid is not None:
        ghi = interp_forecast_grid(forecast_grid, 'GHI', t_utc, s_query_km, 0.0)
        poa_drive_raw = interp_forecast_grid(forecast_grid, 'POA_drive', t_utc,
s_query_km, ghi)
        poa_stop_ideal_raw = interp_forecast_grid(
            forecast_grid, 'POA_stop_ideal', t_utc, s_query_km, poa_drive_raw
        )
        poa_stop_raw = poa_drive_raw + args.stop_tilt_fraction * (poa_stop_ideal_raw -
poa_drive_raw)
```

```

        poa_control_stop_raw = poa_drive_raw + args.control_stop_tilt_fraction * (
            poa_stop_ideal_raw - poa_drive_raw
        )
        tamb = interp_forecast_grid(forecast_grid, 'Tamb_C', t_utc, s_query_km, 30.0)
        tcell_drive = interp_forecast_grid(
            forecast_grid,
            'Tcell_drive_C',
            t_utc,
            s_query_km,
            interp_forecast_grid(forecast_grid, 'Tcell_C', t_utc, s_query_km, tamb),
        )
        tcell_stop_ideal = interp_forecast_grid(
...

```

7.64 L2236 関数 main.apply_stop_poa

- 行範囲: L2236–L2242
- 所属: main
- 定義: `apply_stop_poa(env, *, control_stop=False)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `get`
- 戻り値の要点: `env`
- 主なローカル代入名: `irradiance_key`, `temperature_key`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `irradiance_key` に `'G_control_stop_poa'` if `control_stop` else `'G_stop_poa'` の結果を代入する。
2. `temperature_key` に `'Tcell_control_stop_C'` if `control_stop` else `'Tcell_stop_C'` の結果を代入する。
3. `env['G_poa']` に `float(env.get(irradiance_key, env.get('G_raw', env.get('G_poa', 0.0))))` の結果を代入する。
4. `env['Tcell_C']` に `float(env.get(temperature_key, env.get('Tcell_C', env.get('Tamb_C', 30.0))))` の結果を代入する。
5. `env['panel_stop_mode']` に `'control_horizontal'` if `control_stop` else `'camp_or_strategy_tilt'` の結果を代入する。
6. `env` を返す。

代表コード断片

```
def apply_stop_poa(env, *, control_stop=False):
    irradiance_key = 'G_control_stop_poa' if control_stop else 'G_stop_poa'
    temperature_key = 'Tcell_control_stop_C' if control_stop else 'Tcell_stop_C'
    env['G_poa'] = float(env.get(irradiance_key, env.get('G_raw', env.get('G_poa', 0.0))
    ))
    env['Tcell_C'] = float(env.get(temperature_key, env.get('Tcell_C', env.get('Tamb_C',
    30.0))))
    env['panel_stop_mode'] = 'control_horizontal' if control_stop else '
    camp_or_strategy_tilt'
    return env
```

7.65 L2244 関数 main.reference_speed_command

- 行範囲: L2244–L2262
- 所属: main
- 定義: reference_speed_command(t_utc:datetime,s_now_kmh:float)->float|None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: clip, float, max, reference_distance_at_time, reference_value_at_time, timedelta
- 戻り値の要点: max(0.0, float(base_speed_kmh) * ref_speed_tracking_gain + correction_kmh) / None / None
- 主なローカル代入名: base_speed_kmh, correction_kmh, lag_kmh, lead_kmh, ref_s_future, ref_s_now, t_future
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 4、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 not ref_speed_tracking_enabled を判定し、真なら内部処理を行う。
2. None を返す。
3. ref_s_now に reference_distance_at_time(t_utc) の結果を代入する。
4. 条件 ref_s_now is None を判定し、真なら内部処理を行う。
5. None を返す。
6. base_speed_kmh に reference_value_at_time(t_utc, 'speed_kmh') の結果を代入する。
7. 条件 base_speed_kmh is None を判定し、真なら内部処理を行う。
8. t_future に t_utc + timedelta(seconds=max(ref_speed_tracking_lookahead_sec, 1.0)) の結果を代入する。
9. ref_s_future に reference_distance_at_time(t_future) の結果を代入する。

10. 条件 `ref_s_future is not None` を判定し、真なら内部処理を行う。
11. `base_speed_kmh` に `max(0.0, (ref_s_future - ref_s_now) * 3600.0 / max(ref_speed_tracking_lookahead_sec, 1.0))` の結果を代入する。
12. 上の条件が偽の場合:
13. `base_speed_kmh` に `0.0` の結果を代入する。
14. `lag_km` に `max(0.0, float(ref_s_now) - float(s_now_km))` の結果を代入する。
15. `lead_km` に `max(0.0, float(s_now_km) - float(ref_s_now))` の結果を代入する。
16. `correction_kmh` に `ref_speed_tracking_lag_gain * lag_km - ref_speed_tracking_lead_gain * lead_km` の結果を代入する。
17. `correction_kmh` に `float(np.clip(correction_kmh, -ref_speed_tracking_max_correction, ref_speed_tracking_max_correction))` の結果を代入する。
18. `max(0.0, float(base_speed_kmh) * ref_speed_tracking_gain + correction_kmh)` を返す。

代表コード断片

```
def reference_speed_command(t_utc: datetime, s_now_km: float) -> float | None:
    if not ref_speed_tracking_enabled:
        return None
    ref_s_now = reference_distance_at_time(t_utc)
    if ref_s_now is None:
        return None
    base_speed_kmh = reference_value_at_time(t_utc, 'speed_kmh')
    if base_speed_kmh is None:
        t_future = t_utc + timedelta(seconds=max(ref_speed_tracking_lookahead_sec, 1.0))
        ref_s_future = reference_distance_at_time(t_future)
        if ref_s_future is not None:
            base_speed_kmh = max(0.0, (ref_s_future - ref_s_now) * 3600.0 / max(
ref_speed_tracking_lookahead_sec, 1.0))
        else:
            base_speed_kmh = 0.0
    lag_km = max(0.0, float(ref_s_now) - float(s_now_km))
    lead_km = max(0.0, float(s_now_km) - float(ref_s_now))
    correction_kmh = ref_speed_tracking_lag_gain * lag_km - ref_speed_tracking_lead_gain
* lead_km
    correction_kmh = float(np.clip(correction_kmh, -ref_speed_tracking_max_correction,
ref_speed_tracking_max_correction))
    return max(0.0, float(base_speed_kmh) * ref_speed_tracking_gain + correction_kmh)
```

7.66 L2264 関数 `main.resolved_headwind_ms`

- 行範囲: L2264–L2272
- 所属: `main`
- 定義: `resolved_headwind_ms(env:dict,s_query_km:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `get`, `get_profile_val`, `isfinite`
- 戻り値の要点: `get_profile_val(route_profile, s_query_km, 'headwind_ms', 0.0) / weather_value`
- 主なローカル代入名: `weather_value`
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `weather_value` に `env.get('headwind_ms')` の結果を代入する。
2. 例外処理を伴う `try` ブロックを実行する。
3. `weather_value` に `float(weather_value)` の結果を代入する。
4. (`TypeError`, `ValueError`) を捕捉した場合:
5. `weather_value` に `float('nan')` の結果を代入する。
6. 条件 `np.isfinite(weather_value)` を判定し、真なら内部処理を行う。
7. `weather_value` を返す。
8. `get_profile_val(route_profile, s_query_km, 'headwind_ms', 0.0)` を返す。

代表コード断片

```
def resolved_headwind_ms(env: dict, s_query_km: float) -> float:
    weather_value = env.get('headwind_ms')
    try:
        weather_value = float(weather_value)
    except (TypeError, ValueError):
        weather_value = float('nan')
    if np.isfinite(weather_value):
        return weather_value
    return get_profile_val(route_profile, s_query_km, 'headwind_ms', 0.0)
```

7.67 L2274 関数 `main.integrate_stationary_duration`

- 行範囲: L2274–L2323
- 所属: `main`
- 定義: `integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait, *, control_stop=False)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `apply_stop_poa`, `choose_stationary_integration_step_seconds`, `clip`, `electrical_balance`, `float`, `forecast_at_time`, `forecast_has_coverage`, `get`, `get_profile_val`, `max`, `min`, `resolved_headwind_ms`
- 戻り値の要点: `(t_cursor, float(z_cur), float(Tb_cur), float(dt_wait - remaining)) / (t_utc, z, Tb, 0.0)`
- 主なローカル代入名: `P_pack`, `Tb_cur`, `dt_local`, `dt_sample`, `dt_wait`, `env`, `headwind_ms`, `loss_int`, `out`, `remaining`, `slope_pct`, `t_cursor`, `z_cur`
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `dt_wait` に `max(0.0, float(dt_wait))` の結果を代入する。
2. 条件 `dt_wait <= 0.0` を判定し、真なら内部処理を行う。
3. `(t_utc, z, Tb, 0.0)` を返す。
4. `slope_pct` に `get_profile_val(route_profile, s_km, 'slope_pct', 0.0)` の結果を代入する。
5. `dt_sample` に `choose_stationary_integration_step_seconds(args.dt, forecast_native_dt_sec, stationary_prediction_step_sec)` の結果を代入する。
6. `remaining` に `dt_wait` の結果を代入する。
7. `t_cursor` に `t_utc` の結果を代入する。
8. `z_cur` に `float(z)` の結果を代入する。
9. `Tb_cur` に `float(Tb)` の結果を代入する。
10. 条件 `remaining > 1e-09 and forecast_has_coverage(t_cursor)` が成り立つ間くり返す。
11. `dt_local` に `min(dt_sample, remaining)` の結果を代入する。
12. `env` に `forecast_at_time(t_cursor, s_km, drive=False)` の結果を代入する。
13. 条件 `control_stop` を判定し、真なら内部処理を行う。
14. `apply_stop_poa(...)` を実行する。
15. `headwind_ms` に `resolved_headwind_ms(env, s_km)` の結果を代入する。
16. `out` に `model.electrical_balance(0.0, slope_pct, z_cur, Tb_cur, env['G_poa'], env['Tcell_C'], headwind_ms=headwind_ms, aux_power_w=model.scheduled_auxiliary_power(is_driving=False, irradiance_wm2=env['G_poa']), ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0))` の結果を代入する。
17. `P_pack` に `float(out['P_pack'])` の結果を代入する。
18. `loss_int` に `float(out['losses_int'])` の結果を代入する。
19. `z_cur` に `model.soc_step(z_cur, P_pack, dt_local, current_a=float(out['I']), Tbat_C=Tb_cur)` の結果を代入する。
20. `Tb_cur` に `Tb_cur + dt_local / 1800.0 * (env['Tamb_C'] - Tb_cur) + loss_int * dt_local / 50000.0` の結果を代入する。
21. `z_cur` に `float(np.clip(z_cur, p.soc_min, p.soc_max))` の結果を代入する。
22. `Tb_cur` に `float(np.clip(Tb_cur, p.T_min, p.T_max))` の結果を代入する。
23. `t_cursor` を `Add` で更新する。
24. `remaining` を `Sub` で更新する。
25. `(t_cursor, float(z_cur), float(Tb_cur), float(dt_wait - remaining))` を返す。

代表コード断片

```
def integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait, *, control_stop=False):
    dt_wait = max(0.0, float(dt_wait))
    if dt_wait <= 0.0:
        return t_utc, z, Tb, 0.0
    slope_pct = get_profile_val(route_profile, s_km, 'slope_pct', 0.0)
    dt_sample = choose_stationary_integration_step_seconds(
        args.dt,
        forecast_native_dt_sec,
        stationary_prediction_step_sec,
    )
    remaining = dt_wait
    t_cursor = t_utc
    z_cur = float(z)
    Tb_cur = float(Tb)
    while remaining > 1.0e-9 and forecast_has_coverage(t_cursor):
        dt_local = min(dt_sample, remaining)
        env = forecast_at_time(t_cursor, s_km, drive=False)
        if control_stop:
            apply_stop_poa(env, control_stop=True)
        headwind_ms = resolved_headwind_ms(env, s_km)
        out = model.electrical_balance(
            0.0,
            slope_pct,
            z_cur,
            Tb_cur,
            env['G_poa'],
            env['Tcell_C'],
            headwind_ms=headwind_ms,
            aux_power_w=model.scheduled_auxiliary_power(
                is_driving=False,
                irradiance_wm2=env['G_poa'],
            ),
            ambient_temp_c=env.get('Tamb_C'),
            elevation_m=env.get('elevation_m', 0.0),
        )
    ...
```

7.68 L2325 関数 main.step_wait

- 行範囲: L2325–L2330
- 所属: main
- 定義: `step_wait(t_utc, z, Tb, s_km)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `integrate_stationary_duration`, `is_drive_time`, `max`, `next_drive_start`, `total_seconds`
- 戻り値の要点: `integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait) / (t_utc, z, Tb, 0.0)`
- 主なローカル代入名: `dt_wait`, `t_start`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `schedule is None or schedule.is_drive_time(t_utc)` を判定し、真なら内部処理を行う。
2. `(t_utc, z, Tb, 0.0)` を返す。
3. `t_start` に `schedule.next_drive_start(t_utc)` の結果を代入する。
4. `dt_wait` に `max(0.0, (t_start - t_utc).total_seconds())` の結果を代入する。
5. `integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait)` を返す。

代表コード断片

```
def step_wait(t_utc, z, Tb, s_km):
    if schedule is None or schedule.is_drive_time(t_utc):
        return t_utc, z, Tb, 0.0
    t_start = schedule.next_drive_start(t_utc)
    dt_wait = max(0.0, (t_start - t_utc).total_seconds())
    return integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait)
```

7.69 L2332 関数 `main.mpc_solve_distance`

- 行範囲: L2332–L3020
- 所属: `main`
- 定義: `mpc_solve_distance(t0_utc, s0_km, z0, Tb0, v0_kmh, v_init=None, v_init_s_km=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `Path`, `ValueError`, `abs`, `absolute_control_distances`, `append`, `array`, `asarray`, `average_profile`, `bool`, `build_balance_seed`, `build_upper_distance_horizon`
- 戻り値の要点: `(v0, segments, u_seq, ctrl_s_abs) / (v0_kmh, [{ 'v_kmh': v0_kmh, 'dt_sec': args.dt }], np.array([v0_kmh], dtype=float), np.array([float(s0_km)], dtype=float)) / (1.0 - alpha) * u_vec[idx] + alpha * u_vec[idx_next] / np.array(u_seed, dtype=float)`
- 主なローカル代入名: `I`, `J`, `Nc`, `Np`, `P_mech_wheel`, `P_pack`, `P_pv`, `Tb`, `Tb_next`, `Tb_seed`, `V`, `_`, `alpha`, `best_score`, `best_v`, `bounds`, `candidate_grid`, `checkpoint_base`, `checkpoint_path`, `checkpoint_payload`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('mpc.upper_lock_initial_policy requires paths.initial_upper_policy_csv')`
- 制御構造の規模: 条件分岐 38、ループ 7、try 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. horizon に `build_upper_distance_horizon(mode=upper_horizon_mode, s0_km=s0_km, race_km=race_km, ds_km=upper_ds_km, horizon_km=upper_horizon_km, max_steps=upper_max_steps, ctrl_km=upper_ctrl_km, adaptive_min_ds_km=upper_adaptive_min_ds_km, adaptive_max_ds_km=upper_adaptive_max_ds_km, adaptive_growth=upper_adaptive_growth)` の結果を代入する。
2. ds_seq に `np.array(horizon.ds_seq_km, dtype=float)` の結果を代入する。
3. seg_s に `np.array(horizon.seg_s_km, dtype=float)` の結果を代入する。
4. Np に `int(len(ds_seq))` の結果を代入する。
5. 条件 `Np <= 0` を判定し、真なら内部処理を行う。
6. (`v0_kmh`, [{`'v_kmh'`: `v0_kmh`, `'dt_sec'`: `args.dt`}], `np.array([v0_kmh], dtype=float)`, `np.array([float(s0_km)], dtype=float)`) を返す。
7. v_min_solver に `max(0.1, float(upper_vmin_kmh))` の結果を代入する。
8. ctrl_s に `np.array(horizon.ctrl_s_km, dtype=float)` の結果を代入する。
9. ctrl_s_abs に `absolute_control_distances(s0_km, ctrl_s)` の結果を代入する。
10. Nc に `int(len(ctrl_s))` の結果を代入する。
11. 条件 `v_init is not None and v_init_s_km is not None and (len(v_init) == len(v_init_s_km))` を判定し、真なら内部処理を行う。
12. x0 に `shift_upper_policy_warm_start(v_init_s_km, v_init, ctrl_s_abs, minimum_speed_kmh=v_min_solver, maximum_speed_kmh=v_max_kmh)` の結果を代入する。
13. 上の条件が偽の場合:
14. 条件 `v_init is not None and len(v_init) == Nc` を判定し、真なら内部処理を行う。
15. x0 に `np.array(v_init, dtype=float)` の結果を代入する。
16. 上の条件が偽の場合:
17. 条件 `initial_upper_policy_df is not None and (not initial_upper_policy_df.empty)` を判定し、真なら内部処理を行う。
18. x0 に `np.interp(ctrl_s_abs, initial_upper_policy_df['s_km'].to_numpy(dtype=float), initial_upper_policy_df['v_kmh'].to_numpy(dtype=float))` の結果を代入する。
19. x0 に `np.clip(x0, v_min_solver, v_max_kmh)` の結果を代入する。
20. 上の条件が偽の場合:
21. x0 に `np.array([v_max_kmh] * Nc, dtype=float)` の結果を代入する。
22. bounds に `[(v_min_solver, v_max_kmh)] * Nc` の結果を代入する。
23. idx に `np.searchsorted(ctrl_s, seg_s, side='right') - 1` の結果を代入する。
24. idx に `np.clip(idx, 0, Nc - 1)` の結果を代入する。
25. idx_next に `np.clip(idx + 1, 0, Nc - 1)` の結果を代入する。
26. denom に `np.maximum(ctrl_s[idx_next] - ctrl_s[idx], 1e-06)` の結果を代入する。
27. alpha に `(seg_s - ctrl_s[idx]) / denom` の結果を代入する。
28. 関数 `expand_ctrl` を定義する。

29. 関数 `build_balance_seed` を定義する。
30. `selected_prediction` に `{}` の結果を代入する。
31. 関数 `cost` を定義する。
32. `sim_log(...)` を実行する。
33. `checkpoint_base` に `Path(args.latest_manifest_json)` if `args.latest_manifest_json` else `Path(args.out_csv)` の結果を代入する。
34. `progress_path` に `checkpoint_base.with_name('upper_solver_progress.json')` の結果を代入する。
35. 関数 `save_solver_progress` を定義する。
36. 条件 `upper_lock_initial_policy` を判定し、真なら内部処理を行う。
37. 条件 `initial_upper_policy_df is None or initial_upper_policy_df.empty` を判定し、真なら内部処理を行う。
38. `ValueError('mpc.upper_lock_initial_policy requires paths.initial_upper_policy_csv')` を送出する。
39. `u_seq` に `np.asarray(x0, dtype=float)` の結果を代入する。
40. `solve_info` に `{'success': True, 'method': 'locked_external_policy', 'label': 'mesh_validation_policy', 'message': 'optimization intentionally bypassed for fixed-policy mesh validation', 'fun': float('nan'), 'nit': 0, 'cem_used': False, 'shgo_used': False, 'shgo_nfev': 0, 'shgo_local_minima': 0, 'discrete_global_proof': False, 'discrete_grid_levels': 0, 'discrete_grid_values': [], 'discrete_grid_candidates': 0, 'discrete_grid_nonfinite': 0, 'discrete_grid_best_fun': float('inf'), 'discrete_grid_best_x': [], 'deterministic_seed_candidates': 1, 'deterministic_seed_nonfinite': 0, 'finite_library_candidates': 1, 'selected_x': u_seq.tolist(), 'selected_no_worse_than_grid': False, 'finite_library_global_proof': False, 'continuous_global_proof': False, 'certificate_scope': 'fixed-policy mesh validation only', 'candidates_evaluated': 1}` の結果を代入する。
41. 上の条件が偽の場合:
42. (`u_seq`, `solve_info`) に `hybrid_bounded_minimize(cost, x0, bounds, maxiter=int(upper_max_iter), structured_seeds=[('balance_seed', build_balance_seed())], cem_enabled=bool(upper_global_search_enabled), cem_mode=upper_global_search_mode, cem_generations=int(upper_cem_generations), cem_population=int(upper_cem_cem_elite=int(upper_cem_elite), local_refine_topk=int(upper_local_refine_topk), rng_seed=int(max(0.0, round(s0_km * 10.0))), shgo_samples=int(upper_shgo_samples), shgo_iters=int(upper_shgo_iters), cert_grid_levels=int(upper_cert_grid_levels), cert_grid_values=[float(value) for value in upper_cert_grid_values], cert_max_evaluations=int(upper_cert_max_evaluations), cert_workers=int(upper_cert_workers), progress_callback=save_solver_progress, cert_progress_interval=int(cfg.get('mpc_upper_cert_progress_interval', 25)))` の結果を代入する。
43. `selected_cost_recheck` に `cost(np.asarray(u_seq, dtype=float), capture_trace=True)` の結果を代入する。
44. 条件 `upper_lock_initial_policy` を判定し、真なら内部処理を行う。
45. `solve_info['fun']` に `float(selected_cost_recheck)` の結果を代入する。
46. `prediction_trace_rows` に `list(selected_prediction.pop('trace', []))` の結果を代入する。
47. `prediction_trace_path` に `checkpoint_base.with_name(f'upper_selected_prediction_trace_{int(round(float(s0_km) * 1000.0)):010d}.csv')` の結果を代入する。
48. 条件 `prediction_trace_rows` を判定し、真なら内部処理を行う。
49. `ensure_parent_dir(...)` を実行する。
50. `pd.DataFrame(prediction_trace_rows).to_csv(...)` を実行する。
51. `solve_info['selected_prediction_trace_csv']` に `str(prediction_trace_path)` の結果を代入する。
52. 上の条件が偽の場合:

53. `solve_info['selected_prediction_trace_csv']` に” の結果を代入する。
54. `solve_info['selected_cost_recheck']` に `float(selected_cost_recheck)` の結果を代入する。
55. `solve_info['selected_prediction']` に `dict(selected_prediction)` の結果を代入する。
56. `solve_info['selected_mission_feasible']` に `bool(selected_prediction.get('mission_feasible', False))` の結果を代入する。
57. `sim_log(...)` を実行する。
58. `checkpoint_path` に `checkpoint_base.with_name('upper_solver_checkpoint.json')` の結果を代入する。
59. `ensure_parent_dir(...)` を実行する。
60. `checkpoint_payload` に `{'generated_utc': datetime.now(timezone.utc).isoformat(), 'profile_yaml': args.profile_yaml, 's_km': float(s0_km), 'prediction_steps': int(Np), 'control_dimensions': int(Nc), 'u_seq_kmh': np.asarray(u_seq, dtype=float).tolist(), 'solve_info': {key: solve_info.get(key) for key in ('success', 'method', 'label', 'message', 'fun', 'nit', 'cem_used', 'shgo_used', 'shgo_nfev', 'shgo_local_minima', 'discrete_global_proof', 'discrete_grid_levels', 'discrete_grid_values', 'discrete_grid_candidates', 'discrete_grid_nonfinite', 'discrete_grid_best_fun', 'discrete_grid_best_x', 'deterministic_seed_candidates', 'finite_library_candidates', 'deterministic_seed_nonfinite', 'selected_no_worse_than_grid', 'selected_x', 'finite_library_global_proof', 'selected_cost_recheck', 'selected_prediction', 'selected_prediction_trace_csv', 'selected_mission_feasible', 'continuous_global_proof', 'certificate_scope', 'candidates_evaluated')}}` の結果を代入する。
61. `with` 文で `checkpoint_path.open('w', encoding='utf-8')` を管理しながら処理する。
62. `json.dump(...)` を実行する。
63. `sim_log(...)` を実行する。
64. `upper_solve_log.append(...)` を実行する。
65. `v_seq` に `expand_ctrl(u_seq)` の結果を代入する。
66. `segments` に `[]` の結果を代入する。
67. `t_utc` に `t0_utc` の結果を代入する。
68. `s_km` に `float(s0_km)` の結果を代入する。
69. `z` に `float(z0)` の結果を代入する。
70. `Tb` に `float(Tb0)` の結果を代入する。
71. `enumerate(v_seq)` を順に走査し、各要素を `(idx_seg, v_k)` に入れて処理する。
72. `(t_utc, z, Tb, _)` に `step_wait(t_utc, z, Tb, s_km)` の結果を代入する。
73. `ds_step_km` に `min(float(ds_seq[idx_seg]), max(0.0, race_km - s_km))` の結果を代入する。
74. 条件 `ds_step_km <= 1e-09` を判定し、真なら内部処理を行う。
75. `Break` 文を実行する。
76. `v_k` に `float(np.clip(v_k, 0.0, v_max_kmh))` の結果を代入する。
77. `vmax_local` に `get_profile_val(speed_profile, s_km, 'v_max_kmh', v_max_kmh)` の結果を代入する。
78. 条件 `vmax_local >= v_min_solver` を判定し、真なら内部処理を行う。
79. `v_k` に `max(v_min_solver, min(v_k, vmax_local))` の結果を代入する。
80. 上の条件が偽の場合:

代表コード断片

```
def mpc_solve_distance(
    t0_utc,
    s0_km,
    z0,
    Tb0,
    v0_kmh,
    v_init=None,
    v_init_s_km=None,
):
    horizon = build_upper_distance_horizon(
        mode=upper_horizon_mode,
        s0_km=s0_km,
        race_km=race_km,
        ds_km=upper_ds_km,
        horizon_km=upper_horizon_km,
        max_steps=upper_max_steps,
        ctrl_km=upper_ctrl_km,
        adaptive_min_ds_km=upper_adaptive_min_ds_km,
        adaptive_max_ds_km=upper_adaptive_max_ds_km,
        adaptive_growth=upper_adaptive_growth,
    )
    ds_seq = np.array(horizon.ds_seq_km, dtype=float)
    seg_s = np.array(horizon.seg_s_km, dtype=float)
    Np = int(len(ds_seq))
    if Np <= 0:
        return (
            v0_kmh,
            [{'v_kmh': v0_kmh, 'dt_sec': args.dt}],
            np.array([v0_kmh], dtype=float),
            np.array([float(s0_km)], dtype=float),
        )

    v_min_solver = max(0.1, float(upper_vmin_kmh))
    ctrl_s = np.array(horizon.ctrl_s_km, dtype=float)
    ctrl_s_abs = absolute_control_distances(s0_km, ctrl_s)
    ...
```

7.70 L2400 関数 main.mpc_solve_distance.expand_ctrl

- 行範囲: L2400–L2401
- 所属: main.mpc_solve_distance
- 定義: expand_ctrl(u_vec)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: $(1.0 - \alpha) * u_vec[idx] + \alpha * u_vec[idx_next]$
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. $(1.0 - \alpha) * u_vec[idx] + \alpha * u_vec[idx_next]$ を返す。

代表コード断片

```
def expand_ctrl(u_vec):  
    return (1.0 - alpha) * u_vec[idx] + alpha * u_vec[idx_next]
```

7.71 L2403 関数 `main.mpc_solve_distance.build_balance_seed`

- 行範囲: L2403–L2503
- 所属: `main.mpc_solve_distance`
- 定義: `build_balance_seed()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `array`, `average_profile`, `clip`, `electrical_balance`, `float`, `forecast_at_time`, `get`, `get_profile_val`, `int`, `integrate_stationary_duration`, `len`
- 戻り値の要点: `np.array(u_seed, dtype=float)`
- 主なローカル代入名: `Tb_seed`, `_`, `best_score`, `best_v`, `candidate_grid`, `crossed_stops`, `ds_ctrl`, `ds_leg_km`, `dt_seed`, `dwell_done`, `dwell_sec`, `edge_s_km`, `env`, `env_leg`, `headwind_leg`, `headwind_ms`, `idx_ctrl`, `limits`, `out`, `p_pack`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 3、`try` 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `u_seed` に `np.array(x0, dtype=float)` の結果を代入する。
2. `z_seed` に `float(z0)` の結果を代入する。
3. `Tb_seed` に `float(Tb0)` の結果を代入する。
4. `s_seed` に `float(s0_km)` の結果を代入する。
5. `t_seed` に `t0_utc` の結果を代入する。
6. `v_prev_seed` に `float(v0_kmh)` の結果を代入する。
7. `range(Nc)` を順に走査し、各要素を `idx_ctrl` に入れて処理する。

8. (t_seed, z_seed, Tb_seed, _) に step_wait(t_seed, z_seed, Tb_seed, s_seed) の結果を代入する。
9. vmax_local に get_profile_val(speed_profile, s_seed, 'v_max_kmh', v_max_kmh) の結果を代入する。
10. vmin_local に v_min_solver の結果を代入する。
11. 条件 schedule is not None を判定し、真なら内部処理を行う。
12. limits に schedule.speed_limits(t_seed) の結果を代入する。
13. 条件 limits is not None を判定し、真なら内部処理を行う。
14. vmin_local に max(vmin_local, float(limits[0])) の結果を代入する。
15. vmax_local に min(vmax_local, float(limits[1])) の結果を代入する。
16. 条件 vmax_local < vmin_local を判定し、真なら内部処理を行う。
17. u_seed[idx_ctrl] に max(0.0, vmax_local) の結果を代入する。
18. Continue 文を実行する。
19. candidate_grid に np.linspace(vmin_local, vmax_local, num=max(5, min(13, int((vmax_local - vmin_local) / 4.0) + 2))) の結果を代入する。
20. env に forecast_at_time(t_seed, s_seed, drive=True) の結果を代入する。
21. slope_pct に get_profile_val(route_profile, s_seed, 'slope_pct', 0.0) の結果を代入する。
22. headwind_ms に resolved_headwind_ms(env, s_seed) の結果を代入する。
23. best_v に float(np.clip(v_prev_seed, vmin_local, vmax_local)) の結果を代入する。
24. best_score に float('inf') の結果を代入する。
25. candidate_grid を順に走査し、各要素を v_test に入れて処理する。
26. out に model.electrical_balance(v_test / 3.6, slope_pct, z_seed, Tb_seed, env['G_poa'], env['Tcell_C'], headwind_ms=headwind_ms, ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0)) の結果を代入する。
27. p_pack に float(out['P_pack']) の結果を代入する。
28. score に abs(p_pack) + 0.35 * (float(v_test) - float(v_prev_seed)) ** 2 の結果を代入する。
29. 条件 score < best_score を判定し、真なら内部処理を行う。
30. best_score に score の結果を代入する。
31. best_v に float(v_test) の結果を代入する。
32. u_seed[idx_ctrl] に best_v の結果を代入する。
33. 条件 idx_ctrl + 1 < len(ctrl_s) を判定し、真なら内部処理を行う。
34. ds_ctrl に max(0.0, float(ctrl_s[idx_ctrl + 1] - ctrl_s[idx_ctrl])) の結果を代入する。
35. 上の条件が偽の場合:
36. ds_ctrl に max(float(ds_seq[-1]), float(upper_ds_km)) の結果を代入する。
37. target_s_km に s_seed + ds_ctrl の結果を代入する。
38. crossed_stops に [stop for stop in stop_queue if s_seed + 1e-09 < float(stop.get('s_km', 0.0)) <= target_s_km + 1e-09] の結果を代入する。

39. stop_by_distance に {float(stop.get('s_km', 0.0)): stop for stop in crossed_stops} の結果を代入する。
40. seed_edges に sorted(set([*stop_by_distance, float(target_s_km)])) の結果を代入する。
41. stopped_in_chunk に False の結果を代入する。
42. seed_edges を順に走査し、各要素を edge_s_km に入れて処理する。
43. ds_leg_km に max(0.0, float(edge_s_km) - s_seed) の結果を代入する。
44. 条件 ds_leg_km > 1e-09 を判定し、真なら内部処理を行う。
45. dt_seed に ds_leg_km / max(best_v, 0.001) * 3600.0 の結果を代入する。
46. env_leg に forecast_at_time(t_seed, s_seed, drive=True) の結果を代入する。
47. slope_leg に average_profile(route_profile, s_seed, edge_s_km, 'slope_pct', 0.0) の結果を代入する。
48. headwind_leg に resolved_headwind_ms(env_leg, s_seed) の結果を代入する。
49. out に model.electrical_balance(best_v / 3.6, slope_leg, z_seed, Tb_seed, env_leg['G_poa'], env_leg['Tcell_C'], headwind_ms=headwind_leg, ambient_temp_c=env_leg.get('Tamb_C'), elevation_m=env_leg.get('elevation_m', 0.0)) の結果を代入する。
50. z_seed に float(np.clip(model.soc_step(z_seed, float(out['P_pack']), dt_seed, current_a=float(out['I']), Tbat_C=Tb_seed), p.soc_min, p.soc_max)) の結果を代入する。
51. Tb_seed に float(np.clip(Tb_seed + dt_seed / 1800.0 * (env_leg['Tamb_C'] - Tb_seed) + float(out['losses_int']) * dt_seed / 50000.0, p.T_min, p.T_max)) の結果を代入する。
52. t_seed を Add で更新する。
53. s_seed に float(edge_s_km) の結果を代入する。
54. stop に stop_by_distance.get(float(edge_s_km)) の結果を代入する。
55. 条件 stop is not None を判定し、真なら内部処理を行う。
56. dwell_sec に max(0.0, float(stop.get('dwell_sec', stop.get('dwell_s', 0.0)))) の結果を代入する。
57. (t_seed, z_seed, Tb_seed, dwell_done) に integrate_stationary_duration(t_seed, z_seed, Tb_seed, s_seed, dwell_sec, control_stop=True) の結果を代入する。
58. stopped_in_chunk に stopped_in_chunk or dwell_done > 1e-09 の結果を代入する。
59. v_prev_seed に 0.0 if stopped_in_chunk else best_v の結果を代入する。
60. np.array(u_seed, dtype=float) を返す。

代表コード断片

```
def build_balance_seed():
    u_seed = np.array(x0, dtype=float)
    z_seed = float(z0)
    Tb_seed = float(Tb0)
    s_seed = float(s0_km)
    t_seed = t0_utc
    v_prev_seed = float(v0_kmh)
    for idx_ctrl in range(Nc):
        t_seed, z_seed, Tb_seed, _ = step_wait(t_seed, z_seed, Tb_seed, s_seed)
        vmax_local = get_profile_val(speed_profile, s_seed, 'v_max_kmh', v_max_kmh)
        vmin_local = v_min_solver
        if schedule is not None:
            limits = schedule.speed_limits(t_seed)
            if limits is not None:
                vmin_local = max(vmin_local, float(limits[0]))
```

```

        vmax_local = min(vmax_local, float(limits[1]))
    if vmax_local < vmin_local:
        u_seed[idx_ctrl] = max(0.0, vmax_local)
        continue
    candidate_grid = np.linspace(vmin_local, vmax_local, num=max(5, min(13, int
((vmax_local - vmin_local) / 4.0) + 2)))
    env = forecast_at_time(t_seed, s_seed, drive=True)
    slope_pct = get_profile_val(route_profile, s_seed, 'slope_pct', 0.0)
    headwind_ms = resolved_headwind_ms(env, s_seed)
    best_v = float(np.clip(v_prev_seed, vmin_local, vmax_local))
    best_score = float("inf")
    for v_test in candidate_grid:
        out = model.electrical_balance(
            v_test / 3.6, slope_pct, z_seed, Tb_seed, env['G_poa'], env['Tcell_C
'],
            headwind_ms=headwind_ms, ambient_temp_c=env.get('Tamb_C'),
            elevation_m=env.get('elevation_m', 0.0),
        )
        p_pack = float(out['P_pack'])
        score = abs(p_pack) + 0.35 * ((float(v_test) - float(v_prev_seed)) ** 2)
        if score < best_score:
            best_score = score
...

```

7.72 L2507 関数 main.mpc_solve_distance.cost

- 行範囲: L2507–L2812
- 所属: main.mpc_solve_distance
- 定義: cost(u_vec, capture_trace=False)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: append, average_profile, bool, clear, clip, current_drive_window, electrical_balance, expand_ctrl, finish_cost, float, forecast_at_time, forecast_has_coverage
- 戻り値の要点: finish_cost(J, 'horizon_complete') / final_value / finish_cost(J + upper_cost_cfg.constraint_penalty * (1.0 + missing_km) ** 2, 'forecast_exhausted') / finish_cost(J + upper_cost_cfg.constraint_penalty * (1.0 + missing_km) ** 2, 'zero_speed_before_horizon_end')
- 主なローカル代入名: I, J, P_mech_wheel, P_pack, P_pv, Tb, Tb_next, V, day_end_crossing, ds_chunk_km, dt_travel, dt_wait, dwell_done, dwell_sec, elapsed_plan_sec, env, final_value, forces, headwind_ms, horizon_target_s_km
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 21、ループ 3、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `z` に `float(z0)` の結果を代入する。
2. `Tb` に `float(Tb0)` の結果を代入する。
3. `s_km` に `float(s0_km)` の結果を代入する。
4. `t_utc` に `t0_utc` の結果を代入する。
5. `v_prev` に `float(v0_kmh)` の結果を代入する。
6. `p_pack_prev` に `None` の結果を代入する。
7. `elapsed_plan_sec` に `0.0` の結果を代入する。
8. `J` に `0.0` の結果を代入する。
9. `soc_violation_sq` に `0.0` の結果を代入する。
10. `min_predicted_soc` に `float(z)` の結果を代入する。
11. `prediction_trace` に `[]` の結果を代入する。
12. `v_seq` に `expand_ctrl(u_vec)` の結果を代入する。
13. `horizon_target_s_km` に `float(s0_km + np.sum(ds_seq))` の結果を代入する。
14. `predicted_stop_idx` に `0` の結果を代入する。
15. 条件 `predicted_stop_idx < len(stop_queue) and float(stop_queue[predicted_stop_idx].get('s_km', 0.0)) <= s_km + 1e-09` が成り立つ間くり返す。
16. `predicted_stop_idx` を `Add` で更新する。
17. 関数 `finish_cost` を定義する。
18. `range(Np)` を順に走査し、各要素を `k_i` に入れて処理する。
19. `remaining_segment_km` に `float(ds_seq[k_i])` の結果を代入する。
20. 条件 `remaining_segment_km > 1e-09` が成り立つ間くり返す。
21. `wait_start_time` に `t_utc` の結果を代入する。
22. `wait_start_soc` に `float(z)` の結果を代入する。
23. `wait_start_tb` に `float(Tb)` の結果を代入する。
24. `dt_wait` に `0.0` の結果を代入する。
25. 条件 `predicted_stop_idx < len(stop_queue)` を判定し、真なら内部処理を行う。
26. `stop_item` に `stop_queue[predicted_stop_idx]` の結果を代入する。
27. `stop_s_km` に `float(stop_item.get('s_km', 0.0))` の結果を代入する。
28. 条件 `s_km >= stop_s_km - 1e-07` を判定し、真なら内部処理を行う。
29. `(t_utc, z, Tb, schedule_wait)` に `step_wait(t_utc, z, Tb, s_km)` の結果を代入する。
30. `dt_wait` を `Add` で更新する。
31. `elapsed_plan_sec` を `Add` で更新する。
32. 条件 `dt_wait > 1e-09` を判定し、真なら内部処理を行う。

33. `v_prev` に 0.0 の結果を代入する。
34. 条件 `capture_trace and dt_wait > 1e-09` を判定し、真なら内部処理を行う。
35. `prediction_trace.append(...)` を実行する。
36. 条件 `not forecast_has_coverage(t_utc)` を判定し、真なら内部処理を行う。
37. `missing_km` に `max(0.0, horizon_target_s_km - s_km)` の結果を代入する。
38. `finish_cost(J + upper_cost_cfg.constraint_penalty * (1.0 + missing_km) ** 2, 'forecast_exhausted')` を返す。
39. `v_k` に `float(v_seq[k_i])` の結果を代入する。
40. `vmax_local` に `get_profile_val(speed_profile, s_km, 'v_max_kmh', v_max_kmh)` の結果を代入する。
41. 条件 `vmax_local >= v_min_solver` を判定し、真なら内部処理を行う。
42. `v_k` に `max(v_min_solver, min(v_k, vmax_local))` の結果を代入する。
43. 上の条件が偽の場合:
44. `v_k` に `max(0.0, min(v_k, vmax_local))` の結果を代入する。
45. `limits` に `schedule.speed_limits(t_utc) if schedule is not None else None` の結果を代入する。
46. 条件 `limits is not None` を判定し、真なら内部処理を行う。
47. `v_k` に `max(float(limits[0]), min(v_k, float(limits[1])))` の結果を代入する。
48. 条件 `v_k <= 1e-06` を判定し、真なら内部処理を行う。
49. `missing_km` に `max(0.0, horizon_target_s_km - s_km)` の結果を代入する。
50. `finish_cost(J + upper_cost_cfg.constraint_penalty * (1.0 + missing_km) ** 2, 'zero_speed_before_horizon_end')` を返す。
51. `max_chunk_sec` に `integration_step_seconds()` の結果を代入する。
52. 条件 `schedule is not None` を判定し、真なら内部処理を行う。
53. `win` に `schedule.current_drive_window(t_utc)` の結果を代入する。
54. 条件 `win is None` を判定し、真なら内部処理を行う。
55. `max_chunk_sec` に `min(max_chunk_sec, max(0.0, (win[1] - t_utc).total_seconds()))` の結果を代入する。
56. 条件 `max_chunk_sec <= 1e-09` を判定し、真なら内部処理を行う。
57. `t_utc` を Add で更新する。
58. Continue 文を実行する。
59. `ds_chunk_km` に `min(remaining_segment_km, v_k * max_chunk_sec / 3600.0)` の結果を代入する。
60. 条件 `predicted_stop_idx < len(stop_queue)` を判定し、真なら内部処理を行う。
61. `stop_s_km` に `float(stop_queue[predicted_stop_idx].get('s_km', 0.0))` の結果を代入する。
62. 条件 `s_km < stop_s_km < s_km + ds_chunk_km` を判定し、真なら内部処理を行う。
63. `dt_travel` に `ds_chunk_km / v_k * 3600.0` の結果を代入する。
64. `env` に `forecast_at_time(t_utc, s_km, drive=True)` の結果を代入する。
65. `slope_pct` に `average_profile(route_profile, s_km, s_km + ds_chunk_km, 'slope_pct', 0.0)` の結果を代入する。

66. headwind_ms に resolved_headwind_ms(env, s_km) の結果を代入する。
67. kinetic_delta_wh に $0.5 * p.m * ((v_k / 3.6) ** 2 - (v_{prev} / 3.6) ** 2) / 3600.0$ の結果を代入する。
68. inertial_power_w に $kinetic_delta_wh * 3600.0 / \max(dt_travel, 1e-09)$ の結果を代入する。
69. out に model.electrical_balance(v_k/3.6, slope_pct, z, Tb, env['G_poa'], env['Tcell_C'], headwind_ms=headwind_ms, inertial_power_w=inertial_power_w, ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0)) の結果を代入する。
70. I に float(out['I']) の結果を代入する。
71. V に float(out['V']) の結果を代入する。
72. P_pv に float(out.get('P_pv', 0.0)) の結果を代入する。
73. P_pack に float(out['P_pack']) の結果を代入する。
74. loss_int に float(out['losses_int']) の結果を代入する。
75. loss_line に float(out.get('losses_line', 0.0)) の結果を代入する。
76. P_mech_wheel に float(out.get('P_mech_wheel', 0.0)) の結果を代入する。
77. kinetic_step_wh に max(0.0, kinetic_delta_wh) の結果を代入する。
78. z_next_raw に model.soc_step(z, P_pack, dt_travel, current_a=I, Tbat_C=Tb) の結果を代入する。
79. min_predicted_soc に min(min_predicted_soc, float(z_next_raw)) の結果を代入する。
80. soc_violation_sq を Add で更新する。

代表コード断片

```
def cost(u_vec, capture_trace=False):
    z = float(z0)
    Tb = float(Tb0)
    s_km = float(s0_km)
    t_utc = t0_utc
    v_prev = float(v0_kmh)
    p_pack_prev = None
    elapsed_plan_sec = 0.0
    J = 0.0
    soc_violation_sq = 0.0
    min_predicted_soc = float(z)
    prediction_trace = []
    v_seq = expand_ctrl(u_vec)
    horizon_target_s_km = float(s0_km + np.sum(ds_seq))
    predicted_stop_idx = 0
    while (
        predicted_stop_idx < len(stop_queue)
        and float(stop_queue[predicted_stop_idx].get('s_km', 0.0)) <= s_km + 1.0e-9
    ):
        predicted_stop_idx += 1

    def finish_cost(value, status):
        status_text = str(status)
        missing_distance_km = max(0.0, horizon_target_s_km - s_km)
        mission_violation_sq = float(soc_violation_sq)
        if status_text != 'horizon_complete':
            missing_fraction = missing_distance_km / max(1.0, horizon_target_s_km -
s0_km)

            # A truncated prediction is always lexicographically worse than
            # every full-horizon feasible candidate, irrespective of soft weights.
            mission_violation_sq += 1.0 + missing_fraction * missing_fraction
```

```

        mission_feasible = bool(
            status_text == 'horizon_complete' and mission_violation_sq <= 1.0e-12
        )
        final_value = float(value)
        if upper_cost_cfg.objective_mode in {'fastest', 'fastest_feasible', '
minimum_time'}:
...

```

7.73 L2528 関数 main.mpc_solve_distance.cost.finish_cost

- 行範囲: L2528–L2567
- 所属: main.mpc_solve_distance.cost
- 定義: finish_cost(value, status)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: bool, clear, float, isoformat, max, str, update
- 戻り値の要点: final_value
- 主なローカル代入名: final_value, missing_distance_km, missing_fraction, mission_feasible, mission_violation_sq, status_text
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. status_text に str(status) の結果を代入する。
2. missing_distance_km に max(0.0, horizon_target_s_km - s_km) の結果を代入する。
3. mission_violation_sq に float(soc_violation_sq) の結果を代入する。
4. 条件 status_text != 'horizon_complete' を判定し、真なら内部処理を行う。
5. missing_fraction に missing_distance_km / max(1.0, horizon_target_s_km - s0_km) の結果を代入する。
6. mission_violation_sq を Add で更新する。
7. mission_feasible に bool(status_text == 'horizon_complete' and mission_violation_sq <= 1e-12) の結果を代入する。
8. final_value に float(value) の結果を代入する。
9. 条件 upper_cost_cfg.objective_mode in {'fastest', 'fastest_feasible', 'minimum_time'} を判定し、真なら内部処理を行う。
10. final_value に float(elapsed_plan_sec) if mission_feasible else 1e+18 + 1000000000000000.0 * mission_violation_sq + float(elapsed_plan_sec) の結果を代入する。

11. 条件 `capture_trace` を判定し、真なら内部処理を行う。
12. `selected_prediction.clear(...)` を実行する。
13. `selected_prediction.update(...)` を実行する。
14. `selected_prediction['trace']` に `prediction_trace` の結果を代入する。
15. `final_value` を返す。

代表コード断片

```
def finish_cost(value, status):
    status_text = str(status)
    missing_distance_km = max(0.0, horizon_target_s_km - s_km)
    mission_violation_sq = float(soc_violation_sq)
    if status_text != 'horizon_complete':
        missing_fraction = missing_distance_km / max(1.0, horizon_target_s_km -
s0_km)
        # A truncated prediction is always lexicographically worse than
        # every full-horizon feasible candidate, irrespective of soft weights.
        mission_violation_sq += 1.0 + missing_fraction * missing_fraction
    mission_feasible = bool(
        status_text == 'horizon_complete' and mission_violation_sq <= 1.0e-12
    )
    final_value = float(value)
    if upper_cost_cfg.objective_mode in {'fastest', 'fastest_feasible', '
minimum_time'}:
        final_value = (
            float(elapsed_plan_sec)
            if mission_feasible
            else 1.0e18 + 1.0e15 * mission_violation_sq + float(elapsed_plan_sec
)
        )
    if capture_trace:
        selected_prediction.clear()
        selected_prediction.update(
            {
                'status': status_text,
                'objective': final_value,
                'terminal_time_utc': t_utc.isoformat(),
                'terminal_distance_km': float(s_km),
                'target_distance_km': float(horizon_target_s_km),
                'missing_distance_km': float(missing_distance_km),
                'terminal_soc': float(z),
                'minimum_soc': float(min_predicted_soc),
                'soc_violation_sq': float(soc_violation_sq),
                'mission_violation_sq': float(mission_violation_sq),
                'mission_feasible': mission_feasible,
                'terminal_tb_c': float(Tb),
            }
        )
    ...
```

7.74 L2818 関数 `main.mpc_solve_distance.save_solver_progress`

- 行範囲: L2818–L2836
- 所属: `main.mpc_solve_distance`
- 定義: `save_solver_progress(progress:dict)->None`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: dict, dump, ensure_parent_dir, float, get, int, isoformat, now, open, replace, sim_log, str
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: handle, payload, tmp_path
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. payload に {'generated_utc': datetime.now(timezone.utc).isoformat(), 'profile_yaml': args.profile_yaml, 's_km': float(s0_km), 'prediction_steps': int(Np), 'control_dimensions': int(Nc), **dict(progress)} の結果を代入する。
2. ensure_parent_dir(...) を実行する。
3. tmp_path に progress_path.with_suffix(progress_path.suffix + '.tmp') の結果を代入する。
4. with 文で tmp_path.open('w', encoding='utf-8') を管理しながら処理する。
5. json.dump(...) を実行する。
6. tmp_path.replace(...) を実行する。
7. sim_log(...) を実行する。

代表コード断片

```
def save_solver_progress(progress: dict) -> None:
    payload = {
        'generated_utc': datetime.now(timezone.utc).isoformat(),
        'profile_yaml': args.profile_yaml,
        's_km': float(s0_km),
        'prediction_steps': int(Np),
        'control_dimensions': int(Nc),
        **dict(progress),
    }
    ensure_parent_dir(str(progress_path))
    tmp_path = progress_path.with_suffix(progress_path.suffix + '.tmp')
    with tmp_path.open('w', encoding='utf-8') as handle:
        json.dump(payload, handle, ensure_ascii=False, indent=2, default=str)
    tmp_path.replace(progress_path)
    sim_log(
        f"upper solve progress: stage={payload.get('stage', '')} "
        f"completed={payload.get('completed', '')}/{payload.get('total', '')} "
        f"best={payload.get('best_fun', '')}"
    )
```

7.75 L3022 関数 `main.propagate_execution_step`

- 行範囲: L3022–L3199
- 所属: `main`
- 定義: `propagate_execution_step(cmd_kmh, z0, Tb0, s0_km, env, *, force_stop=False, race_distance_km=None, max_elapsed_sec=None, env_provider=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `advance_rate_limiter_to_distance_boundary`, `append`, `bool`, `clear`, `clip`, `dict`, `electrical_balance`, `env_provider`, `float`, `get`, `limit_step_duration_to_distance`, `max`
- 戻り値の要点: `(z, Tb, s, {'elapsed_sec': float(elapsed_sec), 'v_exec_kmh': float(sum_v_exec_dt / denom), 'v_exec_target_kmh': float(sum_v_target_dt / denom), 'v_exec_ms': float(sum_v_exec_dt / denom / 3.6), 'out': avg_out, 'forces': avg_forces, 'substeps': substeps}) / (z0, Tb0, float(race_distance_km), {'elapsed_sec': 0.0, 'v_exec_kmh': 0.0, 'v_exec_target_kmh': 0.0, 'v_exec_ms': 0.0, 'out': {}, 'forces': {}, 'substeps': []})`
- 主なローカル代入名: `Tb`, `Tb_before`, `_`, `acceleration_ms2`, `avg_forces`, `avg_out`, `cmd_kmh`, `delayed_cmd`, `denom`, `dt_sub`, `elapsed_sec`, `env_sub`, `exec_sim_time_sec`, `exec_target_kmh`, `force_metric_keys`, `forces`, `headwind_ms`, `inertial_power_w`, `key`, `kinetic_delta_j`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 10、ループ 4、`try` 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `Nonlocal` 文を実行する。
2. `slope_pct` に `float(env['slope_pct'])` の結果を代入する。
3. `headwind_ms` に `float(env['headwind_ms'])` の結果を代入する。
4. `step_sec` に `max(1e-06, float(args.dt))` の結果を代入する。
5. 条件 `max_elapsed_sec is not None` を判定し、真なら内部処理を行う。
6. `step_sec` に `min(step_sec, max(1e-06, float(max_elapsed_sec)))` の結果を代入する。
7. `cmd_kmh` に `float(np.clip(cmd_kmh, 0.0, v_max_kmh))` の結果を代入する。
8. 条件 `race_distance_km is not None and s0_km >= float(race_distance_km) - DISTANCE_EPS_KM` を判定し、真なら内部処理を行う。
9. `(z0, Tb0, float(race_distance_km), {'elapsed_sec': 0.0, 'v_exec_kmh': 0.0, 'v_exec_target_kmh': 0.0, 'v_exec_ms': 0.0, 'out': {}, 'forces': {}, 'substeps': []})` を返す。
10. `out_metric_keys` に `('P_pv_raw', 'P_pv_unlimited', 'P_pv_limit_loss', 'P_pv', 'eta_panel', 'eta_mppt', 'P_aux', 'P_road_load', 'P_inertia', 'P_mech', 'P_mech_wheel', 'P_dc_to_drv', 'P_reg_to_dc', 'P_pack', 'I', 'V', 'OCV', 'Rint', 'Rline', 'Rpolarization', 'Rtotal', 'iv_discriminant', 'eta_charge', 'losses_int', 'losses_rint', 'losses_line', 'losses_polarization', 'eff_drv', 'eff_reg', 'torque_drive_nm', 'torque_regen_nm', 'omega_motor_radps', 'omega_wheel_radps')` の結果を代入する。

11. `force_metric_keys` に ('F_aero', 'F_roll', 'F_grade', 'F_total', 'theta', 'v_rel_ms', 'normal_force_n', 'Crr_eff', 'slope_scaled_pct') の結果を代入する。
12. 条件 `exec_model_enabled` を判定し、真なら内部処理を行う。
13. 条件 `force_stop or cmd_kmh <= 0.1` を判定し、真なら内部処理を行う。
14. `exec_delay_queue.clear(...)` を実行する。
15. `exec_delay_queue.append(...)` を実行する。
16. 上の条件が偽の場合:
17. `exec_delay_queue.append(...)` を実行する。
18. `z` に `float(z0)` の結果を代入する。
19. `Tb` に `float(Tb0)` の結果を代入する。
20. `s` に `float(s0_km)` の結果を代入する。
21. `remaining` に `step_sec` の結果を代入する。
22. `elapsed_sec` に `0.0` の結果を代入する。
23. `sum_v_exec_dt` に `0.0` の結果を代入する。
24. `sum_v_target_dt` に `0.0` の結果を代入する。
25. `sum_metrics` に `{key: 0.0 for key in out_metric_keys + force_metric_keys}` の結果を代入する。
26. `substeps` に `[]` の結果を代入する。
27. 条件 `remaining > 1e-09` が成り立つ間くり返す。
28. 条件 `race_distance_km is not None and s >= float(race_distance_km) - DISTANCE_EPS_KM` を判定し、真なら内部処理を行う。
29. `s` に `float(race_distance_km)` の結果を代入する。
30. `Break` 文を実行する。
31. `dt_sub` に `min(exec_inner_dt_sec, remaining)` の結果を代入する。
32. `v_previous_kmh` に `float(v_exec)` の結果を代入する。
33. 条件 `exec_model_enabled` を判定し、真なら内部処理を行う。
34. 条件 `exec_delay_queue and float(exec_delay_queue[0][0]) <= exec_sim_time_sec + 1e-09` が成り立つ間くり返す。
35. `(_, delayed_cmd)` に `exec_delay_queue.popleft()` の結果を代入する。
36. `exec_target_kmh` に `float(np.clip(delayed_cmd, 0.0, v_max_kmh))` の結果を代入する。
37. `(v_exec, dt_sub)` に `advance_rate_limiter_to_distance_boundary(exec_limiter, exec_target_kmh, start_time_sec=exec_sim_time_sec, step_sec=dt_sub, s0_km=s, distance_limit_km=race_distance_km)` の結果を代入する。
38. `(v_exec, stop_snapped)` に `snap_execution_stop_speed_kmh(v_exec, exec_target_kmh, stop_requested=bool(force_stop or cmd_kmh <= 0.1), deadband_kmh=exec_deadband_kmh, quantize_step_kmh=exec_quantize_step_kmh)` の結果を代入する。
39. 条件 `stop_snapped` を判定し、真なら内部処理を行う。
40. `exec_limiter.reset(...)` を実行する。

41. 上の条件が偽の場合:
42. `exec_target_kmh` に `float(cmd_kmh)` の結果を代入する。
43. `v_exec` に `float(cmd_kmh)` の結果を代入する。
44. `dt_sub` に `limit_step_duration_to_distance(dt_sub, v_exec, s, race_distance_km)` の結果を代入する。
45. 条件 `dt_sub <= 1e-09` を判定し、真なら内部処理を行う。
46. 条件 `race_distance_km is not None` を判定し、真なら内部処理を行う。
47. `s` に `min(float(race_distance_km), float(s))` の結果を代入する。
48. Break 文を実行する。
49. `env_sub` に `env_provider(elapsed_sec, s)` if `env_provider` is not None else `env` の結果を代入する。
50. `slope_pct` に `float(env_sub['slope_pct'])` の結果を代入する。
51. `headwind_ms` に `float(env_sub['headwind_ms'])` の結果を代入する。
52. `v_ms` に `v_exec / 3.6` の結果を代入する。
53. `v_previous_ms` に `v_previous_kmh / 3.6` の結果を代入する。
54. `acceleration_ms2` に `(v_ms - v_previous_ms) / max(dt_sub, 1e-09)` の結果を代入する。
55. `kinetic_delta_j` に `0.5 * model.p.m * (v_ms * v_ms - v_previous_ms * v_previous_ms)` の結果を代入する。
56. `inertial_power_w` に `kinetic_delta_j / max(dt_sub, 1e-09)` の結果を代入する。
57. `z_before` に `float(z)` の結果を代入する。
58. `Tb_before` に `float(Tb)` の結果を代入する。
59. `s_before` に `float(s)` の結果を代入する。
60. `out` に `model.electrical_balance(v_ms, slope_pct, z, Tb, env_sub['G_poa'], env_sub['Tcell_C'], headwind_ms=headwind_ms, aux_power_w=env_sub.get('aux_power_w'), inertial_power_w=inertial_power_w, ambient_temp_c=env_sub.get('Tamb_C'), elevation_m=env_sub.get('elevation_m', 0.0))` の結果を代入する。
61. `loss_int` に `float(out['losses_int'])` の結果を代入する。
62. `forces` に `model.resistive_forces(v_ms, slope_pct, headwind_ms=headwind_ms, ambient_temp_c=env_sub.get('Tamb_C'), elevation_m=env_sub.get('elevation_m', 0.0))` の結果を代入する。
63. `z` に `model.soc_step(z, float(out['P_pack']), dt_sub, current_a=float(out['I']), Tbat_C=Tb)` の結果を代入する。
64. `Tb` に `Tb + dt_sub / 1800.0 * (env_sub['Tamb_C'] - Tb) + loss_int * dt_sub / 50000.0` の結果を代入する。
65. `z` に `float(np.clip(z, model.p.soc_min, model.p.soc_max))` の結果を代入する。
66. `Tb` に `float(np.clip(Tb, model.p.T_min, model.p.T_max))` の結果を代入する。
67. `s` を Add で更新する。
68. 条件 `race_distance_km is not None` を判定し、真なら内部処理を行う。
69. `race_distance` に `float(race_distance_km)` の結果を代入する。
70. `s` に `race_distance` if `s >= race_distance - DISTANCE_EPS_KM` else `min(race_distance, float(s))` の結果を代入する。
71. `elapsed_sec` を Add で更新する。

72. sum_v_exec_dt を Add で更新する。
73. sum_v_target_dt を Add で更新する。
74. out_metric_keys を順に走査し、各要素を key に入れて処理する。
75. sum_metrics[key] を Add で更新する。
76. force_metric_keys を順に走査し、各要素を key に入れて処理する。
77. sum_metrics[key] を Add で更新する。
78. substeps.append(...) を実行する。
79. exec_sim_time_sec を Add で更新する。
80. remaining を Sub で更新する。

代表コード断片

```
def propagate_execution_step(
    cmd_kmh,
    z0,
    Tb0,
    s0_km,
    env,
    *,
    force_stop=False,
    race_distance_km=None,
    max_elapsed_sec=None,
    env_provider=None,
):
    nonlocal exec_sim_time_sec, exec_target_kmh, v_exec
    slope_pct = float(env['slope_pct'])
    headwind_ms = float(env['headwind_ms'])
    step_sec = max(1.0e-6, float(args.dt))
    if max_elapsed_sec is not None:
        step_sec = min(step_sec, max(1.0e-6, float(max_elapsed_sec)))
    cmd_kmh = float(np.clip(cmd_kmh, 0.0, v_max_kmh))
    if race_distance_km is not None and s0_km >= (float(race_distance_km) -
DISTANCE_EPS_KM):
        return z0, Tb0, float(race_distance_km), {
            'elapsed_sec': 0.0,
            'v_exec_kmh': 0.0,
            'v_exec_target_kmh': 0.0,
            'v_exec_ms': 0.0,
            'out': {},
            'forces': {},
            'substeps': [],
        }
    out_metric_keys = (
        'P_pv_raw', 'P_pv_unlimited', 'P_pv_limit_loss', 'P_pv',
        'eta_panel', 'eta_mppt', 'P_aux',
        'P_road_load', 'P_inertia', 'P_mech', 'P_mech_wheel', 'P_dc_to_drv', '
P_reg_to_dc',
        'P_pack', 'I', 'V', 'OCV', 'Rint', 'Rline', 'Rpolarization', 'Rtotal',
        'iv_discriminant', 'eta_charge', 'losses_int', 'losses_rint', 'losses_line', '
losses_polarization',
    ...
```

7.76 L3201 関数 `main.write_execution_detail`

- 行範囲: L3201–L3364
- 所属: `main`
- 定義: `write_execution_detail(substep, outer_time_utc, *, drive_now, hard_stop, upper_speed_cmd_kmh, speed_safety_override_active, soc_guard_intervened, plan_id, outer_step_index, outer_step_dt_sec, outer_boundary_reason)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `append`, `astimezone`, `bool`, `float`, `get`, `int`, `isoformat`, `max`, `min`, `select_drive_mode`, `startswith`
- 戻り値の要点: `row`
- 主なローカル代入名: `acceleration_ms2_sub`, `detail_min_soc`, `dt_sub`, `energy_fields`, `env_sub`, `forces_sub`, `label`, `lower_command_index`, `out_sub`, `power_key`, `row`, `s_end`, `s_start`, `soc_guard_intervention_rows`, `soc_guard_intervention_sec`, `step_end_utc`, `step_time_utc`, `torque_drive_nm`, `v_cmd_sub`, `v_exec_ms_sub`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `Nonlocal` 文を実行する。
2. `Nonlocal` 文を実行する。
3. `dt_sub` に `float(substep['dt_sec'])` の結果を代入する。
4. `step_time_utc` に `outer_time_utc + timedelta(seconds=float(substep['elapsed_from_step_sec']))` の結果を代入する。
5. `step_end_utc` に `step_time_utc + timedelta(seconds=dt_sub)` の結果を代入する。
6. `out_sub` に `substep['out']` の結果を代入する。
7. `forces_sub` に `substep['forces']` の結果を代入する。
8. `env_sub` に `substep['env']` の結果を代入する。
9. `z_start` に `float(substep['z_start'])` の結果を代入する。
10. `z_end` に `float(substep['z_end'])` の結果を代入する。
11. `s_start` に `float(substep['s_start_kmh'])` の結果を代入する。
12. `s_end` に `float(substep['s_end_kmh'])` の結果を代入する。
13. `v_cmd_sub` に `float(substep['v_cmd_kmh'])` の結果を代入する。
14. `v_exec_sub` に `float(substep['v_exec_kmh'])` の結果を代入する。

15. `v_exec_previous_sub` に `float(substep.get('v_exec_previous_kmh', v_exec_sub))` の結果を代入する。
16. `v_target_sub` に `float(substep['v_exec_target_kmh'])` の結果を代入する。
17. `v_exec_ms_sub` に `v_exec_sub / 3.6` の結果を代入する。
18. `acceleration_ms2_sub` に `float(substep.get('acceleration_ms2', 0.0))` の結果を代入する。
19. `energy_fields` に `{}` の結果を代入する。
20. `energy_power_keys` を順に走査し、各要素を `power_key` に入れて処理する。
21. `label` に `power_key[2:] if power_key.startswith('P_') else power_key` の結果を代入する。
22. `value_wh` に `float(out_sub.get(power_key, 0.0)) * dt_sub / 3600.0` の結果を代入する。
23. `cumulative_energy_wh[power_key]` を Add で更新する。
24. `energy_fields[f'E_{label}_step_Wh']` に `value_wh` の結果を代入する。
25. `energy_fields[f'E_{label}_cumulative_Wh']` に `cumulative_energy_wh[power_key]` の結果を代入する。
26. `lower_command_index` を Add で更新する。
27. 条件 `soc_guard_intervened` を判定し、真なら内部処理を行う。
28. `soc_guard_intervention_rows` を Add で更新する。
29. `soc_guard_intervention_sec` を Add で更新する。
30. `detail_min_soc` に `min(detail_min_soc, z_start, z_end)` の結果を代入する。
31. `detail_tracking_errors.append(...)` を実行する。
32. `torque_drive_nm` に `float(out_sub.get('torque_drive_nm', 0.0))` の結果を代入する。
33. `row` に `{'lower_command_index': int(lower_command_index), 'outer_step_index': int(outer_step_index), 'time_utc': step_time_utc.isoformat(), 'time_end_utc': step_end_utc.isoformat(), 'time_local': step_time_utc.astimezone(), 'step_dt_sec': dt_sub, 'detail_target_dt_sec': detail_interval_sec, 'detail_step_kind': 'lower_command_cycle' if abs(dt_sub - detail_interval_sec) <= 1e-09 else 'boundary_remainder', 'lower_command_rate_hz': 1.0 / max(dt_sub, 1e-09), 'outer_step_requested_dt_sec': float(args.dt), 'outer_step_actual_dt_sec': float(outer_step_dt_sec), 'outer_step_boundary_reason': str(outer_boundary_reason or 'planner_step_complete'), 'is_drive_window': bool(drive_now), 'is_forced_stop': bool(hard_stop), 'plan_id': int(plan_id), 's_kmh': s_start, 's_end_kmh': s_end, 'ds_step_kmh': s_end - s_start, 'upper_speed_cmd_kmh': float(upper_speed_cmd_kmh), 'speed_safety_override_active': bool(speed_safety_override_active), 'soc_guard_intervened': bool(soc_guard_intervened), 'lower_reference_target_kmh': v_target_sub, 'lower_speed_cmd_kmh': v_exec_sub, 'v_cmd_kmh': v_cmd_sub, 'v_exec_kmh': v_exec_sub, 'v_exec_previous_kmh': v_exec_previous_sub, 'v_exec_target_kmh': v_target_sub, 'v_cmd_ms': v_cmd_sub / 3.6, 'v_exec_ms': v_exec_ms_sub, 'acceleration_ms2': acceleration_ms2_sub, 'soc': z_start, 'soc_end': z_end, 'usable_battery_start_Wh': max(0.0, (z_start - model.p.soc_min) * model.p.E_nom_Wh), 'usable_battery_end_Wh': max(0.0, (z_end - model.p.soc_min) * model.p.E_nom_Wh), 'battery_energy_used_step_Wh': float(out_sub.get('P_pack', 0.0)) * dt_sub / 3600.0, 'battery_charge_used_step_Ah': (z_start - z_end) * float(model.p.Q_nom_Ah), 'soc_state_definition': 'charge' if float(model.p.Q_nom_Ah) > 0.0 else 'energy', 'Tb_C': float(substep['Tb_start']), 'Tb_end_C': float(substep['Tb_end']), 'slope_pct': float(env_sub.get('slope_pct', 0.0)), 'slope_scaled_pct': float(forces_sub.get('slope', env_sub.get('slope_pct', 0.0))), 'headwind_ms': float(env_sub.get('headwind_ms', 0.0)), 'v_rel_ms': float(forces_sub.get('v_rel_ms', v_exec_ms_sub + env_sub.get('headwind_ms', 0.0))), 'G_raw': float(env_sub.get('G_raw', env_sub.get('G_poa', 0.0))), 'G_drive_poa': float(env_sub.get('G_drive_poa', env_sub.get('G_raw', 0.0))), 'G_stop_poa': float(env_sub.get('G_stop_poa', env_sub.get('G_raw', 0.0))), 'G_control_stop_poa': float(env_sub.get('G_control_stop_poa', env_sub.get('G_raw', 0.0))), 'G_poa': float(env_sub.get('G_poa', 0.0)), 'Tamb_C': float(env_sub.get('Tamb_C', 30.0)), 'Tcell_drive_C': float(env_sub.get('Tcell_drive_C', env_sub.get('Tcell_C', 40.0))), 'Tcell_stop_C': float(env_sub.get('Tcell_stop_C', env_sub.get('Tcell_C', 40.0)))}`

```

40.0))), 'Tcell_control_stop_C': float(env_sub.get('Tcell_control_stop_C', env_sub.get('Tcell_C', 40.0))),
'Tcell_C': float(env_sub.get('Tcell_C', 40.0)), 'panel_stop_mode': str(env_sub.get('panel_stop_mode',
'drive')), 'P_pv_raw': float(out_sub.get('P_pv_raw', 0.0)), 'P_pv_unlimited': float(out_sub.get('P_pv_unlimited',
out_sub.get('P_pv', 0.0))), 'P_pv_limit_loss': float(out_sub.get('P_pv_limit_loss', 0.0)), 'eta_panel':
float(out_sub.get('eta_panel', 0.0)), 'eta_mppt': float(out_sub.get('eta_mppt', 0.0)), 'P_pv': float(out_sub.get('P_pv',
0.0)), 'P_solar_w': float(out_sub.get('P_pv', 0.0)), 'P_aux': float(out_sub.get('P_aux', 0.0)), 'P_road_load':
float(out_sub.get('P_road_load', 0.0)), 'P_inertia': float(out_sub.get('P_inertia', 0.0)), 'P_mech': float(out_sub.get('P_mech',
0.0)), 'P_mech_wheel': float(out_sub.get('P_mech_wheel', 0.0)), 'P_dc_to_drv': float(out_sub.get('P_dc_to_drv',
0.0)), 'P_reg_to_dc': float(out_sub.get('P_reg_to_dc', 0.0)), 'P_pack': float(out_sub.get('P_pack', 0.0)),
'P_vehicle_load_w': float(out_sub.get('P_dc_to_drv', 0.0)) - float(out_sub.get('P_reg_to_dc', 0.0)) +
float(out_sub.get('P_aux', 0.0)), 'P_net_battery_w': float(out_sub.get('P_pack', 0.0)), 'P_battery_terminal':
float(out_sub.get('I', 0.0)) * float(out_sub.get('V', 0.0)), 'power_balance_residual_W': float(out_sub.get('P_pack',
0.0)) - float(out_sub.get('I', 0.0)) * float(out_sub.get('V', 0.0)), 'I': float(out_sub.get('I', 0.0)), 'V':
float(out_sub.get('V', 0.0)), 'OCV': float(out_sub.get('OCV', 0.0)), 'Rint': float(out_sub.get('Rint',
0.0)), 'Rline': float(out_sub.get('Rline', 0.0)), 'Rpolarization': float(out_sub.get('Rpolarization', 0.0)),
'Rtotal': float(out_sub.get('Rtotal', 0.0)), 'iv_discriminant': float(out_sub.get('iv_discriminant', 0.0)),
'eta_charge': float(out_sub.get('eta_charge', 1.0)), 'losses_int': float(out_sub.get('losses_int', 0.0)),
'losses_rint': float(out_sub.get('losses_rint', 0.0)), 'losses_line': float(out_sub.get('losses_line', 0.0)),
'losses_polarization': float(out_sub.get('losses_polarization', 0.0)), 'eff_drv': float(out_sub.get('eff_drv',
0.0)), 'eff_reg': float(out_sub.get('eff_reg', 0.0)), 'torque_drive_nm': torque_drive_nm, 'torque_regen_nm':
float(out_sub.get('torque_regen_nm', 0.0)), 'omega_motor_radps': float(out_sub.get('omega_motor_radps',
0.0)), 'omega_wheel_radps': float(out_sub.get('omega_wheel_radps', 0.0)), 'drive_mode_selected':
model.select_drive_mode(v_exec_ms_sub, torque_drive_nm), 'F_aero': float(forces_sub.get('F_aero',
0.0)), 'F_roll': float(forces_sub.get('F_roll', 0.0)), 'F_grade': float(forces_sub.get('F_grade', 0.0)),
'F_total': float(forces_sub.get('F_total', 0.0)), 'normal_force_n': float(forces_sub.get('normal_force_n', 0.0)),
'theta_rad': float(forces_sub.get('theta', 0.0)), 'Crr_eff': float(forces_sub.get('Crr_eff', model.p.Crr)), 'elevation_m':
float(env_sub.get('elevation_m', 0.0)), 'air_density_kgm3': float(forces_sub.get('air_density_kgm3',
model.p.rho))} の結果を代入する。

```

34. row.update(...) を実行する。

35. row.update(...) を実行する。

36. detail_stream.write(...) を実行する。

37. row を返す。

代表コード断片

```

def write_execution_detail(
    substep,
    outer_time_utc,
    *,
    drive_now,
    hard_stop,
    upper_speed_cmd_kmh,
    speed_safety_override_active,
    soc_guard_intervened,
    plan_id,
    outer_step_index,
    outer_step_dt_sec,
    outer_boundary_reason,
):
    nonlocal lower_command_index, detail_min_soc
    nonlocal soc_guard_intervention_rows, soc_guard_intervention_sec
    dt_sub = float(substep['dt_sec'])
    step_time_utc = outer_time_utc + timedelta(seconds=float(substep['
elapsed_from_step_sec']))
    step_end_utc = step_time_utc + timedelta(seconds=dt_sub)
    out_sub = substep['out']

```

```

forces_sub = substep['forces']
env_sub = substep['env']
z_start = float(substep['z_start'])
z_end = float(substep['z_end'])
s_start = float(substep['s_start_kmh'])
s_end = float(substep['s_end_kmh'])
v_cmd_sub = float(substep['v_cmd_kmh'])
v_exec_sub = float(substep['v_exec_kmh'])
v_exec_previous_sub = float(substep.get('v_exec_previous_kmh', v_exec_sub))
v_target_sub = float(substep['v_exec_target_kmh'])
v_exec_ms_sub = v_exec_sub / 3.6
acceleration_ms2_sub = float(substep.get('acceleration_ms2', 0.0))

energy_fields = {}
for power_key in energy_power_keys:
...

```

7.77 CLI 引数

行	引数
1487	--profile_yaml
1488	--forecast_csv
1489	--forecast_fill_csv
1490	--progress_reference_csv
1491	--forecast_time_mode
1492	--forecast_time_tz
1493	--route_profile_csv
1494	--speed_profile_csv
1495	--params_yaml
1496	--stop_yaml
1497	--drive_schedule_yaml
1498	--panel_eff_map
1499	--mppt_eff_map
1500	--ocv_soc_map
1501	--drive_eff_map
1502	--regen_eff_map
1503	--rint_map
1504	--drive_map_eco
1505	--drive_map_power
1506	--regen_map_eco
1507	--regen_map_power
1508	--dt
1509	--horizon_steps
1510	--soc0
1511	--Tb0
1512	--v0_kmh
1513	--start_utc
1514	--forecast_start_time_utc
1515	--start_index
1516	--start_s_km

```
1517 --resume_csv
1518 --resume_s_km
1519 --out_csv
1520 --out_detail_csv
1521 --report_html
1522 --summary_json
1523 --resolved_yaml
1524 --latest_manifest_json
1525 --override
1526 --soc_guard_margin
1527 --soc_guard_mode
1528 --solar_gain
1529 --poa_gain_drive
1530 --poa_gain_stop
1531 --stop_tilt_fraction
1532 --control_stop_tilt_fraction
1533 --energy_budget
1534 --exec_model_enabled
1535 --exec_inner_dt_sec
1536 --detail_rate_hz
1537 --exec_tau_sec
1538 --exec_accel_limit_kmhps
1539 --exec_decel_limit_kmhps
1540 --exec_deadband_kmh
1541 --exec_quantize_step_kmh
1542 --exec_reaction_delay_sec
1543 --upper_mode
1544 --upper_ds_km
1545 --upper_horizon_km
1546 --upper_max_steps
1547 --upper_horizon_mode
1548 --upper_adaptive_min_ds_km
1549 --upper_adaptive_max_ds_km
1550 --upper_adaptive_growth
1551 --upper_replan_km
1552 --upper_replan_sec
1553 --upper_max_iter
1554 --upper_global_search_enabled
1555 --upper_global_search_mode
1556 --upper_cem_generations
1557 --upper_cem_population
1558 --upper_cem_elite
1559 --upper_local_refine_topk
1560 --upper_shgo_samples
1561 --upper_shgo_iters
1562 --upper_cert_grid_levels
1563 --upper_cert_max_evaluations
1564 --upper_cert_workers
```

```
1570    --upper_ctrl_km
1571    --upper_vmin_kmh
```

8 処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

9 このファイルの位置づけ

この資料群では、scripts/solar_sim.py を **offline core** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の profile / launch / telemetry と後段の planner / logger / report へ繋がる。